

Федеральное агентство по образованию

**ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)**

Кафедра автоматизированных систем управления (АСУ)

В.М. Зюзьков

ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ

Учебное пособие

2005

Корректор: Осипова Е.А.

Зюзьков В.М.

Функциональное программирование : учебное пособие. —
Томск: Томский межвузовский центр дистанционного образова-
ния, 2005. — 140 с.

Пособие содержит курс лекций по дисциплине «Функциональное
и логическое программирование».

Для студентов, обучающихся с использованием дистанционных об-
разовательных технологий.

© Зюзьков В.М., 2005

© Томский межвузовский центр
дистанционного образования, 2005

ОГЛАВЛЕНИЕ

1 ВВЕДЕНИЕ.....	4
2 ЧТО ТАКОЕ ДЕКЛАРАТИВНОЕ ПРОГРАММИРОВАНИЕ?	5
2.1 Машина фон Неймана и процедурное программирование....	5
2.2 От программирования процедурного к декларативному	7
3 ФУНКЦИОНАЛЬНЫЙ ВЗГЛЯД НА ВЫЧИСЛЕНИЯ	10
4 ОСНОВЫ ЯЗЫКА ЛИСП	18
4.1 О языке Лисп (история и особенности).....	18
4.2 Основные структуры данных (символы, числа, списки)	20
4.3 Понятие функции.....	24
4.4 Базовые функции	27
4.5 Определение функций.....	35
4.6 Некоторые специальные формы	37
5 РЕКУРСИЯ	46
5.1 Знакомство с рекурсией	46
5.2 Встроенные рекурсивные функции для списков	53
5.3 Различные виды рекурсии	58
6 ЛАМБДА-ИСЧИСЛЕНИЕ	67
6.1 Ламбда-исчисление как формальная система	67
6.2 Ламбда-исчисление как язык программирования.....	89
7 ФУНКЦИИ ВЫСОКОГО ПОРЯДКА.....	112
8 НЕСКОЛЬКО БОЛЕЕ СЛОЖНЫХ ЗАДАЧ	131
9 ЗАКЛЮЧЕНИЕ	139
ЛИТЕРАТУРА	140

1 ВВЕДЕНИЕ

Природа любого профессионализма (в том числе и в программировании) такова, что чем он выше, тем он уже. В отличие от профессионализма, образованность — чем выше, тем шире. Высокообразованный программист — это не только безукоризненный специалист в определенной области программирования, но и человек, имеющий широкий взгляд на современное программирование независимо от языков и платформ, которые он сейчас конкретно использует.

Существует несколько методологий программирования. Методология программирования — это объединенная единым философским подходом совокупность методов, применяемых в процессе разработки программного обеспечения.

Декларативное программирование — одна из существующих парадигм программирования. Чтобы отличить императивное программирование от декларативного и оценить их пригодность для решения конкретных задач, надо иметь в голове и ту и другую методологию.

Мы будем изучать два новых языка программирования: Lisp (Лисп) и Prolog (Пролог), принципиально отличающихся от тех языков, которые вы уже знаете.

При изложении теоретического материала мы руководствуемся принципом: «лучше знать меньше, да хорошо», нежели иметь поверхностное знакомство со многими вопросами. В частности, функциональное программирование на разных языках программирования в качестве своего ядра содержит то, что называется «чистое функциональное программирование». И умение программировать на языке Лисп в чистом функциональном стиле — вот то главное, чем должен овладеть будущий программист.

2 ЧТО ТАКОЕ ДЕКЛАРАТИВНОЕ ПРОГРАММИРОВАНИЕ?

Я с легкостью сделаю на C++ за месяц то, что вы с трудом напишите на Лиспе за неделю.

Программистский фольклор

2.1 Машина фон Неймана и процедурное программирование

Материальная основа программирования — современный компьютер — есть воплощение модели, предложенной математиком фон Нейманом.

Что такое компьютер фон Неймана? Когда фон Нейман и другие задумывали его более 50 лет назад, это была изящная, практичная и объединяющая идея, которая упрощала ряд существовавших тогда инженерных и программистских задач. Хотя условия, породившие архитектуру этого компьютера, с тех пор радикально изменились, тем не менее мы по-прежнему идентифицируем понятие «компьютера» с этой концепцией пятидесятилетней давности.

В своей простейшей форме компьютер фон Неймана состоит из трех частей: центрального процессорного устройства, памяти и соединительной шины, которая может за один шаг передавать только одно слово между процессором и памятью (и посылать некий адрес в память). Можно назвать (Дж. Бэкус) эту шину «*бутылочным горлышком*» (*узким местом*) фон Неймана. В памяти размещается программа и данные, с которыми эта программа работает.

Задача программы состоит в том, чтобы неким существенным образом изменить содержимое памяти; если считать, что эта задача должна быть выполнена исключительно перекачиванием через узость фон Неймана, то становится ясной причина такого названия.

Ирония ситуации состоит в том, что большую часть потока через эту узость составляют не полезные данные, а всего лишь

имена данных, а также операции и данные, служащие лишь для вычисления таких имен. Прежде чем слово можно будет послать через шину, его адрес должен находиться в процессоре; поэтому он должен либо быть послан через шину из памяти, либо генерироваться посредством некоторой операции процессора. Если адрес посылается из памяти, то адрес этого адреса должен либо быть послан из памяти, либо генерироваться в процессоре и т.д. С другой стороны, если адрес генерируется в процессоре, он должен генерировать либо по фиксированному правилу (например, «добавить 1 к счетчику исполняемых команд»), либо по команде, которая была послана через шину, в последнем случае ее адрес нужно было прежде послать... и т.д.

Итак, программирование на традиционных языках в основном сводится к планированию и спецификации огромного потока слов через узость фон Неймана, причем большая часть этого потока состоит не из самих значащих данных, а из сведений о том, где их искать.

Обычные языки программирования в основном являются высокоуровневыми, сложными версиями компьютера фон Неймана. Такие языки называются процедурными; к ним относятся такие популярные языки, как Pascal и C (C++).

Несмотря на все свою интеллектуальную привлекательность, программирование является трудным занятием. После сорока лет многие перестают программировать под предлогом, что устали. Основной причиной такого положения является субъективное чувство «нарушения энергетического баланса» — много тратят, но мало получают. Программисты тратят много лет и усилий, чтобы научиться программировать в императивном (процедурном) стиле. Но императивный стиль программирования — стиль, в котором выполняются программы на компьютере, — совершенно не свойственен человеческой природе. Традиционное процедурное программирование приучает программиста мыслить понятиями «машины фон Неймана», а не математическими понятиями. Традиционный программист — по своим знаниям в первую очередь должен быть инженером.

Процедурное программирование наиболее пригодно для решения задач, в которых последовательное исполнение каких-либо команд является естественным. Примером здесь может служить

управление современными аппаратными средствами. Поскольку практически все современные компьютеры императивны, эта методология позволяет порождать достаточно эффективный исполняемый код. С ростом сложности задачи процедурные программы становятся все менее и менее читаемыми. Программирование и отладка действительно больших программ (например, компиляторов), написанных исключительно на основе методологии императивного программирования, может затянуться на долгие годы.

2.2 От программирования процедурного к декларативному

Процедурные языки имеют одну общую черту: базовый оператор в них — это оператор присваивания, который заставляет компьютер переместить данные из одного места в другое. Принципиально иной подход к программированию заключается в том, чтобы описывать вычисление, используя уравнения, функции, логические выводы и т.п. Особое внимание в декларативном программировании уделяется тому, **что** нужно сделать, а не тому, **как** это нужно сделать.

Декларативное программирование обычно применяется для решения тех задач, которые трудно сформулировать в терминах последовательных операций. В эту категорию попадают практически все задачи, связанные с искусственным интеллектом. Это такие задачи, как обработка естественного языка, экспертные консультирующие системы, проблемы зрительного восприятия и многие другие.

Мы рассмотрим особенности двух ветвей декларативного программирования: **функционального**, основанного на математическом понятии функции, которая не изменяет свое окружение, в отличие от функций в процедурных языках, допускающих побочные эффекты, и **логического**, в котором программы выражены в виде формул математической логики и компьютер для решения задачи пытается вывести логические следствия из них.

Попробуем более полно ответить на вопрос: «Почему следует учиться декларативному программированию?»

Сначала о том, чем определяется уровень языка программирования.

Интуитивно любой программист отличит язык программирования высокого уровня от языка программирования низкого уровня. В чем состоит различие? Чем определяется уровень? Программирование представляет собой отображение в программах объектов, понятий и явлений природной области задачи. Чем более адекватно можно выполнить это отображение, тем выше уровень языка программирования.

Декларативное программирование требует высокого уровня абстрагирования. Но повышение уровня абстрагирования — необходимое требование для программиста. Эдсгер Дейкстра (Edsger Dijkstra) [6] подчеркивал, что программист должен обладать умением абстрагировать: «Язык программирования — это лишь средство описания абстрактных конструкций. Программист должен иметь способность полностью абстрагироваться от несущественных деталей, думая на нескольких уровнях абстракции одновременно».

Декларативные языки при классификации по степени абстракции от аппаратуры относятся к языкам *сверхвысокого уровня*. Команды исполняются на полностью абстрактной машине, полностью скрыт доступ к памяти, и возможно скрыть поток управления.

Концептуальная целостность языка характеризуется свойствами совокупности понятий, служащих для описания этого языка и включает три взаимосвязанных аспекта.

- *Экономия понятий* — язык должен достигать своей максимальной мощности минимальным количеством понятий.
- *Ортогональность понятий* — между понятиями не должно быть взаимного влияния. Если понятие используется в различных контекстах, то правило его использования должно быть одним и тем же. К сожалению, даже лучшие языки, такие как Pascal, допускают не ортогональные конструкции. Например, пользователь может определить процедуры только с фиксированным числом параметров, однако некоторые стандартные процедуры (например, write) могут быть вызваны с переменным числом параметров.

- *Единообразие понятий* — требование согласованного единого подхода к описанию и использованию всех понятий.

Декларативные языки являются, как правило, концептуально целостными. Язык функционального программирования Haskell в этом отношении является стандартом.

Освоение декларативного программирования необходимо любому профессиональному программисту, и это не только потому, что повышает уровень образованности, но и также в прагматических целях — использование этих языков в некоторых достаточно широких предметных областях эффективно.

Декларативное программирование дает более фундаментальное освоение программирования и всей компьютерной науки (в первую очередь это относится к функциональному программированию).

Некоторая трудность в изучении декларативных языков по сравнению с императивным программированием является оправданной и не должна отпугивать. Дело в том, что, изучая функциональное и логическое программирование, мы действительно проникаем в самую сердцевину теории вычислимости. Причем это делаем совсем не так, как нас учили программировать.

Трудность в изучении декларативных языков в первую очередь вызвана привычкой использовать только процедурный подход. Очень трудно привыкнуть к мысли, что традиционный императивный способ программирования не единственный и, возможно, не самый лучший.

3 ФУНКЦИОНАЛЬНЫЙ ВЗГЛЯД НА ВЫЧИСЛЕНИЯ

На протяжении последних 400 лет центральным понятием математики является понятие **функции**. Математические функции выражают связь между параметрами (входом) и результатом (выходом) некоторого процесса. Так как вычисление это тоже процесс, имеющий вход и выход, функция — вполне подходящее средство задания вычислений. Программа есть формальное описание конкретного вычисления. Если мы игнорируем процедурные детали («как» вычисления делаются) и только принимаем во внимание результат («что» вычисления сделали), то мы можем рассматривать программу как «черный ящик», который получает какой-то вход и возвращает какой-то выход. Другими словами, программа может быть рассмотрена как математическая функция.

Мы можем думать о программах, процедурах и функциях в языке программирования, как если бы они представляют математическое понятие функции:

- в случае программы, x представляет вход и y представляет выход;
- в случае процедуры или функции, x представляет параметры, а y представляет возвращаемую величину.

В любом случае мы можем ссылаться на x как «вход» и на y как «выход». Поэтому функциональный взгляд на программирование не делает различий между программой, процедурой или функцией. Но он, однако, всегда различает входную и выходную величины.

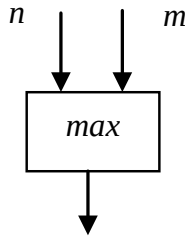
В языке программирования мы также должны различать **определение функции** и **применение функции**: первое описывает, как функция будет вычисляться, используя формальные параметры, в то время как функциональное применение есть вызов описанной функции, использующий фактические параметры или аргументы.

Рассматривая функции как черный ящик, мы можем использовать его как конструктивный блок, и с помощью объединения (когда выход одного ящика подается на вход другого) таких ящиков можно порождать более сложные операции. Такой процесс «объединения» ящиков называется **функциональной композицией**.

Пусть дана математическая функция с действительными числами $\max: \mathbf{R} \times \mathbf{R} \rightarrow \mathbf{R}$, определяющая наибольшее значение из двух чисел:

$$\begin{aligned} \max(m, n) &= m, \text{ если } m > n, \\ &= n \text{ в противном случае.} \end{aligned}$$

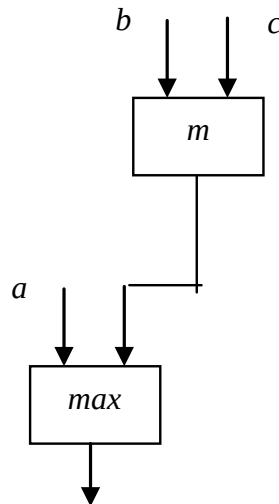
Ее представление в виде черного ящика очень просто:



Можно использовать $\max$ как блок для более сложной функции. Предположим, требуется построить функцию, которая находила бы максимум не из двух чисел, а из трех. Эту новую функцию (назовем ее $\max3$) определим следующим образом:

$$\begin{aligned} \max3(a, b, c) &= a, \text{ если } a \geq b \text{ и } a > c \text{ или } a \geq c \text{ и } a > b, \\ &= b, \text{ если } b \geq a \text{ и } b > c \text{ или } b \geq c \text{ и } b > a, \\ &= c, \text{ если } c \geq a \text{ и } c > b \text{ или } c \geq b \text{ и } c > a, \\ &= a \text{ в противном случае.} \end{aligned}$$

Это довольно неудобное определение. Гораздо более элегантный способ определения функции $\max3$ состоит в использовании уже определенной функции $\max$:



Это можно записать следующим образом:

$$\text{max3}(a,b,c) = \text{max}(a, \text{max}(b,c)).$$

Поскольку функция обеспечивает детерминированное отображение тройки чисел в число, то ее можно рассматривать как черный ящик с тремя входами и одним выходом.

Таким образом, задав множество предварительно определенных черных ящиков-функций, называемых **встроенными функциями** или **примитивами**, для выполнения простых операций, подобных базовым арифметическим, можно построить новые функции, т.е. новые черные ящики для выполнения более сложных операций с помощью этих примитивов. Далее, эти новые функции можно использовать как блоки для построения еще более сложных функций и т.д.

Программа на функциональном языке состоит из множества определений функций и выражения, чья величина рассматривается как результат программы. Математической моделью функциональных языков является ламбда-исчисление. Примерами функциональных языков являются Clean¹, Haskell и др. Многие другие языки, такие, как Лисп, имеют чисто функциональное подмножество, но также содержат нефункциональные конструкции.

Чтобы понять решение на императивном языке, мы должны понять, что машина будет делать при выполнении каждого оператора программы. При функциональном решении, с другой стороны, мы не должны думать о том, как программа будет выполняться на компьютере; отсутствует всякое упоминание об изменяемом состоянии программы или о последовательном выполнении инструкций. Функциональное решение является фактически формулировкой самой задачи, а не рецептом ее решения, и именно в этом смысле мы говорим о функциональной программе как о спецификации того, что нужно сделать, вместо последовательности инструкций, описывающих, как это сделать. Таким образом, функциональное программирование является одним из видов декларативного программирования.

Функциональное программирование, по сравнению с другими парадигмами программирования, предлагает наиболее точный изоморфизм между постановкой задачи и программой.

¹ Clean: <http://www.cs.kun.nl/~clean>

В функциональных языках все вычисления выполняются через оценку выражений, чтобы выдать какие-то величины.

В математике переменные всегда представляют какие-то конкретные величины, в то время как в императивном программировании переменные ссылаются на конкретное место в памяти и так же на величину. В математике нет понятия «место в памяти», так что выражение $x = x + 1$ не имеет смысла. Функциональный взгляд на программирование поэтому должен уничтожить понятие переменной, иное чем имя величины. Точно также исчезает присваивание как допустимый оператор.

Этот взгляд на функциональное программирование приводит к понятию **«чистое функциональное программирование»**. Большинство функциональных языков, в том числе и Лисп, оставляет некоторое понятие переменной и присваивания, и поэтому «не чисты», но все же возможно программировать эффективно, используя чистый подход. Язык Haskell, например, является чистым функциональным языком.

Одним из следствий отсутствия переменных и присваивания является невозможность циклов: цикл должен иметь управляющую переменную, которая должна менять свое значение во время выполнения цикла, но это невозможно без переменных и присваиваний. Вместо этого мы должны использовать рекурсию.

Фундаментальное свойство математических функций, которое дает нам возможность собрать воедино черные ящики, — это **функциональность (прозрачность по ссылкам)**.

Принцип *прозрачности по ссылкам* утверждает: «Значение целого не меняется, когда какая-то часть целого заменяется на равную часть». Выражение E является прозрачным по ссылкам, если любое подвыражение и его величина (как результат его вычисления) могут быть взаимно заменены без изменения величины E .

Это означает, что каждое выражение определяет единственную величину, которую нельзя изменить ни путем ее вычисления, ни предоставлением различным частям программы возможности совместно использовать это выражение. Вычисление выражения просто изменяет форму выражения, но не изменяет его величину. Все ссылки на некоторую величину эквивалентны самой этой величине, и тот факт, что на выражение можно ссылаться из другой части программы, никак не влияет на величину этого выражения.

Из-за отсутствия переменных и присваиваний в языке отсутствует понятие состояния функции. В императивном программировании два главных фактора действуют на внутреннее состояние процедуры:

- предыдущие вычисления (предшествующие вызову процедуры), включающие побочный эффект от предыдущих вызовов самой процедуры;
- порядок оценки параметров при вызове.

В императивных языках (например, Паскаль) функции могут ссылаться на глобальные данные и разрешается применять деструктивное (разрушающее) присваивание, что может привести к изменению значения функции при повторном ее вызове. Такие динамические изменения в величине данных часто именуются **побочными эффектами**. Благодаря им значение функции может изменяться, даже если ее аргументы и остаются без изменения всякий раз, когда к ней обращаются.

Пример нефункциональности Паскаля:

```
var flag: boolean;

function f (n:integer):integer;
begin
    if flag then f:=n
                else f:= 2*n;
    flag:=not flag
end;

begin
    flag:=true;
    writeln(f(1)+f(2));      {будет напечатано 5}
    writeln(f(2)+f(1));      {будет напечатано 4}
end.
```

Функция f не является математической («чистой») функцией. Операции, подобные этой, в математике не разрешены, поскольку математические рассуждения базируются на идее равенства и возможности замены одного выражения другим, означающим то же самое, т.е. определяющим ту же величину.

В функциональном программировании, однако, значение функции зависит только от значений ее параметров, но не от предыдущих вычислений, включая и вызовы самой функции.

Величина любой функции также не может зависеть от порядка вычисления ее параметров (тем самым функциональные языки становятся подходящими для параллельных приложений).

Прозрачная по ссылкам функция без параметров должна всегда выдавать тот же самый результат и поэтому ничем не отличается от константы. Более того, различные вызовы одной и той же функции с теми же аргументами будут всегда возвращать один и тот же результат.

В функциональном программировании имеется возможность оперировать функциями всевозможными способами, без ограничений. В частности функция сама по себе может рассматриваться как структура данных, которая может быть аргументом другой функции и которая может возвращаться как результат вычисления функции.

Функциональные языки все в большей и большей степени используются в образовании, потому что они способствуют выражать концепции и вычислительные структуры программирования на высоком уровне абстракции. Появление языка Haskell, который стал де-факто стандартом современного функционального программирования, вновь вызвало интерес к функциональным языкам.

Многие зарубежные университеты в области компьютерных наук используют функциональное программирование в качестве основы курса для тех, кто углубленно изучает программирование, и на нескольких факультетах в качестве первого языка программирования студенты изучают функциональный язык.

Создание больших программных систем очень дорого и трудно. Еще более трудоемко и дорогостояще сопровождение таких систем. Для преодоления кризиса в программировании возникла методология объектно-ориентированного программирования. Важно отметить, что функциональное программирование, в частности язык Лисп, одно из первых реализовало существенные свойства методологии объектно-ориентированного программирования: инкапсуляцию, наследование и полиморфизм.

Функциональные языки позволяют разрабатывать и сопровождать программные системы быстро и дешево.

Мы будем изучать функциональное программирование на примере языка Лисп. Это вызвано следующими обстоятельствами:

- из всех функциональных языков Лисп наиболее известен и широко распространен;

- имеется хорошая учебная литература по Лиспу²;
- используя всего несколько встроенных функций Лиспа, можно достаточно хорошо научиться программировать в чисто функциональном стиле;
- в рамках обучения с использованием дистанционных технологий обучение программированию на Лиспе не требует консультирования с преподавателем.

Элегантность и красота Лиспа часто не заметна неопытному взгляду, поскольку большинство Лисп-приложений включает множество дополнительных операторов, собственно к функциональному программированию не относящихся. Современные диалекты Лиспа в буквальном смысле задыхаются от программных конструкций, заимствованных из языка Fortran, некоторые из которых оказались полезными, а другие просто вызывают отвращение (справедливости ради нужно отметить, что некоторые обладают и тем и другим).

Главное в предмете функционального программирования — это научиться программировать в «чисто» функциональном стиле. Этому можно научиться и программируя на Лиспе. Но приходится «держаться в ежовых рукавицах» — так и тянет использовать привычные процедурные конструкции (присваивания, циклы и др.).³ Мы будем изучать только «чистое» ядро Лиспа.

Функциональное программирование основано на элегантной математической теории — ламбда-исчислении. Есть ли необходимость изучать ламбда-исчисление, чтобы научиться функционально программировать? Несомненно, да! Приведем мнение российского логика и программиста Н.Н. Непейводы [7]: «Можно сформулировать неформальный принцип. *Чем эффективнее составленная программа, тем сложнее может быть ее обоснование и тем более абстрактных понятий оно может потребовать.*

² Очень хорош, например, учебник [9].

³ Программируя на Лиспе, программист может, как уже говорилось выше, использовать привычную технику процедурного (императивного) программирования. Но этот путь не приведет к личностному росту. Стоит ли изучать новый язык, если программировать по-старому?! Освоение функциональной парадигмы поможет взглянуть на привычные вещи по-новому и понять, что можно программировать и по-другому и, по-видимому, с большей эффективностью.

Так что те, кто утверждает, что нужно пользоваться лишь реальными понятиями и изгонять идеальные, абстрактные, рискуют проиграть не только в сложности и глубине рассуждений, но и в силе реальных методов».

В дальнейшем, занимаясь профессиональным программированием, можно перейти и к более перспективным, на наш взгляд, функциональным языкам. В первую очередь отметим язык Haskell.⁴ Для тех, кому понравится функциональное программирование, но программировать приходится только при решении прикладных задач (с использованием математического аппарата), можно рекомендовать систему Mathematica (последняя версия 5.2) — это не только пакет по математике (самый лучший!), но и мощный язык, на котором можно программировать в чисто функциональном стиле.⁵ Сайт системы Mathematica: www.wolfram.com. В продаже имеются лазерные диски с этой системой.

⁴ Web-сайт языка Haskell (www.haskell.org) дает много полезных ресурсов, включающих: определение языка Haskell и библиотек; обучающий материал по программированию на языке Haskell; свободно распространяемые реализации языка (для некоммерческого использования); приложения, написанные на языке Haskell.

⁵ Функциональное программирование — это программирование для математиков.

4 ОСНОВЫ ЯЗЫКА ЛИСП

4.1 О языке Лисп (история и особенности)

Лисп (LISP) был первым чисто функциональным языком. Он был разработан Джоном Маккарти в 1961 году. Хотя оригинальный Лисп был чисто функциональным в смысле прозрачности ссылок, появившиеся в последующие годы диалекты включили в себя многие императивные особенности. Существует, однако, «**чистое**» подмножество языка Лисп, встроенное во все диалекты, которое само по себе можно использовать для написания функциональных программ в нашем понимании этого термина. Хотя различные реализации Лиспа отличаются довольно значительно по выбору ключевых слов, имен примитивных функций и структуре программы в целом, в их основе лежат одни и те же принципы.

Мы будем строить наше изложение на синтаксисе диалекта Лиспа, называемого XLisp (David Betz, Великобритания, 1988), практически согласованного со стандартом Common Lisp.

Символьная обработка. Традиционные машины (фон Неймана) и языки программирования первоначально разрабатывались для осуществления численных вычислений и обработки больших объемов данных и не подходят столь же хорошо для символьной обработки, которая имеет дело со сложными структурами данных и базами знаний, содержащими правила принятия решений и другие многообразные объекты. Символьная обработка позволяет эффективно работать с такими структурами, как предложения естественного языка, значения слов и предложений, нечеткие понятия и т.д., и на их основе принимать решения, проводить рассуждения и осуществлять другие, свойственные человеку способы обращения с данными.

В этих применениях предполагается представление в подходящей форме символьных и сложно структурированных данных, и работа с ними часто ведется в заранее непредсказуемых ситуациях.

Лисп является наиболее важным языком программирования, используемым в исследованиях по искусственному интеллекту

и в математической лингвистике. Название языка «Лисп» происходит из «listing processing» (обработка списков).

Лисп опередил свое время. Обычно языки программирования не изобретают, а проектируют. Однако по отношению к Лиспу можно говорить об изобретении. Первоначальная версия языка, в частности, содержала множество понятий и принципов, которые казались очень странными, но многие из которых позже оказались существенным нововведением. Кроме этого, возможность добавления в Лисп в течение десятилетий многих новых черт подтвердила свойство расширяемости этого языка. Вокруг Лиспа возникла широкая культура, охватывающая различные школы и разнообразные диалекты языка, различные системы и методы программирования, программные среды и Лисп-машины.

Одинаковая форма данных и программы. И то и другое представляется списочной структурой, имеющей одинаковую форму. Таким образом, программы могут обрабатывать и преобразовывать другие программы и даже самих себя. Можно введенное и сформированное в результате вычислений выражение данных проинтерпретировать в качестве программы и непосредственно выполнить (так называемое **программирование, управляемое данными**). Универсальный единообразный и простой лисповский синтаксис списка не зависит от применения, и с его помощью легко определять новые формы записи, представления и абстракции. Даже сама структура языка является, таким образом, расширяемой и может быть заново определена. В то же время достаточно просто написание интерпретаторов, компиляторов, преобразователей, редакторов и других средств.

Хранение данных, не зависящее от места. Списки, представляющие программы и данные, состоят из списочных ячеек, расположение и порядок которых в памяти не существенны. Структура списка определяется логически на основе имен символов и указателей.

Автоматическое и динамическое управление памятью. Пользователь не должен заботиться об учете памяти. Система резервирует и освобождает память автоматически в соответствии с потребностью. Когда память кончается, запускается специальный мусорщик. Он собирает неиспользуемые символы и списки,

включает их в работу путем вторичного использования. Среда Лиспа постоянно содержится в порядке.

Лисп — безтиповой язык программирования. В Лиспе имена символов, переменных, списков, функций и других объектов не закреплены предварительно за какими-нибудь типами данных. Типы, в общем, не связаны с именами объектов данных, а сопровождают сами объекты. Таким образом, переменные могут в различные моменты времени представлять различные объекты разных типов.

Заблуждения и предрассудки. Ощущение трудности и необычности составления функциональных программ вполне законно и естественно. Мы — жертвы императивного программирования и машин фон Неймана. Естественно, используемые в программировании на Лиспе структуры данных и управляющие структуры часто сложны, поскольку проблемы искусственного интеллекта из-за своей сложности предполагают сложные структуры и программы. Иерархические списочные структуры и Лисп как раз и задумывались для работы со сложными проблемами. Искусственное упрощение структур означало бы пренебрежение действительной сложностью проблем.

4.2 Основные структуры данных (символы, числа, списки)

Символы используются для представления других объектов. В языке Лисп используются **символы** и построенные из них **символьные структуры**. Символ — это имя, состоящее из букв, цифр и специальных знаков, которые обозначают какой-нибудь предмет, объект, вещь или действие из реального мира. В Лиспе символы обозначают числа, другие символы или более сложные структуры, функции и другие лисповские объекты.

Символы в языке Common Lisp. Символы могут состоять из букв, цифр и некоторых других знаков, а именно

+ - * / @ \$ % ^ & _ > < . (точка) ? ! { } [] :

В состав символов можно включать пробел, запятую, точку с запятой, круглые скобки. Для этого либо символ с двух сторон ограничивается вертикальной чертой (|), либо перед каждым

таким специальным знаком, входящим в символ, ставится обратная косая черта (\).

Русские буквы в XLisp не используются.

Символы могут состоять из строчных и прописных латинских букв, которые отождествляются и представляются прописными буквами.

Числа являются константами. В языке используются числа в обычной записи. Числа не являются символами, так как число не может представлять иные лисповские объекты, кроме самого себя, т.е. своего числового значения. Примеры чисел различных типов: 56 (целое число), 67.89 (вещественное число с фиксированной точкой), 56.45E-3 (вещественное число в научной нотации), 5/8 (рациональное число).

Логические значения T и NIL. Символы T и NIL имеют специальное назначение: T обозначает логическое значение «истина», NIL обозначает «ложь». Символы T и NIL всегда имеют одно и то же фиксированное встроенное значение. Их нельзя использовать в качестве имен других лисповских объектов.

Важное замечание: значение «истина» также изображается любым объектом Лиспа, отличным от символа NIL.

Константы и переменные. Числа и логические значения T и NIL являются константами, остальные символы — переменными, которые используются для обозначения других лисповских объектов.

Атомы = символы + числа. Символы и числа представляют собой те простейшие объекты Лиспа, из которых строятся остальные структуры. Они называются атомарными объектами или атомами.

Построение списков из атомов и подсписков. Список в Лиспе — упорядоченная последовательность, элементами которой являются атомы или списки (подсписки). Списки заключаются в круглые скобки, элементы списка разделяются пробелами. Список всегда начинается с открывающей скобки и заканчивается закрывающей скобкой.

Пример

(56 g (a (b) s) 1/2 + ())

Это список из шести элементов; третий элемент (a (b) s) сам является списком из трех элементов, причем (b) — список из одного элемента — символа b.

Список — это многоуровневая структура данных, в которой открывающие и закрывающие скобки должны быть в строгом соответствии (сбалансированы).

Пустой список. Список, в котором нет ни одного элемента, называется пустым и обозначается «()» или NIL. Пустой список — это не то же самое, что «ничего». Он выполняет ту же роль, что и нуль в арифметике. NIL может быть, например, элементом других списков:

NIL ; то же самое, что и ()
 (NIL) ; список, состоящий из атома NIL
 (()) ; то же, что и (NIL)
 ((())) ; то же, что и ((NIL))
 (NIL ()) ; список из двух пустых списков

Атомы и списки называются **символьными выражениями**.

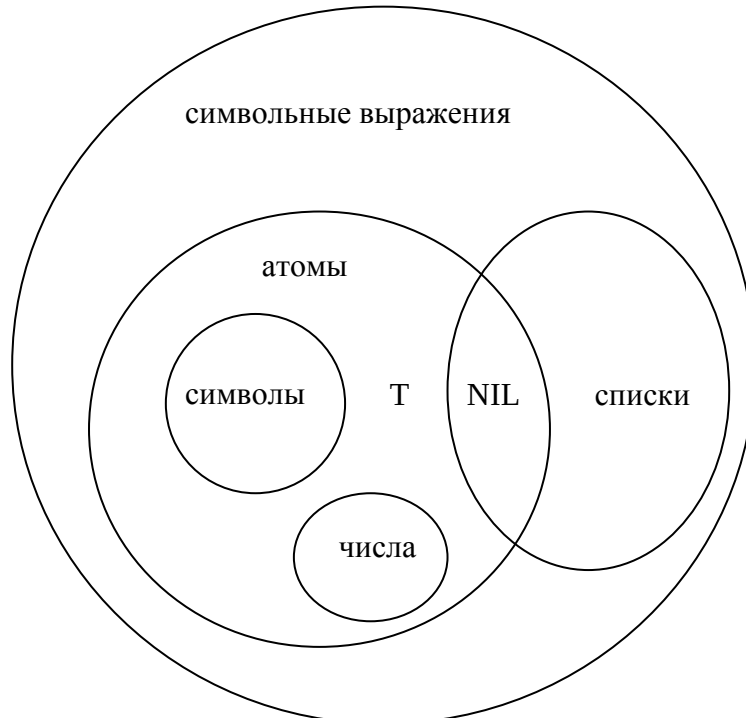


Рис. 1 — Символьные выражения

Список как средство представления знания. Списки можно использовать для представления всевозможных знаний. Например, информацию о семье можно представлять списком с тремя элементами: муж, жена и дети. Поскольку в разных семьях имеется различное количество детей, информацию о детях естественно представить в виде списка, который позволяет включать в него любое количество элементов. Информация о каждом человеке, в свою очередь, представляется в виде списка, состоящего из компонентов: имя, пол, возраст (для детей также указывается пол). Для удобства чтения информации о семье в подсписки можно добавить элементы — метки (husband — муж, wife — жена, children — дети):

((husband Tom 45)

(wife Ann 44)

(children (Pat woman 22)

(Bob man 18)))

Запись в виде списка гибка, свободна по форме и одновременно достаточно точна и понятна. Как видно из примера, такая запись с раскрывающим структуру расположением текста помогает получить общее представление о структуре и вникнуть в нее. При вводе выражений в Лисп-систему пустые строки, пробелы и расположение текста игнорируются.

Планирование способа представления данных и знаний и решение проблемы их изображения являются важнейшими моментами при программировании на языке Лисп. Речь идет о нахождении способа записи, который облегчал программирование и был естественен как для описания задачи, так и для работы с данными. Часто удается предложить форму записи, в какой-то мере аналогичную самой задаче и позволяющую упростить дальнейшую работу. Програмируя на языке Лисп, задачу обычно пробуют разбить на более простые составляющие. В структуре данных пытаются максимально отобразить все, что известно о задаче, и, таким образом, упростить программирование.

Различная интерпретация списков. Структурные данные и программы в Лиспе представляются в виде списков. Например, список (+ 3 4) можно интерпретировать в зависимости от окружения и использования, либо как действие, дающее в результате число 7, либо как список, состоящий из трех элементов.

4.3 Понятие функции

В языке Лисп имеется множество встроенных функций (которые уже определены), но, кроме того, программист может определить свои собственные функции. В данной версии Лиспа новые определения функций помещаются в текстовый файл, который должен быть загружен Лисп-системой, чтобы новые функции можно было использовать.

Мы будем использовать и определять только те функции Лиспа, которые являются «чистыми», т.е. не имеют побочных эффектов и значение функции зависит только от значений ее параметров. Таким образом, будем учиться «чистому» функциональному программированию.

Единообразная префиксная нотация. В математике и обычных языках программирования вызов функции записывается, как правило, в так называемой префиксной записи, в которой имя функции стоит перед скобками, окружающими аргументы:

$f(x)$, $g(x,y)$, $h(x, g(y,z))$ и т.д.

В арифметических выражениях используется инфиксная запись, в которой имя функции или действия располагается между аргументами:

$x+y$, $x-y$, $x*(y+z)$ и т.д.

В языке Лисп как для вызова функций, так и для записи выражений принята единообразная префиксная форма записи, при которой как имя функции или действия, так и сами аргументы записываются внутри скобок:

$(f\ x)$, $(g\ x\ y)$, $(h\ x\ (g\ y\ z))$ и т.д.

Таким же образом записываются арифметические действия:

$(+ x\ y)$, $(* x (+ y\ z))$ и т.д.

Здесь символы «+», «-» и «*» являются именами функций, осуществляющих сложение, вычитание и умножение чисел.

Сначала этот новый способ записи покажется затруднительным, но он весьма быстро станет привычным и даже понравится по мере того, как станут очевидны его преимущества.

Единообразная и простая структура вызова функции удобна как для программиста, так и для вычисляющего значения выражений интерпретатора Лиспа. Вычисляя значение функций, не нужно осуществлять сложный синтаксический анализ выражений, так как сразу по первому символу текущего выражения система уже знает, с какой структурой имеет дело и как её интерпретировать или как с ней обращаться.

Диалог с интерпретатором Лиспа. Лисп обычно используется в диалоге (интерактивно) в режиме интерпретации.

Обычный цикл работы Лиспа:

- приглашение интерпретатора для ввода >,
- пользователь вводит выражение,
- интерпретатор вычисляет выражение и выдает значение этого выражения.

Пример

```
> (+ 5 6)
11
> nil
NIL
> 600
600
```

Иерархия вызовов. В конструкцию вводимой функции могут, в свою очередь, входить функциональные подвыражения. Тогда аргументы вычисляемой функции заменяются новыми вычислениями, ведущими к определению значений этих аргументов. Таким путем вычисления сводятся, в конечном счете, к вычислению выражений, значения которых как, например, констант можно определить непосредственно.

Вычисляя значения выражения, интерпретатор Лиспа сначала пытается слева направо вычислить значения аргументов внешнего

вызова. Если первый аргумент является константой или иным объектом, значение которого можно определить непосредственно, то вычисление аргументов вычисляемой функции можно продолжить. Если аргументом будет вложенный вызов функции, то перед ее вычислением определяются значения ее аргументов, и т.д.

Наконец, опустившись на самый нижний уровень, можно начать в качестве значений аргументов предыдущего уровня подставлять значения, даваемые вложенными вычислениями. Последним возвращается значение всего выражения. Как и в математике, в Лиспе в первую очередь вычисляются выражения, заключенные в скобки:

> (* (+ 1 2) (+ 3 4))	
21	; = (1+2)*(3+4)
> (* (+ 1 2) (* 1 (+ 2 3)))	
15	; = (1+2)*(1*(2+3))

QUOTE блокирует вычисление выражения. В некоторых случаях не надо вычислять значение выражения, а нужно само выражение. Нас, например, может не интересовать значение функционального вызова (+ 2 3), равное 5, а мы хотим обрабатывать форму (+ 2 3) как список. В таких случаях используют специальную функцию QUOTE с одним аргументом, которая ничего с ним не делает, даже не вычисляет его значение, а возвращает в качестве значения вызова сам этот аргумент:

```
> (quote (+ 2 3))
(+ 2 3)
```

В качестве краткого обозначения для функции QUOTE используют апостроф перед аргументом:

```
'<выражение>
```

Поэтому

```
> '(+ 2 '(* 3 5))
(+ 2 (quote (* 3 5)))
```

Числа и константы T и NIL не надо предварять апострофом, так как в этом случае интерпретатор считает, что данные атомы и их значения совпадают.

В приведенных выше примерах на месте имени функции в функциональном вызове апостроф не использовался. В данной версии Лиспа не допускается использование в качестве первого элемента вызова функции вычисляемого выражения:

Примеры

```
> (+ 4 5)
error: bad function – (quote +)
> (2 3 4)
error: bad function – 2
```

В обоих случаях интерпретатор, в соответствии со своим алгоритмом, ожидает после открывающей скобки имя функции, но вопреки правилам, там обнаруживается либо не атомарное выражение, либо число, — и то и другое не являются именами функций.

4.4 Базовые функции

Большинство Лисп-программ можно записать, используя только пять простейших функций над символическими выражениями и одну специальную форму (условное выражение):

- полиморфный предикат равенства (`equal X Y`): $S \times S \rightarrow \{T, NIL\}$, где S — множество символьных выражений;
- предикат, проверяющий, является ли символическое выражение X атомом
(`atom X`): $S \rightarrow \{T, NIL\}$;
- функция (`first X`): $S \rightarrow S$, извлекающая первый элемент списка X ;
- функция (`rest X`): $S \rightarrow S$, возвращающая остальные элементы списка X , кроме первого;

- функция $(\text{cons } X \ Y): S \rightarrow S$, которая создает новый список, добавляя элемент X в начало списка Y .

Совокупности операции композиции функций и условного выражения $(\text{if } \dots)$ вполне достаточно для того, чтобы вычислить любую обобщенную рекурсивную функцию.

Проверка типов аргументов функции происходит динамически, во время выполнения.

First, rest, cons — базовые функции для построения, разбора и анализа списков.

Рекурсивно список в Лиспе можно определить так:

- список может быть пустым или
- если список не пуст, то он состоит из **головы** (это первый элемент) и **хвоста** (это список, образованный из всех оставшихся элементов исходного списка).

Функция $(\text{first } \langle \text{список} \rangle)$ вычисляет в качестве значения голову списка:

```
> (first '(a b c))
A
> (first '((1 2) (3 4) 5))
(1 2)
> (first nil)
NIL
```

Если применить функцию `first` к атому (отличному от `nil`), то выдается сообщение об ошибке — «аргумент не список»:

```
> (first t)
error: bad argument type – T
```

Функция $(\text{rest } \langle \text{список} \rangle)$ вычисляет в качестве значения хвост списка:

```

> (rest '(a b c))
(B C)
> (rest '((1 2) (3 4) 5))
((3 4) 5)
> (rest nil)
NIL

```

Если применить функцию `rest` к атому (отличному от `nil`), то выдается сообщение об ошибке — «аргумент не список»:

```

> (rest t)
error: bad argument type – T

```

Традиционно (в связи с первоначальной реализацией Лиспа) для функций `first` и `rest` используются также обозначения `car` и `cdr`:

```

> (car '(a b c))
A
> (cdr '(a b c))
(B C)

```

Функция `cons` (`construct`) строит новый список из переданных в качестве аргументов головы и хвоста:

```

> (cons 'a '(b c))
(A B C)
> (cons '(a b) '(c d))
((A B) C D)
> (cons (+ 1 2) '(+ 4 6))
(3 + 4 6)
> (cons nil nil)
(NIL)
> (cons '(a b c) nil)
((A B C))
> (cons nil '(a b c))
(NIL A B C)

```

Очевидна связь между функциями `first`, `rest` и `cons`: `(cons (first s) (rest s)) = s`:

```
> (cons (first '(a b c)) (rest '(a b c)))
(A B C)
```

Комбинируя селекторы `car=first` и `cdr=rest`, можно выделить произвольный элемент списка. Например,

```
> (cdr (cdr (car '((a b c) (d e) (f)))))
(C)
```

Вложенные вызовы `car` и `cdr` можно записывать в сокращенном виде в виде одного вызова функции: `(c...r список)` — вместо точек допускается не более четырех символов:

```
(cadr x) ⇔ (car (cdr x))
(cddar x) ⇔ (cdr (cdr (car x)))
```

Предикаты — функции, которые проверяют выполнение некоторого условия и возвращают в качестве результата логическое значение `T` («истина») или `NIL` («ложь»)⁶.

Функция `(atom <выражение>)` проверяет, является ли `<выражение>` атомом:

```
> (atom '(a b c))
NIL
> (atom 7)
T
> (atom nil)
T
> (atom (atom (+ 3 4)))
T
```

⁶ Важно запомнить, что, во-первых, символ `NIL` в Лиспе представляет как пустой список, так и булевское значение «ложь» и, во-вторых, любое символьное выражение, отличное от `NIL`, также может служить для представления логического значения «ложь».

Функция (`eq <символ1> <символ2>`) проверяет тождественность двух символов:

```
> (eq t (atom 'atom))
T
(eq nil t)
NIL
```

Функция (`= <число1> <число2>`) проверяет равенство чисел (количество чисел может быть больше двух):

```
> (= 5 6)
NIL
> (= 5.0 5)
T
> (= 1/2 2/4)
T
> (= 2 4/2 2.00 0.2e1)
T
```

Как видите, предикат `=` может сравнивать числа разных типов; в свою очередь, предикат `eq` сравнивает числа только одинакового типа, но, кроме того, этот предикат может сравнивать символы. Если же один из аргументов является числом, а другой символом, то предикат выдает `NIL`.

```
> (eq 50e-1 5.0)
T
> (eq 5.0e3 5000)
NIL
> (eq 'abc 'ab)
NIL
> (eq 'a 'a)
T
> (eq 5 'a)
NIL
```

Таким образом, этот предикат сравнивает атомы на совпадение (за исключением вещественных чисел — их изображение может отличаться, но предикат выдает Т). Предикат используется во многих встроенных функциях, осуществляющих более сложные операции сравнения. *Его использование для сравнения списков — типичная ошибка.*

Предикат `equal` работает как `eq`, но, кроме того, проверяет одинаковость двух списков:

```
> (eq ' (a b) ' (a b))
NIL
> (equal ' (a b) ' (a b))
T
> (equal ' (a b) ' (a c))
NIL
> (equal ' (a b) ' a)
NIL
```

Условное предложение `if`. Для реализации ветвления в Лиспе можно использовать специальную форму (`if` <логическое выражение> <выражение-then> <выражение-else>). Эта форма не является обычной функцией Лиспа, поскольку вычисление ее значения никогда не требует вычисления всех трех аргументов. Вычисление условного предложения `if` сводится к вычислению логического выражения `i`, в зависимости от истинностного значения последнего, вычисляется выражение-then или выражение-else.

```
> (if (atom 5) (+ 2 3) (+ 20 30))
5
> (if (= 3 5) (+ 2 3) (+ 20 30))
50
> (if 5 (+ 2 3) (+ 20 30))
5
```

В последнем примере не надо забывать, что любое выражение, значение которого не NIL, воспринимается в Лиспе как истинное.

Форма `if` может использоваться без выражения-else; в этом случае подразумевается, что значение выражения-else есть `NIL`.

```
> (if (= 2 3) 1)
NIL
```

Другие примитивы. Несмотря на то, что обычную обработку списков всегда можно свести к описанным ранее трем базовым функциям (`cons`, `first`, `rest`), двум базовым предикатам (`atom`, `equal`) и условному предложению `if`, программирование лишь с их помощью было бы очень громоздким и трудоемким. Поэтому в язык включено множество встроенных функций, в том числе и некоторые примитивы.

Функция (`null <выражение>`) выдает `T`, если значение выражения есть `NIL`.

```
> (null '(3 4 5))
NIL
> (null ())
T
> (null 'a)
NIL
> (null (atom 'b)))
NIL
> (null (atom '(a b c))))
T
```

Функция (`list x1 x2 x3 ...`) возвращает в качестве своего значения список из значений аргументов:

```
> (list 1 2)
(1 2)
> (list 'a '(b c) 'd)
(A (B C) D)
> (list (list 'a) 'b nil)
((A) B NIL)
> (list nil)
(NIL)
```

Функция `listp` выдает булевское значение, определяя, является ли аргумент списком или нет:

```
> (listp 5)
NIL
> (listp '(1 2 3))
T
```

Для чисел используются также **операции сравнения** `/=`, `>`, `>=`, `<`, `<=` с очевидной семантикой.

Логические операции над булевскими значениями задаются функциями `not` (отрицание), `and` (конъюнкция) и `or` (дизъюнкция).

Арифметические функции. В Лиспе с целыми или действительными числами могут использоваться различные арифметические операции, причем количество аргументов у некоторых функций может быть разное.

- **(+ x1 x2 ... xn)** возвращает $x1 + x2 + x3 + \dots + xn$.
- **(-x1 x2 ... xn)** возвращает $x1 - x2 - x3 - \dots - xn$.
- **(* y1 y2 ... yn)** возвращает $y1 \times y2 \times y3 \times \dots \times yn$.
- **(/ x1 x2 ... xn)** возвращает $x1/x2/\dots/xn$.
- Специальные функции для прибавления и вычитания единицы: $(1+ x)$ и $(1 - x)$.

```
> (* 4 5 6)
120
> (- 1 2 3)
-4
```

- **(sqrt x)** возвращает квадратный корень $\sqrt{x}$.
- **(mod m n)** вычисляет остаток от деления целого числа m на целое число n .
- **(truncate n)** отбрасывает дробную часть n .
- **(numberp n)** выдает булевское значение, определяя, является ли аргумент числом.
- **(zerop n)** булевское значение, определяя, является ли аргумент нулем.

```

> (mod 5 2)
1
> (truncate 4.7)
4
> (numberp 6.7)
T
> (zerop 8)
NIL

```

4.5 Определение функций

Программирование на Лисп сводится к определению необходимых функций, а выполнение программы — к вызову функции с нужными аргументами.

Чтобы определить функцию, необходимо:

- дать имя функции;
- определить формальные параметры функции;
- определить, что должна делать функция.

Для задания новых функций в Лиспе используется специальная форма **defun**.

```
(defun <имя-функции> <параметры> <тело-функции>)
```

Синтаксически <имя-функции> является атомом, <параметры> есть список атомов — аргументов функции, <тело-функции> — символьное выражение, как правило, содержащее параметры, и значение которого при замене формальных параметров на фактические является значением функции.

Пусть нам требуется написать функцию `insert-plus`, которая для двух символьных выражений `s1` и `s2` создает список `(s1 + s2)`. Функцию можно определить так:

```
(defun insert-plus (s1 s2) (list s1 '+ s2))
```

Определение такой функции можно сделать в диалоге с интерпретатором XLisp'a:

```
> (defun insert-plus (s1 s2) (list s1 '+ s2))
INSERT-PLUS
```

Как видим, значением определения функции `defun` является имя функции (в данном случае — `insert-plus`).

Более практично программу для решения какой-либо задачи оформлять в виде определений необходимых функций и помещать в текстовый файл (с расширением **lsp**). При наборе программы воспользуйтесь каким-либо внешним текстовым редактором (XLisp не имеет собственного редактора). Например, вы можете пользоваться стандартной программой Notepad («блокнот»). Набранные программы сохраняйте в виде текстовых файлов с расширением `lsp` в рабочем каталоге. По умолчанию рабочим каталогом будет `XLisp\bin`, но вы можете это изменить: для этого создайте ярлык для `XLisp.exe` и измените рабочий каталог в свойствах этого ярлыка. Загрузить программу, находясь в среде интерпретатора, вы можете, если выберете пункт меню `File | Load Lisp source...`. Отметим следующую особенность работы в Лиспе с блокнотом. После набора текста программы обязательно перейдите курсором на новую строку, иначе XLisp при загрузке файла с программой выдаст ошибку «неожиданный конец файла».

После загрузки программы интерпретатор Лиспа будет уже «знать» определения новых функций, и вы можете вызвать эти функции с какими-то фактическими параметрами и получить результат, например:

```
> (insert-plus 5 7)
(5 + 7)

> (insert-plus 'a (insert-plus 5 7))
(A + (5 + 7))

> (insert-plus 'a '(insert-plus 5 7))
(A + (INSERT-PLUS 5 7))
```

В Лиспе передача параметров производится в функцию по **значению**, т.е. формальный параметр в функции связывается с тем же значением, что и значение фактического параметра.

Изменение значения формального параметра не оказывает влияния на значения фактических параметров. После вычисления функции созданные на это время связи параметров ликвидируются и происходит возврат к тому состоянию, которое было до вызова функции. Параметры функции являются **локальными переменными** и имеют значение только внутри функции.

Тело функции должно быть вычисляемым символьным выражением. Такие выражения в Лиспе называются формами.

Форма — символьное выражение, значение которого может быть вычислено интерпретатором:

- константы;
- символы, которые используются в качестве переменных⁷;
- определения функций и вызовы функций;
- специальные формы (QUOTE, IF и другие).

В языке Лисп можно определять анонимные функции, т.е. не давая им имена. Мы вернемся к этому вопросу после изучения лямбда-исчисления.

4.6 Некоторые специальные формы

Имя и значение символа. Символы в Лиспе можно использовать как переменные. В этом случае они могут обозначать некоторые выражения. У символов изначально нет какого-нибудь значения, как у констант. Попытка вычислить символ приводит к ошибке:

```
> a
error: unbound variable – A
```

При помощи формы **set** символу можно присвоить (set) или связать (binding) с ним некоторое значение. set вычисляет оба аргумента.

⁷ О таких символах речь пойдет в следующем разделе.

```
> (set 'a '(1 2 3))
(1 2 3)
```

```
> a
(1 2 3)
```

Форма **setq** отличается от **set** тем, что она вычисляет только свой второй аргумент. Об автоматическом блокировании вычисления первого аргумента напоминает буква **q**.

```
> (setq b a)
(1 2 3)
```

```
> b
(1 2 3)
```

Таким образом, эти формы имеют побочный эффект. Эффект формы состоит в образовании связи между символом и его значением, а значением формы является связываемое значение.

В Лиспе все действия возвращает некоторое значение. Значение имеется даже у таких действий, основное предназначение которых заключается в осуществлении побочного эффекта, такого, например, как связывание символа или вывод результатов на печать. Формы с побочным эффектом в Лиспе можно рассматривать как псевдофункции, поскольку они всегда возвращают некоторое значение.

Псевдофункции **set** и **setq** представляют оператор присваивания в языке Лисп. Использование этих псевдофункций **нарушает** принцип прозрачности по ссылкам. Поскольку мы изучаем чистое функциональное программирование, то формы **set** и **setq** если и будут использоваться, то только для одноразового присваивания значений символам, используемым в качестве глобальных переменных. Например,

```
> (setq a 5)
5
```

```
> (isert-plus a a)
(5 + 5)
```

Использование псевдофункций `set` и `setq` внутри тела какой-либо функции несовместимо с чистым функциональным программированием. Следующие два определения функции, увеличивающей аргумент на два, различаются по отношению к чистому функциональному программированию. Первое определение нарушает принцип прозрачности по ссылкам, второе — нет.

```
(defun inc1 (n) (setq n (+ n 2)))
(defun inc2 (n) (+ n 2))
```

Интерпретатор Лиспа называется **eval**, и его можно так же, как и другие функции, вызывать из программы. Лишний вызов интерпретатора может, например, снять эффект блокировки вычисления или позволяет найти значение значения выражения.

```
> '(+ 2 3)
(+ 2 3)

> (eval '(+ 2 3))
5

> (setq x '(a b c))
(A B C)

> 'x
X

> (eval 'x)
(A B C)

> (setq y 'x)
X
> y
X

> (eval y)
(A B C)
```

Сейчас можно описать более точно основной цикл работы интерпретатора Лиспа в виде последовательности вызовов:

```
(print '>')
(setq a (read))
(setq b (eval a))
(print b)
(print '>')
```

Разветвление вычислений: условное предложение cond. Это синтаксическая форма, позволяющая управлять вычислениями на основе определяемых предикатами условий. Структура условного предложения такова:

```
(cond (p1 a1)
      (p2 a2)
      .....
      (pn an))
```

Предикатами p_i и результирующими выражениями a_i могут быть произвольные формы. Значение предложения `cond` определяется следующим образом:

- Выражения p_i , являющиеся предикатами, вычисляются последовательно слева направо (сверху вниз) до тех пор, пока не встретится выражение, значением которого не является NIL, т.е. логическим значением которого является истина.
- Вычисляется результирующее выражение, соответствующее этому предикату, и полученное значение возвращается в качестве значения всего предложения `cond`.
- Если истинного предиката нет, то значением `cond` является NIL.

Рекомендуется в качестве последнего предиката использовать символ T.

Пример:

```
(defun sign (x)
  (cond ((> x 0) 'positive)
        ((< x 0) 'negative))
        (t 'zero))
)
```

В условном предложении может отсутствовать результирующее выражение `ai` или на его месте может быть последовательность форм:

```
(cond (p1 a1)
      .....
      (pi)
      .....
      (pk ak)
      .....)
```

Если условию не ставится в соответствие результирующее выражение, то в качестве результата предложения `cond` при истинности предиката выдается само значение предиката.

```
(if <условие> <то-форма> <иначе-форма>)
эквивалентно
(cond (<условие> <то-форма>)
      (T <иначе-форма>))
```

Let создает локальную связь. Вычисление вызова функции создает на время вычисления новые связи для формальных параметров функции. Новые связи внутри формы можно создавать и с помощью предложения `let`. Эта структура (немного упрощенно) выглядит так:

```
(let ((<перем1> <знач1>) (<перем2> <знач2>) ... )
  (<выражение>)
)
```

Форма `let` вычисляется так, что сначала локальные переменные `перем1`, `перем2`, ... из первого «аргумента» формы связываются (**одновременно**) с соответствующими значениями `знач1`, `знач2`, Затем вычисляется значение выражения, которое и возвращается в качестве значения формы `let`. Как и у функций, после окончания вычисления связи локальных переменных `перем1`, `перем2`, ... ликвидируются и любые изменения их значений не будут видны извне. Пример:

```
(defun f (x)
  (let ((y1 (* x x))
        (y2 (+ x x)))
    (= y1 y2))
)

> (f 0)
T
> (f 1)
NIL
> y1
error: unbound variable — Y1
```

Форма `let` позволяет использовать локальные переменные в теле функции. Но в Лиспе возможно использование локальных функций. **Локальные функции** создаются с помощью форм `flet` и `labels`.

Определения форм `flet`, `labels`

```
(flet (<binding> ...) <expr>)

(labels (<binding> ...) <expr>)
```

где `<binding>` — функциональные связи, каждая из которых имеет вид `(<sym> <farg> <body>)`,

`<sym>` — имя локальной функции,
`<farg>` — список формальных параметров,
`<body>` — тело локальной функции и

<expr> — вычисляемое выражение формы, содержащее вызовы локальных функций.

Форма labels используется в случае, когда хотя бы одна из локальных функций является рекурсивной.

Рассмотрим задачу, в которой требуется написать программу, вычисляющую следующую арифметическую функцию:

$$f(x) = \begin{cases} (\max(x, 4))^2 + 1, & \text{если } x \leq 1; \\ 2(\max(x, 4))^2, & \text{если } x > 1. \end{cases}$$

Первый вариант программы неудачный, поскольку значение $(\max(x, 4))^2$ вычисляется дважды при $x > 1$ (функция max — встроенная).

```
(defun f1 (x)
  (+ (* (max x 4) (max x 4))
    (if (<= x 1) 1 (* (max x 4) (max x 4)))
  )
)
```

В следующем варианте значение $(\max(x, 4))^2$ вычисляется только однажды, но значение $\max(x, 4)$ вычисляется дважды:

```
(defun f2 (x)
  (if (<= x 1) (1+ (* (max x 4) (max x 4)))
    (* 2 (max x 4) (max x 4))
  )
)
```

Можно определить вспомогательную функцию для вычисления $(\max(x, 4))^2$:

```
(defun g (x) (* (max x 4) (max x 4)))
```

тогда варианты 1 и 2 превращаются в варианты 3 и 4 соответственно. Программа становится яснее, но прежние недостатки остаются.

```
(defun f3 (x) (+ (g x) (if (<= x 1) 1 (g x))))
```

```
(defun f4 (x) (if (<= x 1) (1+ (g x)) (* 2 (g x))))
```

В следующем варианте вводится локальная переменная для хранения значения $(\max(x,4))^2$:

```
(defun f5 (x)
  (let
    ((a (* (max x 4) (max 4 x))))
    (+ a (if (<= x 1) 1 a))
  )
)
```

Значение $(\max 4 x)$ по-прежнему вычисляется дважды. Можно ввести локальную функцию с тем же эффектом:

```
(defun f6 (x)
  (flet
    ((a (y) (* (max y 4) (max 4 y))))
    (if (<= x 1) (1+ (a x)) (* 2 (a x)))
  )
)
```

И, наконец, в последних двух вариантах $(\max x 4)$ вычисляется только один раз:

```
(defun g1 (x)
  (let ((a (max x 4)))
    (* a a))
)
```

```
(defun f7 (x) (if (<= x 1) (1+ (g1 x)) (* 2 (g1 x))))
```

```
(defun f8 (x)
  (flet
    ((g1 (y)
      (let ((a (max y 4)))

```

```
      (* a a))  
    ))  
(if (<= x 1) (1+ (g1 x)) (* 2 (g1 x)))  
)  
)
```

«... на столе появился маленький Витька Корнеев, точная копия настоящего, но величиной с руку. Он щелкнул маленькими пальчиками и создал микродубль еще меньшего размера. Тот тоже щелкнул пальцами. Появился дубль величиной с авторучку. Потом величиной со спичечный коробок. Потом с наперсток».

*А. и Б. Стругацкие
«Понедельник начинается
в субботу»*

5 РЕКУРСИЯ

5.1 Знакомство с рекурсией

Предположим вам нужно написать функцию для подсчета суммы первых n неотрицательных целых чисел. С помощью процедурного языка (скажем Паскаля) задача легко решается с помощью следующей программы:

```
function sum(n:integer):integer;
var s,i:integer;
begin
  s:=0;
  for i:=0 to n do
    s:=s+i;
  sum:=s
end;
```

Здесь для обновления значения счетчика s , который при выходе из цикла содержит требуемый результат, используется цикл **for**. **В чистом функциональном языке не существует циклических конструкций и оператора присваивания, изменяющего значения счетчика.** В таких языках задача решается с использованием рекурсивной функции.

Обсудим понятие рекурсии. Объект называется **рекурсивным**, если он содержит сам себя или определен с помощью самого себя.

Рекурсия встречается не только в математике, но и в обычной жизни. Каждый сталкивается с рекурсией, когда стоит с зеркальцем перед большим зеркалом. Рекурсия встречается обычно и в природе: деревья имеют рекурсивное строение (ветки образуются из других веток), реки образуются из впадающих в них рек. Клетки делятся рекурсивно. Продолжение жизни связано с рекурсивным процессом. Молекулы ДНК и вирусы размножаются, копируя себя, живые существа имеют потомство, которое, в свою очередь, тоже имеет потомство, и т.д. Рекурсия распространена и в языке, и в поведении так же, как и в способах рассуждения и познания. Рекурсия в языке, например, может быть в структуре или в содержании:

«Петя сказал, что Вася сказал, что...»

«Знаю, что знаю, но не помню»

«Сделать; заставить сделать; заставить, чтобы заставили сделать;...»

«Замени x этим предложением»

«Запомни и передай это сообщение»

...

Литературным примером может служить «рассказ в рассказе», как-то: известное стихотворение «У попа была собака,...», роман Яна Потоцкого «Рукопись, найденная в Сарагосе», рассказ Хулио Кортасара «Непрерывность парков» или рассказ Виктора Пелевина «Водонапорная башня» (и по форме — состоит из одного предложения, и по содержанию). Музыкальные формы и действия также могут быть рекурсивными во многих отношениях (например, канон или fuga, в которых мелодия сопровождается той же мелодией с задержкой, и другие). Целенаправленное поведение и решение проблем так же являются рекурсивными процессами.

Рекурсия является особенно мощным средством в математических определениях. Известны примеры рекурсивных определений натуральных чисел, древовидных структур и некоторых функций:

Натуральные числа:

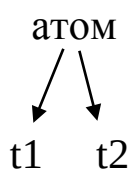
1) 1 есть натуральное число;

2) целое число, следующее за натуральным, есть натуральное число.

Древовидные структуры:

1) атомарный объект есть дерево;

2) если $t1$ и $t2$ — деревья, то



есть дерево (нарисованное сверху вниз).

Функция факториал $n!$ для неотрицательных целых чисел:

1) $0! = 1$;

2) если $n > 0$, то $n! = n(n-1)!$.

Очевидно, что мощность рекурсии связана с тем, что она позволяет определить бесконечное множество объектов с помощью конечного высказывания.

Рекурсивная функция — это функция, вызывающая сама себя. Конечно, вызываемая функция должна решать более простую задачу, чем вызывающая, иначе рекурсия никогда не закончится, поэтому одна из трудностей при написании рекурсивной функции состоит в том, чтобы выделить из текущей задачи одну или несколько более простых подзадач, аналогичных исходной. Функция, таким образом, может вызывать сама себя для решения более простых задач, а окончательное решение может быть получено из возвращаемых результатов. Чтобы решить нашу задачу нахождения суммы с использованием рекурсии, рассмотрим структуру выражения для суммы первых n неотрицательных целых чисел:

$$\text{sum}(n) = 1 + 2 + 3 + 4 + \dots + (n-1) + n$$

Из этой формулы можно увидеть связь между суммой первых n целых и суммой первых $n-1$ целых: если удалить $+n$ в конце выражения, останется выражение, эквивалентное $\text{sum}(n-1)$. Такую связь часто называют *рекуррентным отношением*, отсюда вытекает, что имея целое число n , можно использовать определенную нами функцию sum для суммирования первых $n-1$ целых и затем получить решение, добавив n к результату, возвращенному функцией $\text{sum}(n-1)$. Кроме того, мы должны определить так

называемый базовый случай, специфицирующий, где рекурсивный процесс должен остановиться. Для нашей функции базовый случай имеет место, когда аргумент равен 0, и мы ожидаем, что функция при этом возвратит 0 (сумма нуля целых чисел равна 0). Все это вместе дает следующее математическое соотношение:

$$\begin{aligned} \text{sum}(0) &= 0; \\ \text{sum}(n) &= \text{sum}(n-1) + n, \text{ если } n > 0. \end{aligned}$$

Определение соответствующей функции на Лиспе выглядит так:

```
(defun sum (n)
  (if (= n 0) 0 (+ n (sum (- n 1))))))
```

Хотя данное определение не требует, чтобы мы думали о том, как работает рекурсия, можно увидеть эффект рекурсии из следующего примера:

```
(sum 4)
→ (+ (sum 3) 4)
→ (+ (+ (sum 2) 3) 4)
→ (+ (+ (+ (sum 1) 2) 3) 4)
→ (+ (+ (+ (+ (sum 0) 1) 2) 3) 4)
→ (+ (+ (+ (+ 0 1) 2) 3) 4)
→ (+ (+ (+ 1 2) 3) 4)
→ (+ (+ 3 3) 4)
→ (+ 6 4)
→ 10
```

Стрелка $\rightarrow$ означает «равно после вычисления»: выражение, стоящее справа от стрелки, есть результат «упрощения» выражения, стоящего слева от стрелки.

Пример. Определить функцию (`sum s`), которая вычисляет сумму элементов списка `s`. Будем использовать рекурсивный алгоритм: для того чтобы вычислить сумму элементов списка `s`,

надо вычислить сумму элементов хвоста списка s плюс первый элемент списка. Более формально этот алгоритм таков:

- если список s — пуст, то результат — 0;
- иначе — результат равен результату рекурсивного вызова $(\text{sum } (\text{rest } s))$ плюс $(\text{first } s)$.

На Лиспе программа для этой задачи выглядит так:

```
(defun sum (s)
  (if (null s) 0
      (+ (first s) (sum (rest s)))))
)
```

Здесь в вычислениях функции sum два взаимоисключающих варианта, поэтому достаточно использовать функцию if . Но можно использовать и функцию cond :

```
(defun sum (s)
  (cond
    ((null s) 0)
    (t (+ (first s) (sum (rest s)))))
)
```

Длина списка. Требуется определить функцию $(\text{length } s)$, вычисляющую число элементов в списке s (требуемая функция в Лиспе уже определена, но мы покажем, что сделать это просто). Будем использовать рекурсивный алгоритм: количество элементов в непустом списке равно количеству элементов в хвосте списка плюс единица.

Более формальный алгоритм:

- если список s — пуст, то результат — 0;
- иначе — результат равен результату рекурсивного вызова $(f (\text{rest } s))$ плюс 1.

Этот алгоритм дает очевидную программу:

```
(defun length (s)
  (if (null s) 0 (1+ (length (rest s)))))
```

Усложним задачу. Требуется определить функцию ($f\ s$), вычисляющую количество положительных чисел в списке s (список может содержать и не числа). Будем использовать рекурсивный алгоритм: для того чтобы вычислить количество положительных чисел списка s , надо вычислить количество положительных чисел хвоста списка s плюс 1 или 0, в зависимости от того является ли первый элемент списка положительным числом.

Более формальный алгоритм:

- если список s — пуст, то результат — 0;
- если ($\text{first } s$) положительное число, то результат на 1 больше рекурсивного вызова ($f\ (\text{rest } s)$);
- иначе — результат равен результату рекурсивного вызова ($f\ (\text{rest } s)$).

На Лиспе соответствующая программа выглядит так:

```
(defun f (s)
  (cond
    ((null s) 0)
    ((and (numberp (first s)) (> (first s) 0)) (+ 1 (f (rest s))))
    (t (f (rest s))))
)
```

В определение f использование `cond` удобно, поскольку у нас три взаимоисключающих варианта.

Еще более усложним задачу. Теперь мы ищем количество положительных чисел в списке, который сам может содержать списки, поэтому мы должны считать положительные числа на всех уровнях. Мы добавляем дополнительную ветвь к предыдущему алгоритму: теперь голова списка сама может быть списком, и следует в этом случае рекурсивно вызывать функцию для головы списка.

Алгоритм:

- если список s — пуст, то результат — 0;
- если ($\text{first } s$) — список, находим сумму рекурсивных вызовов ($f\ (\text{first } s)$) и ($f\ (\text{rest } s)$);

- если (first s) положительное число, то результат на 1 больше рекурсивного вызова (f (rest s));
- иначе — результат равен результату рекурсивного вызова (f (rest s)).

На Лиспе соответствующая программа выглядит так:

```
(defun f (s)
  (cond
    ((null s) 0)
    ((listp (first s)) (+ (f (first s)) (f (rest s))))
    ((and (numberp (first s)) (> (first s) 0)) (+ 1 (f (rest s))))
    (t (f (rest s))))
)
```

Количество вхождений элемента. Теперь определим функцию (count s x), которая подсчитывает сколько раз элемент x входит в список s. Предпоследняя из рассмотренных функций аналогична функции count. Просто сейчас нужно проверять, совпадает ли первый элемент списка с элементом x, а до этого мы проверяли, является ли первый элемент положительным числом.

Алгоритм (count s x):

- если список s — пуст, то результат — 0;
- если x совпадает с (first s), то результат на 1 больше рекурсивного вызова (count (rest s) x);
- иначе — результат равен результату рекурсивного вызова (count (rest s) x).

```
(defun count (s x)
  (cond ((null s) 0)
        ((equal (first s) x) (1+ (count (rest s) x)))
        (t (count (rest s) x)))
)
```

5.2 Встроенные рекурсивные функции для списков

Поскольку список является рекурсивным объектом⁸, то естественно ожидать, что функции, обрабатывающие списки, являются рекурсивными. Познакомимся с рекурсивными определениями некоторых встроенных функций.

Функция `member` проверяет, принадлежит ли элемент списку. Элемент может принадлежать списку, если он совпадает с головой списка или содержится в хвосте списка. В соответствии с этим имеем алгоритм функции (`member s`):

- если список `s` — пуст, то результат — `NIL`;
- если `x` совпадает с (`first s`), то результат — `T`;
- иначе — результат равен результату рекурсивного вызова (`member (rest s) x`).

```
(defun member (x s)
  (cond ((null s) nil)
        ((equal (first s) x) t)
        (t (member x (rest s)))))
)
```

В рекурсивном определении существенен порядок следования условий. При определении порядка следования основным моментом является то, что сначала проверяются всевозможные условия окончания, а затем ситуации, требующие продолжения вычислений. При помощи условий окончания последовательно проверяются все особые случаи и программируются соответствующие результирующие выражения. На исключенные таким образом случаи можно в последующем не обращать внимания.

Предикат `member` в языке `XLisp` определен несколько в другом виде, чем мы его представили.

Во-первых, вместо значения `T` функция возвращает подсписок исходного списка, начиная с элемента, который совпадает с искомым.

⁸ Список либо пуст, либо состоит из головы — произвольного символического выражения и хвоста, который также является списком.

```
> (member 5 '(1 2 5 6))
(5 6)
```

Во-вторых, вместо предиката сравнения `equal` используется предикат `eq`, поэтому встроенный `member` не может находить элементы, являющиеся списками. Но можно вместо предиката `eq`, являющегося умолчанием, задать при вызове с помощью ключевого параметра `:test` другой предикат. Например, в следующем примере для сравнения элементов используется предикат `equal`, который применим и к спискам:

```
> (member '(c d) '((a b) (c d)) :test 'equal)
((c d))
```

Функция `append` соединяет два списка. Приведем пример работы:

```
> (append '(1 2) '(3 4 5))
(1 2 3 4 5)
```

Для определения функции `(append x y)` применим рекурсию: для того чтобы соединить два списка `x` и `y`, соединяем хвост списка `x` со списком `y` и к результату в качестве головы добавляем голову списка `x`.

Более точно алгоритм можно записать так:

- если список `x` — пуст, то результат — список `y`;
- иначе рекурсивно вызываем `(append (rest x) y)` и к результату в качестве головы добавляем `(first x)`.

Сразу же получаем функцию на Лиспе:

```
(defun append (x y)
  (if (null x) y
      (cons (first x) (append (rest x) y))))
)
```

С помощью функции `trace`⁹ можно устроить трассировку `append`.

```
> (trace append)
(APPEND)
> (append '(1 2) '(3 4 5))
; 4> APPEND called with args:
;   (1 2)
;   (3 4 5)
; | 5> APPEND called with args:
; |   (2)
; |   (3 4 5)
; || 6> APPEND called with args:
; ||   NIL
; ||   (3 4 5)
; || 6< APPEND returns value:
; ||   (3 4 5)
; | 5< APPEND returns value:
; |   (2 3 4 5)
; 4< APPEND returns value:
;   (1 2 3 4 5)

(1 2 3 4 5)
```

Функция `reverse` обращает список. Приведем пример работы:

```
> (reverse '(1 2 3 4 5))
(5 4 3 2 1)
```

Для определения функции `(reverse x)` применим рекурсию: для того чтобы обратить список `x`, обращаем хвост списка `x` и к нему в конец добавляем голову списка `x`.

Более точно алгоритм можно записать так:

- если список `x` — пуст, то результат — список `x`;

⁹ Хотя функция `trace` имеется в `XLisp`'е, здесь приведена более наглядная иллюстрация работы функции `trace` для некоторого другого диалекта Лиспа — работа `trace` в `XLisp`'е менее наглядна.

- иначе рекурсивно вызываем (reverse (rest x) y) и результат соединяем с (list (first x)).

Определение функции:

```
(defun reverse (x)
  (if (null x) nil
      (append (reverse (rest x)) (list (first x)))))
)
```

Проделаем трассировку:

```
(trace reverse)
(REVERSE)
```

```
> (REVERSE '(a b c))
; 1> REVERSE called with arg:
;   (A B C)
; | 2> REVERSE called with arg:
; |   (B C)
; || 3> REVERSE called with arg:
; ||   (C)
; ||| 4> REVERSE called with arg:
; |||   NIL
; ||| 4< REVERSE returns value:
; |||   NIL
; || 3< REVERSE returns value:
; ||   (C)
; | 2< REVERSE returns value:
; |   (C B)
; 1< REVERSE returns value:
;   (C B A)
```

```
(C B A)
```


Функция `remove` удаляет элемент из списка. Если элемент `x` входит несколько раз в список `s`, то удаляется только первое вхождение элемента. Легко сформулировать рекурсивный алгоритм функции (`remove x s`):

- если список `s` — пуст, то результат — список `s`;
- если `(first s)` совпадает с элементом `x`, то в качестве результата возвращается `(rest s)`;
- иначе рекурсивно вызываем `(remove x (rest s))` и к результату добавляем в качестве головы `(first s)`.

```
(defun remove (x s)
  (cond ((null s) nil)
        ((eql x (first s)) (rest s))
        (t (cons (first s) (remove x (rest s))))))
)
```

```
> (remove 2 '(1 2 3 4))
(1 3 4)
> (remove 'b '(a (b c)))
(A (B C))
> (remove '(a b) '((a b) (c d)))
((A B) (C D))
```

Встроенная функция `remove` использует предикат `eql` (поэтому в последнем примере элемент-список не удаляется). Можно задать непосредственно предикат сравнения для функции `remove` при её вызове ключевым параметром `:test` таким же образом, как и для функции `member`. Тогда:

```
> (remove '(a b) '((a b) (c d)) :test 'equal)
((c d))
```

Если надо удалить все вхождения, даже если голова списка совпадает с удаляемым элементом, то рекурсивно удаляем элемент также из хвоста. Функция `remove-all` удаляет все вхождения элемента.

```
(defun remove-all (x s)
  (cond ((null s) nil)
        ((eql x (first s)) (remove-all x (rest s)))
        (t (cons (first s) (remove-all x (rest s))))))
)
```

5.3 Различные виды рекурсии

Прямая, косвенная и линейная рекурсии

Во всех рассмотренных рекурсивных программах применялась **прямая рекурсия**, т.е. рекурсивный вызов следовал непосредственно из тела самой функции. **Косвенная** (или **взаимная**) рекурсия возникает, когда имеется циклическая последовательность вызовов нескольких функций $F_1, F_2, \dots, F_k$ друг друга: F_1 вызывает F_2 , F_2 вызывает F_3 , ..., F_k вызывает F_1 ($k > 1$).

Рассмотрим задачу. Имеется натуральное число n , требуется определить, является ли количество единиц в двоичном представлении числа n четным или нечетным. Эту задачу можно решить с помощью двух взаимно рекурсивных логических функций:

- (even-ones n) выдает «истину», если количество единиц четно;
- (odd-ones n) выдает «истину», если количество единиц нечетно.

Нетрудно написать функции:

```
(defun even-ones (n)
  (cond
    ((zerop n) t)
    ((zerop (mod n 2)) (even-ones (truncate (/ n 2))))
    (t (odd-ones (truncate (/ n 2))))))
)

(defun odd-ones (n)
  (cond
    ((zerop n) nil)
    ((zerop (mod n 2)) (odd-ones (truncate (/ n 2))))
    (t (even-ones (truncate (/ n 2))))))
)
```

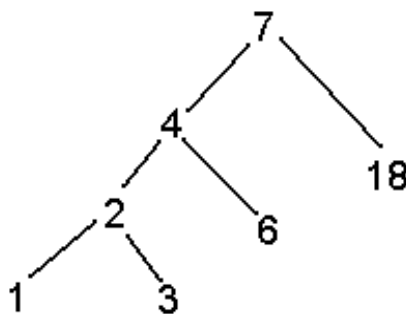
Иногда косвенную рекурсию удастся свести к обычной рекурсии. Например, в нашем случае можно было бы перейти к прямой рекурсии.

```
(defun ones (n)
  (cond
    ((zerop n) t)
    ((= n 1) nil)
    ((zerop (mod n 2)) (ones (truncate (/ n 2))))
    (t (not (ones (truncate (/ n 2))))))
  )
)
```

Однако при подобных преобразованиях не стоит забывать, что во многих случаях совокупности функций с взаимной рекурсией оказываются более обозримыми и легче читаемыми, чем равнозначные им системы с прямой рекурсией.

Если в теле функции присутствует только один прямой рекурсивный вызов, то такая рекурсия называется **линейной**. Например, линейной рекурсивной функцией является функция `ones`. Рассмотренная в разделе 5.1 функция `f`, которая вычисляет количество положительных чисел в списке, который сам может содержать списки, не является линейно рекурсивной. Она содержит два рекурсивных вызова: для головы списка и для хвоста.

С помощью многоуровневых списков можно представлять сложные иерархические структуры. Например, пустое бинарное дерево изображается символом `nil`, а непустое — списком из трех элементов (<левое поддерево> <корень> <правое поддерево>). Так, дерево на следующем рисунке изображается списком `((nil 1 nil) 2 (nil 3 nil)) 4 (nil 6 nil) 7 (nil 18 nil)`.



Рассмотрим программу сортировки списка чисел с помощью бинарного упорядоченного дерева. Список сортируется в два этапа: сначала его преобразуем в упорядоченное бинарное дерево, а потом дерево преобразуем в список, читая его справа налево.

В тексте программы символ «; – точка с запятой» означает начало комментария, он действует до конца строки. Встроенные функции `second` и `third` выдают второй элемент и третий элемент списка соответственно.

; x - исходный список, результат — упорядоченный список

```
(defun treeSort (x)
  (flatten (makeTree x)))
```

; создание упорядоченного дерева из списка x

```
(defun makeTree (x)
  (if (null x) nil (insert (first x) (makeTree (rest x))))
)
```

; вставка элемента n в упорядоченное дерево x

```
(defun insert (n x)
  (if (null x) (list nil n nil)
      (let ( (left (first x))
              (node (second x))
              (right (third x)))
        (if (<= n node) (list (insert n left) node right)
            (list left node (insert n right)))
      )))
```

; разглаживание дерева

```
(defun flatten (x)
  (if (null x) nil
      (append (flatten (first x))
                (cons (second x) (flatten (third x))))
  )
)
```

Накапливающий параметр — аккумулятор и хвостовая рекурсия

Бывает так, что при выполнении функции исключительно серьезно встает проблема расхода памяти. Эту проблему можно пояснить на примере функции, вычисляющей факториал числа:

```
(defun ! (n)
  (if (zerop n) 1 (* n (! (1- n))))
)
```

Если провести трассировку вычисления этой функции с аргументом 3, то можно будет увидеть следующую последовательность вычислений:

```
(! 3) →
3 * (! 2) →
3 * 2 * (! 1) →
3 * 2 * 1 * (! 0) →
3 * 2 * 1 * 1 →
3 * 2 * 1 →
3 * 2 →
6
```

На примере этого вычисления наглядно видно, что при рекурсивных вызовах функций очень сильно используется память. В данном случае количество памяти пропорционально значению аргумента. Возникает резонный вопрос: можно ли так написать функцию вычисления факториала (и ей подобные), чтобы память использовалась минимально?

Чтобы ответить на данный вопрос положительно, необходимо рассмотреть понятие **аккумулятора** (накопителя). Для этого можно рассмотреть следующий пример вычисления факториала с **накапливающим параметром**:

```
(defun fac (n)
  (labels ((f (n m)
    (if (zerop n) m (f (1- n) (* n m)))))
```

```

    ))
  (f n 1))
)

```

В этом примере второй параметр m функции f выполняет роль аккумулярующей переменной, именно в ней содержится результат, который возвращается по окончании рекурсии. Сама же рекурсия в этом случае принимает вид «хвостовой», память при этом расходуется только на хранение адресов возврата значения функции.

Хвостовая рекурсия представляет собой специальный вид рекурсии, в которой имеется единственный вызов рекурсивной функции и при этом этот вызов выполняется после всех вычислений (для функции! рекурсия не является хвостовой).

При реализации вычисления хвостовой рекурсии могут выполняться при помощи итераций в постоянном объеме памяти. На практике это обозначает, что «хороший» транслятор функционального языка должен «уметь» распознавать хвостовую рекурсию и реализовывать её в виде цикла. В свою очередь, метод накапливающего параметра не всегда приводит к хвостовой рекурсии, однако он однозначно помогает уменьшить общий объём памяти.

Хвостовая рекурсия уменьшает время работы. Рассмотрим вычисление числа Фибоначчи по его номеру. Математическое определение последовательности чисел Фибоначчи 1, 1, 2, 3, 5, 8, 13, 21, 34, ... таково: первые два числа $\text{fib}(1)$ и $\text{fib}(2)$ — единицы, а каждое следующее число последовательности получается суммированием двух предыдущих.

Определение «в лоб»:

```

(defun fib1 (n)
  (cond
    ((= n 1) 1)
    ((= n 2) 1)
    (t (+ (fib1 (1- n)) (fib1 (- n 2)))))
  )
)

```

Этот алгоритм является экспоненциальным по времени.

Линейный алгоритм получаем, вводя два накапливающих параметра (эти параметры хранят два текущих числа Фибоначчи).

```
(defun fib2 (n) (f 1 1 1))
```

```
(defun f (k a b) (if (= k n) a (f (1+ k) b (+ a b))))
```

Возвратимся к функции `reverse`, рассмотренной в разделе 5.2. Время работы этой функции (измеряемой, скажем, в количестве операций `cons`) имеет квадратичную зависимость от длины списка. Использование накапливающего параметра позволяет эту зависимость сделать линейной:

```
(defun reverse1 (s)
  (labels
    ((f (x y)
      (if (null x) y
          (f (rest x) (cons (first x) y))))
    ))
  (f s nil))
)
```

Возникает вопрос: любую ли функцию можно преобразовать для вычисления с аккумулятором? Очевидно, что ответ на этот вопрос отрицателен. Построение функций с накапливающим параметром — приём не универсальный, и он не гарантирует получения хвостовой рекурсии. С другой стороны, построение определений с накапливающим параметром является делом творческим.

Рассмотрим задачу нахождения суммы и произведения элементов списка чисел (результат выдается в виде списка из двух значений). Для этой задачи можно предложить различные варианты программы.

; 1 вариант
; программирование «в лоб»

```
(defun sumpr (x)
  (if (null x) (list 0 1)
      (list (+ (first x) (first (sumpr (rest x))))
            (* (first x) (second (sumpr (rest x)))))))
)
```

; 2 вариант
; использование let

```
(defun sumpr (x)
  (if (null x) (list 0 1)
      (let ((z (sumpr (rest x))))
        (list (+ (first x) (first z))
              (* (first x) (second z))))))
)
```

; 3 вариант
; еще одно использование let

```
(defun sumpr (x)
  (if (null x) (list 0 1)
      (let ((n (first x)) (z (sumpr (rest x))))
        (list (+ n (first z))
              (* n (second z))))))
)
```

; 4 вариант
; еще одно использование let

```
(defun sumpr (x)
  (if (null x) (list 0 1)
      (let ((n (first x)) (z (sumpr (rest x))))
        (let ((a (first z)) (b (second z)))
          (list (+ n a) (* n b))))))
)
```


; 5 вариант

; использование накапливающих параметров

```
(defun sumpr (x) (sp x 0 1))
```

```
(defun sp (x s p)
  (if (null x) (list s p)
      (sp (rest x) (+ (first x) s) (* (first x) p))))
)
```

; 6 вариант

; использование накапливающих параметров и локальной функции

```
(defun sumpr (x)
  (labels ((sp (y s p)
            (if (null y) (list s p)
                (sp (rest y) (+ s (first y)) (* p (first y))))))
    (sp x 0 1))
)
```

Удаленная рекурсия

Рекурсия называется **удаленной** (или рекурсией высокого порядка), если в теле функции при рекурсивных вызовах, в выражениях, являющихся фактическими параметрами, снова встречаются рекурсивные вызовы этой функции.

Подобный пример дает нам классическая в теории рекурсии функция Аккермана:

$$\begin{aligned} ak(0, m) &= m+1; \\ ak(n, 0) &= ak(n-1, 1), \text{ при } n>0; \\ ak(n, m) &= ak(n-1, ak(n, m-1)), \text{ при } n>0 \text{ и } m>0. \end{aligned}$$

Нетрудно написать функцию на Лиспе:

```
(defun ak (n m)
  (cond
    ((zerop n) (1+ m))
    ((zerop m) (ak (1- n) 1))
    (t (ak (1- n) (ak n (1- m)))))
```

)
)

Вычислить её непосредственно, исходя из определения, удастся лишь для некоторых малых значений n и m . Связано это со сложностью и необычностью удаленной рекурсии.

Для $m = 0..4$ справедливы соотношения:

$$\begin{aligned} ak(0, n) &= n+1; \quad ak(1, n) = n+2; \quad ak(2, n) = 2n+3; \quad ak(3, n) = 2^{n+3} - 3; \\ ak(4, n) &= 2^{s(n)} - 3, \text{ где } s(0) = 2^2, \quad s(n+1) = 2^{s(n)} \quad (n=0, 1, \dots). \end{aligned}$$

Удаленная рекурсия не обязательно приводит к громоздким вычислениям. Например, если $sum(n)$ — сумма цифр целого неотрицательного числа n и итерированную сумму $qsum(n)$ («цифровой корень») определить равной значению первого из элементов числовой последовательности $n_p = sum(n_{p-1})$ ($n_0 = n$; $p=1, 2, \dots$), меньшего десяти, то находить $qsum(n)$ можно с помощью следующей удаленной рекурсии:

```
(defun qsum (n)
  (if (< n 10) n (qsum (+ (qsum (truncate (/ n 10))) (mod n 10))))
)
```

Заметим, что если k — количество десятичных цифр числа n , то для вычисления $qsum(n)$ требуется не более $2k$ рекурсивных вызовов. Впрочем, вычисление $qsum(n)$ можно было бы организовать и с помощью простой линейной рекурсии:

```
(defun qsum1 (n)
  (if (< n 10) n (qsum (+ (truncate (/ n 10)) (mod n 10))))
)
```

*«... Витькины дубли на столе
продолжали работать. Самый ма-
ленький был уже ростом с муравья».*

*А. и Б. Стругацкие
«Понедельник начинается
в субботу»*

6 ЛАМБДА-ИСЧИСЛЕНИЕ

*Желтая река течет тысячи
миль на север...*

*Затем поворачивает на восток
и течет непрерывно,*

*Не важно, как она изгибается
и поворачивается,*

*Её воды выходят из источника
на горе Кунь-Лунь.*

Железная флейта

6.1 Ламбда-исчисление как формальная система

Значение ламбда-исчисления

Ламбда-исчисление было изобретено Алонзом Чёрчем (Alonzo Church, американский логик и математик, 1903–1995) около 1930 г. Чёрч первоначально строил λ -исчисление как часть общей системы функций, которая должна стать основанием математики. Но из-за найденных парадоксов эта система оказалась противоречивой. Книга Чёрча [2] содержит непротиворечивую подтеорию его первоначальной системы, имеющую дело только с функциональной частью. Эта теория и есть λ -исчисление.



Алонзо Чёрч

Лямбда-исчисление — это безтиповая теория, рассматривающая функции как *пра-вила*, а не как графики. В противоположность подходу Дирихле (вводимому функции как множество пар, состоящих из аргумента и значения) более старое понятие определяет функцию как процесс перехода от аргумента к значению. С первой точки зрения x^2-4 и $(x+2)(x-2)$ — разные обозначения одной и той же функции; со второй точки зрения это разные функции.

Что значит «функция $5x^3+2$ »? Если кто-то хочет быть точным, он вводит по этому поводу функциональный символ, например f , и говорит: «функция $f: \mathbf{R} \rightarrow \mathbf{R}$, определенная соотношением $f(x) = 5x^3+2$ ». При этом, очевидно, что переменную x можно здесь, не меняя смысла, заменить на другую переменную u . Лямбда-запись устраняет произвольность в выборе f в качестве функционального символа. Она предлагает вместо f выражение « $\lambda x. 5x^3+2$ ».

Кроме того, обычная запись $f(x)$ может обозначать как имя функции f , так и вызов функции с аргументом x . Для более строгого подхода это необходимо различать. В лямбда-обозначениях вызов функции с аргументом x выглядит как $(\lambda x. 5x^3+2)x$.

Функции как правила рассматриваются в полной общности. Например, мы можем считать, что функции заданы определениями на обычном русском языке и применяются к аргументам, также описанным по-русски. Также мы можем рассматривать функции заданными программами и применяемые к другим программам. В обоих случаях перед нами — *безтиповая структура*, где объекты изучения являются одновременно и функциями, и аргументами. Это отправная точка безтипового λ -исчисления. В частности, функция может применяться к самой себе.

Лямбда-исчисление представляет класс (частичных) функций (λ -определимые функции), который в точности характеризует неформальное понятие эффективной вычислимости. Другими словами, λ -исчисление наряду с другими подходами формализует понятие алгоритма. И в настоящее время лямбда-исчисление является основной формализацией. Одной из причин этого является

следующий факт — это единственная формализация, которая, хотя и с некоторыми неудобствами, действительно может быть непосредственно использована для написания программ.

Лямбда-исчисление стало объектом особенно пристального внимания в информатике после того, как выяснилось, что оно представляет собой удобную теоретическую модель современного функционального программирования [8]. Кроме того, большинство конструкций традиционных языков программирования может быть более или менее непосредственно отображено в конструкции лямбда-исчисления.

Функциональные языки являются в основном удобной формой синтаксической записи для конструкций различных вариантов лямбда-исчисления (соответствующие отклонения в языке от синтаксиса лямбда-выражений называются «синтаксическим сахаром»). Некоторые современные языки (Haskell, Clean) имеют 100% соответствие своей семантики с семантикой подразумеваемых конструкций лямбда-исчисления. Причем λ -выражения используются в качестве промежуточного кода, в который можно транслировать исходную программу. Функциональные языки «улучшают» нотацию λ -исчисления в прагматическом смысле, но при этом, в какой-то мере, теряется элегантность и простота последнего.

Изучение и понимание многих сложных ситуаций в программировании, например, таких, как автоаппликативность (самоприменимость) или авторепликативность (самовоспроизводство), сильно облегчается, если уже имеется опыт работы в λ -исчислении, где выделены в чистом виде основные идеи и трудности.

Язык лямбда-исчисления является сейчас одним из важнейших выразительных средств в логике, информатике, математической лингвистике, искусственном интеллекте и когнитивной науке.

Синтаксис и семантика λ -исчисления

Ламбда-исчисление есть язык для определения функций. Выражения языка называются λ -выражениями, и каждое такое выражение обозначает функцию. Далее мы рассмотрим, как функции могут представлять различные структуры данных: числа, списки и т.д. Для некоторых λ -выражений мы будем использовать имена (или сокращенные обозначения), они будут записываться полужирным шрифтом.

Определение λ -выражений (λ -термов)

Имеется три вида λ -выражений:

- *Переменные*: x , y и т.д.
- *Функциональная аппликация*: если M и N есть произвольные λ -термы, то можно построить новый λ -терм (MN) (обозначающий применение, или *аппликацию*, оператора M к аргументу N). Например, если $\langle m \rangle$, $\langle n \rangle$ обозначает функцию, представляющую пару чисел m и n и **sum** обозначает функцию сложения в λ -исчислении, то аппликация **(sum $\langle m \rangle$, $\langle n \rangle$)** обозначает $\langle m+n \rangle$ (т.е. ламбда-терм, представляющий число $m+n$).
- *Абстракция*: По любой переменной x и любому λ -терму M можно построить новый λ -терм $(\lambda x.M)$ (обозначающий функцию от x , определяемую λ -термом M). Такая конструкция называется λ -абстракция. Например, $\lambda x.$ **sum** (x , $\langle 1 \rangle$) обозначает функцию от x , которая увеличивает аргумент на 1.

Тождественная функция представляется λ -термом $(\lambda x. x)$. Аппликация $(\lambda x. x) E$ дает E .

Еще пример. Пусть λ -выражения $\langle 0 \rangle$, $\langle 1 \rangle$, $\langle 2 \rangle$, ... представляют числа 0, 1, 2, ..., соответственно; и **add** есть λ -выражение, обозначающее функцию, удовлетворяющее правилу:

$$((\mathbf{add} \langle m \rangle) \langle n \rangle) = \langle m+n \rangle.$$

Тогда $(\lambda x. ((\mathbf{add} \langle 1 \rangle) x))$ есть λ -выражение, обозначающее функцию, что преобразует $\langle n \rangle$ в $\langle n+1 \rangle$, и $(\lambda x. (\lambda y. ((\mathbf{add} x) y)))$ есть

λ -выражение, обозначающее функцию, которая преобразует $\langle m \rangle$ в функцию $(\lambda y. ((\mathbf{add} \langle m \rangle) y))$.

Символ x после λ называется *связанной переменной абстракции* и соответствует понятию формального параметра в традиционной процедуре или функции. Выражение справа от точки называется *телом абстракции*, и, подобно коду традиционной процедуры или функции, оно описывает, что нужно сделать с параметром, поступившим на вход функции. Мы читаем символ λ как «функция от» и точку $(.)$ как «которая возвращает».

Чтобы уменьшить число применяемых скобок, будем использовать следующие соглашения.

- Операция аппликации лево-ассоциативна, т.е. $E_1 E_2 \dots E_n$ означает $((\dots (E_1 E_2) \dots) E_n)$. Например, $E_1 E_2 E_3$ обозначает $((E_1 E_2) E_3)$.
- $\lambda V. E_1 E_2 \dots E_n$ обозначает $(\lambda V. (E_1 E_2 \dots E_n))$. Область действия λV распространяется направо так далеко, как это возможно, т.е., например, $\lambda x.xu$ означает $\lambda x.(xu)$, а не $(\lambda x.x)u$.
- Операция абстракции является право-ассоциативной, т.е. $\lambda V_1. \lambda V_2. \dots \lambda V_n. E$ обозначает $(\lambda V_1. (\lambda V_2. (\dots (\lambda V_n. E) \dots)))$. Запись $\lambda V_1. \lambda V_2. \dots \lambda V_n. E$ иногда еще более сокращают $\lambda V_1 V_2 \dots V_n. E$. Так, например, $\lambda x y z. E$ обозначает $(\lambda x. (\lambda y. (\lambda z. E)))$.

Лямбда-терм определяется рекурсивно и его грамматику можно определить в виде следующей формы Бэкуса-Наура:

$$Exp = Var \mid \lambda Var . Exp \mid Exp Exp$$

В соответствии с этой грамматикой лямбда-термы представляются в виде синтаксических деревьев, а не в виде последовательности символов. Отсюда следует, что соглашения об ассоциативности операции применения функции, эквивалентность выражений вида $\lambda x y. S$ и $\lambda x. \lambda y. S$, неоднозначность в именах переменных проистекают только из необходимости представления лямбда-термов в удобном человеку виде и не являются частью формальной системы.

Вычисление λ -выражений

Переменная, расположенная не на месте связанной переменной, может быть *связанной* или *свободной*, что определяется с помощью следующих правил:

1. Переменная x оказывается свободной в выражении x .
2. Все x , имеющиеся в $\lambda x.M$, являются связанными. Если кроме x в $\lambda x.M$ есть переменная y , то последняя будет свободной или связанной в зависимости от того, свободна она или связана в M .
3. Переменная, встречающаяся в термах M или N выражения (MN) , будет связанной или свободной в общем терме в зависимости от того, свободна она или связана в M или N . Свободные (связанные) переменные — это переменные, которые, по крайней мере, один раз появляются в терме в свободном (связанном) виде.

Нам понадобится следующее определение *подстановки* терма в другой терм вместо свободного вхождения переменной. Для любых λ -термов M , N и переменной x через $[N/x]M$ обозначим результат подстановки N вместо каждого свободного вхождения x в M и замены всех λy в M таким образом, чтобы свободные переменные из N не стали связанными в $[N/x]M$. Мы употребляем запись $M \equiv N$ для обозначения того, что термы M и N синтаксически совпадают.

Подстановка

- a) $[N/x]x \equiv N$;
- b) $[N/x]y \equiv y$, если переменная y не совпадает с x ;
- c) $[N/x](PQ) \equiv ([N/x]P [N/x]Q)$;
- d) $[N/x](\lambda x.P) \equiv \lambda x.P$;
- e) $[N/x](\lambda y.P) \equiv \lambda y.[N/x]P$, если y не имеет свободных вхождений в N и x имеет свободное вхождение в P ;
- f) $[N/x](\lambda y.P) \equiv \lambda z.[N/x]([z/y]P)$, если y имеет свободное вхождение в N и x имеет свободное вхождение в P и z — любая переменная, не имеющая свободных вхождений в N .

Следующие примеры пояснят суть определения. Пусть $M \equiv \lambda y.yx$.

Если $N \equiv vx$, то $[(vx)/x](\lambda y.yx) \equiv \lambda y. [(vx)/x](yx)$ согласно (e)
 $\equiv \lambda y. y(vx)$ согласно (a).

Если $N \equiv ux$, то $[(yx)/x](\lambda y.ux) \equiv \lambda z. [(yx)/x](zx)$ согласно (f)
 $\equiv \lambda z.z(yx)$ согласно (a).

Введенное понятие подстановки требует должной мотивации. Для этого необходимо более обстоятельно обсудить понятия свободной и связанной переменных [5].

В античной математике (формальные) переменные практически не использовались. Под влиянием Виета в конце 16-го столетия переменные стали стандартным инструментом математики. Было, однако, много недоразумений по поводу природы переменных. Например, целочисленная переменная была «изменяющейся величиной», которая иногда четна, иногда нечетна и может быть «зафиксирована» на время доказательства. Такие сущности, однако, не существуют.

Фреге и Пирс прояснили истинную природу переменных: это — синтаксические объекты, и следует различать свободные и связанные переменные.

Свободная переменная — это синтаксический объект, встречающийся в некотором контексте, вместо которого можно подставлять другие синтаксические объекты. Когда терм, содержащий свободные переменные, интерпретируется в некоторой структуре M , нужно выбрать элементы области M , чтобы интерпретировать эти переменные.

С другой стороны, связанные переменные не допускают ни подстановки, ни выбора интерпретации. В следующих примерах переменная x связанная:

$$\int_0^1 x^2 + 1 \, dx; \quad \sum_{x=0}^{100} x^2 + 1; \quad \exists x(x^2 + 1 = 0); \quad \lambda x.x^2 + 1.$$

Если терм t подставляется (без предосторожностей) вместо свободных вхождений переменной y в выражение $A \equiv A(y)$, то некоторые переменные в t могут стать связанными. Пусть $A(y)$ — истинная формула

$$\int_0^1 (x^2 + y) \, dx = y + \frac{1}{3},$$

и пусть $t \equiv x$; тогда $A(x)$ — ложная формула

$$\int_0^1 (x^2 + x) \, dx = x + \frac{1}{3}.$$

Это явление называется *коллизией переменных*.

Говорят, что подстановка терма t вместо y в A корректна, если никакая свободная переменная терма t не становится связанной после подстановки t вместо (свободных вхождений) переменной y в A . Именно некорректная подстановка вызвала только что упомянутую коллизию переменных. Переименовывая связанные переменные в A , мы всегда можем избежать коллизии переменных. Например, подстановка $t \equiv x$ вместо y в формулу

$$\int_0^1 (u^2 + y) du = y + \frac{1}{3}$$

является корректной.

Если бы пункт (f) в определении подстановки был опущен, то мы столкнулись бы со следующим нежелательным явлением. Хотя $\lambda v.x$ и $\lambda u.x$ обозначают одну и ту же функцию (константу, чье значение всегда есть x), после подстановки v вместо x они стали бы обозначать разные функции: $[v/x](\lambda u.x) \equiv \lambda u.v$, $[v/x](\lambda v.x) \equiv \lambda v.v$.

Мы рассмотрели, как λ -нотация может быть использована для представления функциональных выражений и сейчас готовы к тому, чтобы определить *правила конверсии* λ -исчисления, которые описывают, как вычислять выражение, т.е. как получать конечное значение выражения из его первоначального вида. Правила конверсии, описанные ниже, являются достаточно общими, так что, например, когда они применяются к λ -терму, представляющему арифметическое выражение, то они моделируют вычисление этого выражения.

Правила конверсии

- **α -конверсия.** Любая абстракция вида $\lambda V. E$ может быть конвертирована к терму $\lambda V'. [V'/V]E$. Мы переименовываем связанные переменные так, чтобы избежать коллизии переменных.
- **β -конверсия.** Любая аппликация вида $(\lambda V. E_1) E_2$ конвертируется к терму $[E_2/V]E_1$.
- **η -конверсия.** Любая абстракция вида $\lambda V. (E \ V)$, в которой V не имеет свободных вхождений в терме E может быть конвертирована к E .

Если какой-то λ -терм E_1 α -конвертируется к терму E_2 , то это обозначается как $E_1 \rightarrow_\alpha E_2$. Аналогично определяются обозначения $\rightarrow_\beta$ и $\rightarrow_\eta$. Наиболее важным видом конверсии является β -конверсия; она единственная может использоваться для моделирования произвольного вычислительного механизма. α -конверсия применяется для технических преобразований связанных переменных. Использование η -конверсии приводит к естественному свойству, что две функции, дающие один и тот же результат для любых аргументов, оказываются равными.

Обсудим более подробно правила конверсии. Ламбда-выражение, к которому может быть применено правило α -конверсии, называется *α -редексом*. «Редекс» есть аббревиатура для «редуцируемое выражение»¹⁰. Правило α -конверсии говорит, что любая связанная переменная может быть корректно переименована. Например, $\lambda x. x \rightarrow_\alpha \lambda y. y$.

Ламбда-выражение, к которому может быть применено правило β -конверсии, называется *β -редексом*. Правило β -конверсии подобно вычислению вызова функции в языке программирования: тело E_1 функции $\lambda V. E_1$ вычисляется в окружении (в контексте), в котором «формальный параметр» V заменяется на «фактический параметр» E_2 .

Примеры:

$$(\lambda x. f x) E \rightarrow_\beta f E$$

$$(\lambda x. (\lambda y. (\mathbf{add} x y))) <3> \rightarrow_\beta \lambda y. \mathbf{add} <3> y$$

$$(\lambda y. \mathbf{add} <3> y) <4> \rightarrow_\beta \mathbf{add} <3> <4>$$

Ламбда-выражение, к которому может быть применено правило η -конверсии, называется *η -редексом*.

Примеры:

$$\lambda x. \mathbf{add} x \rightarrow_\eta \mathbf{add}$$

$$\lambda y. \mathbf{add} x y \rightarrow_\eta \mathbf{add} x$$

¹⁰ redex – REDucible EXpression.

Но следующая конверсия

$$\lambda x. \mathbf{add} \ x \ x \rightarrow_{\eta} \mathbf{add} \ x$$

не имеет место, поскольку переменная x свободна в $\mathbf{add} \ x$.

Лямбда-исчисление можно рассматривать как формальную теорию с равенством. Отношение равенства формализуется с помощью следующих аксиом и правил вывода.

Равенство

Для любых λ -термов M, N, P и любой переменной x имеем:

1. Если $M \rightarrow_{\beta} N$, то $M = N$;
2. Если $M \rightarrow_{\alpha} N$, то $M = N$;
3. Если $M \rightarrow_{\eta} N$, то $M = N$;
4. $M = M$;
5. Если $M = N$, то $N = M$;
6. Если $M = N$ и $N = P$, то $M = P$;
7. Если $M = N$, то $M P = N P$;
8. Если $M = N$, то $P M = P N$;
9. ξ -правило: если $M = N$, то $\lambda x. M = \lambda x. N$.

Примеры равенств:

1. $\lambda zx.((\lambda y.y)x) = \lambda zy.y$, поскольку $\lambda zx.((\lambda y.y)x) \equiv \lambda z.(\lambda x.((\lambda y.y)x))$ и $\lambda x.(\lambda y.y)x \rightarrow_{\eta} \lambda y.y$;
2. $(\lambda zx.((\lambda y.y)x))a \ b = b$, поскольку $\lambda zx.((\lambda y.y)x) = \lambda zy.y$ и $(\lambda zy.y) \ a \ b \rightarrow_{\beta} (\lambda y.y) \ b$ и $(\lambda y.y) \ b \rightarrow_{\beta} b$.

В общефилософском смысле два свойства называются экстенционально эквивалентными, если они принадлежат в точности одним и тем же объектам. В математике, например, принят экстенциональный взгляд на множества, т.е. два множества считаются одинаковыми, если они содержат одни и те же элементы. Аналогично мы говорим, что две функции равны, если они имеют одну и ту же область определения и для любых значений аргумента из этой области определения вычисляют один и тот же

результат. Вследствие наличия η -конверсии отношение равенства лямбда-термов экстенционально.

Принцип экстенциональности. Если переменная x не свободна в M и N , то из $M\ x = N\ x$ следует $M = N$.

Доказательство. По ξ -правилу из $M\ x = N\ x$ следует $\lambda x. M\ x = \lambda x. N\ x$, и в силу η -конверсии получаем $M = N$.

Более общим понятием, по сравнению с конверсией, является понятие *обобщенной конверсии* (по причинам, которые станут понятными позже, она также называется *одношаговой редукцией*). Терм M обобщенно α -конвертируется (β -конвертируется, η -конвертируется) к терму N , если последний получается из M в результате конверсии какого-то подтерма в M , являющегося α -редексом (β -редексом, η -редексом, соответственно). Обобщенная конверсии обозначается так же как и просто конверсия: $M \rightarrow_\alpha N$ (аналогично и для других видов обобщенной конверсии).

В следующих примерах конвертируемые редексы подчеркнуты.

$$\begin{aligned} ((\lambda x. (\lambda y. (\text{add } x\ y))) <3>) <4> &\rightarrow_\beta (\lambda y. \text{add } <3> y) <4> \\ (\lambda y. \text{add } <3> y) <4> &\rightarrow_\beta \text{add } <3> <4> \end{aligned}$$

Отметим, что в первой конверсии сам терм $((\lambda x. (\lambda y. (\text{add } x\ y))) <3>) <4>$ не является редексом. Мы будем иногда писать последовательность конверсий, подобных двум предыдущим как:

$$((\lambda x. (\lambda y. (\text{add } x\ y))) <3>) <4> \rightarrow_\beta (\lambda y. \text{add } <3> y) <4> \rightarrow_\beta \text{add } <3> <4>$$

В следующих двух примерах исходное выражение одно и то же, но последовательности конвертируемых редексов различны (используемые редексы подчеркнуты):

$$\begin{aligned} 1) & (\lambda f. \lambda x. f\ <3> x) (\lambda y. \lambda x. \text{add } x\ y) <0> \\ & \rightarrow_\beta (\lambda x. (\lambda y. \lambda x. \text{add } x\ y) <3> x) <0> \\ & \rightarrow_\alpha (\lambda z. (\lambda y. \lambda x. \text{add } x\ y) <3> z) <0> \quad \text{— выбрали один из двух возможных редексов} \\ & \rightarrow_\beta (\lambda y. \lambda x. \text{add } x\ y) <3> <0> \\ & \rightarrow_\beta (\lambda z. \text{add } z\ <3>) <0> \end{aligned}$$

$\rightarrow_{\beta} \mathbf{add} \langle 0 \rangle \langle 3 \rangle$

2) $(\lambda f. \lambda x. f \langle 3 \rangle x) (\lambda y. \lambda x. \mathbf{add} x y) \langle 0 \rangle$

$\rightarrow_{\beta} (\lambda x. (\lambda y. \lambda x. \mathbf{add} x y) \langle 3 \rangle x) \langle 0 \rangle$ — выбрали один из двух возможных редексов

$\rightarrow_{\beta} (\lambda x. (\lambda x. \mathbf{add} x \langle 3 \rangle) x) \langle 0 \rangle$

$\rightarrow_{\alpha} (\lambda z. (\lambda x. \mathbf{add} x \langle 3 \rangle) z) \langle 0 \rangle$ — снова делаем произвольный выбор

$\rightarrow_{\beta} (\lambda x. \mathbf{add} x \langle 3 \rangle) \langle 0 \rangle$

$\rightarrow_{\beta} \mathbf{add} \langle 0 \rangle \langle 3 \rangle$

Заметим, что в данном случае независимо от выбора редексов мы пришли к одинаковому результату.

Нормальные формы

Если λ -выражение E' получается из λ -выражения E с помощью последовательности обобщенных конверсий, то естественно считать, что E' получается в результате «вычисления» E , которое более точно выражается через понятие *редукции*.

Определение отношения редукции

Пусть E и E' есть λ -выражения. Говорят, что терм E редуцируется к терму E' (и обозначают как $E \rightarrow E'$), если $E \equiv E'$ или существуют выражения $E_1, E_2, \dots, E_n$ такие, что:

1. $E \equiv E_1$

2. $E_n \equiv E'$

3. Для каждого i выполнено одно из трех отношений $E_i \rightarrow_{\alpha} E_{i+1}$, $E_i \rightarrow_{\beta} E_{i+1}$ или $E_i \rightarrow_{\eta} E_{i+1}$.

Последний пример предыдущего пункта говорит, что

$$(\lambda f. \lambda x. f \langle 3 \rangle x) (\lambda y. \lambda x. \mathbf{add} x y) \langle 0 \rangle \rightarrow \mathbf{add} \langle 0 \rangle \langle 3 \rangle$$

Три правила конверсии сохраняют значения λ -выражений, т.е. если терм E редуцируется к терму E' , то имеем $E = E'$. Это следствие нашего определения равенства λ -выражений.

Несмотря на то, что термин «редукция» подразумевает уменьшение размера лямбда-терма, в действительности это может быть не так, что показывает следующий пример:

$$\begin{aligned} (\lambda x. xxx) (\lambda x. xxx) &\rightarrow (\lambda x. xxx) (\lambda x. xxx) (\lambda x. xxx) \\ &\rightarrow (\lambda x. xxx) (\lambda x. xxx) (\lambda x. xxx) (\lambda x. xxx) \\ &\rightarrow \dots \end{aligned}$$

Говорят, что λ -выражение находится в *нормальной форме*, если к нему нельзя применить никакое β - или η -правило конверсии. Другими словами, λ -выражение — в нормальной форме, если оно не содержит β - или η -редексов. Нормальная форма, таким образом, соответствует понятию конца вычислений в традиционном программировании. Отсюда немедленно вытекает наивная схема вычислений:

*while существует хотя бы один β - или η -редекс
do преобразовывать один из редексов
end
(выражение теперь в нормальной форме).*

Будем говорить, что λ -выражение M имеет **нормальную форму**, если существует выражение N в нормальной форме, причем $M = N$.

Различные варианты конверсии в процессе редукции могут приводить к принципиально различным последствиям.

Пример (конвертируемые редексы подчеркнуты):

$$\begin{aligned} &(\lambda x. \lambda y. y) ((\lambda z. \underline{z z}) (\lambda z. \underline{z z})) \\ &\rightarrow (\lambda x. \lambda y. y) ((\lambda z. \underline{z z}) (\lambda z. \underline{z z})) \\ &\rightarrow (\lambda x. \lambda y. y) ((\lambda z. \underline{z z}) (\lambda z. \underline{z z})) \\ &\rightarrow \dots \\ &\text{(бесконечный процесс редукции)} \\ &(\lambda x. \lambda y. y) ((\lambda z. \underline{z z}) (\lambda z. \underline{z z})) \\ &\rightarrow \lambda y. y \\ &\text{(редукция закончилась)} \end{aligned}$$

Порядок редукций (стратегия выбора редексов)

Самым левым редексом называется редекс, первый символ которого текстуально расположен в λ -выражении левее всех остальных редексов. (Аналогично определяется *самый правый редекс*.)

Самым внешним редексом называется редекс, который не содержится внутри никакого другого редекса.

Самым внутренним редексом называется редекс, не содержащий других редексов.

В контексте функциональных языков и λ -исчисления существуют два важных порядка редукций [8].

Аппликативный порядок редукций (АПР), который предписывает вначале преобразовывать самый левый из самых внутренних редексов.

Нормальный порядок редукций (НПР), который предписывает вначале преобразовывать самый левый из самых внешних редексов.

В терме $(\lambda x. \lambda y. y) ((\lambda z. zz) (\lambda z. z z))$ подчеркнут самый левый из самых внутренних редексов — если его последовательно конвертировать, то редукция никогда не закончится.

В терме $(\lambda x. \lambda y. y) ((\lambda z. z z) (\lambda z. zz))$ подчеркнут самый левый из самых внешних редексов — редукция в этом случае заканчивается за один шаг.

Функция $\lambda x. \lambda y. y$ — это классический пример функции, которая отбрасывает свой аргумент. НПР в таких случаях эффективно откладывает вычисление любых редексов внутри выражения аргумента до тех пор, пока это возможно, в расчете на то, что такое вычисление может оказаться ненужным.

В функциональных языках стратегии НПР соответствуют так называемые *ленивые вычисления*. «Не делай ничего, пока это не потребуется» — механизм вызова по необходимости (все аргументы передаются функции в не вычисленном виде и вычисляются только тогда, когда в них возникает необходимость внутри тела функции). Haskell — один из языков с ленивыми вычислениями.

Стратегия АПР приводит к *энергичным вычислениям*. «Делай все, что можешь»; другими словами, не надо заботиться

о том, пригодится ли, в конечном счете, полученный результат. Таким образом, реализуется механизм вызова по значению (значение аргумента передается в тело функции). В языке Лисп реализуются, как правило, энергичные вычисления.

Теорема Чёрча-Россера [5, с.78]. Если $E \rightarrow E_1$ и $E \rightarrow E_2$, то существует терм E' , такой, что $E_1 \rightarrow E'$ и $E_2 \rightarrow E'$.

Следствие 1. Если $E_1 = E_2$, то существует терм E' , такой, что $E_1 \rightarrow E'$ и $E_2 \rightarrow E'$.

Следствие 2. Если выражение E может быть приведено двумя различными способами к двум нормальным формам, то эти нормальные формы совпадают или могут быть получены одна из другой с помощью замены связанных переменных.

Теорема стандартизации [5, с. 298–303]. Если выражение E имеет нормальную форму, то НПР гарантирует достижение этой нормальной формы (с точностью до замены связанных переменных).

Приведем сводку наших знаний о нормальных формах λ -выражений.

Любое ли выражение имеет нормальную форму?	Нет.
Влияет ли порядок редукции на достижение нормальной формы?	Да. ПР гарантирует достижение нормальной формы выражения, если она существует.
Как соотносятся две нормальные формы, полученные различными β - или η -конверсиями из одного выражения?	Эти нормальные формы совпадают или отличаются с точностью до α -конверсии.

Если два λ -выражения находятся в нормальной форме и не совпадают (даже после α -конверсии), то эти λ -выражения не равны. Это утверждение следует из предыдущих выводов.

Комбинаторы

Теория комбинаторов была развита российским математиком Шейфинкелем [3] перед тем, как лямбда-обозначения были введены Чёрчем. Вскоре Карри переоткрыл эту теорию независимо

от Шейфинкеля и Чёрча. Теорию комбинаторов, так же как и λ -исчисление, можно использовать для представления функций. Комбинаторы также обеспечивают хороший промежуточный код для обычных компьютеров: несколько лучших компиляторов для ленивых функциональных языков базируются на комбинаторах [8].

Существует два эквивалентных способа формализации теории комбинаторов:

- 1) внутри λ -исчисления;
- 2) как полностью независимую теорию.

Следуя первому подходу, определим **комбинатор** просто как λ -терм, не имеющий свободных переменных. Такой терм также называется замкнутым, — он имеет фиксированное значение независимо от значения любой переменной.

Следующие комбинаторы имеют традиционное обозначение и часто встречаются в литературе, посвященной λ -исчислению.

$$\begin{aligned}
 \mathbf{B} &\equiv \lambda xyz . x(yz) \\
 \mathbf{C} &\equiv \lambda xyz . xzy \\
 \mathbf{I} &\equiv \lambda x . x \\
 \mathbf{K} &\equiv \lambda xy . x \\
 \mathbf{M} &\equiv \lambda x . xx \\
 \mathbf{N} &\equiv \lambda fghx . f(gx)(hx) \\
 \mathbf{S} &\equiv \lambda xyz . xz(yz) \\
 \mathbf{W} &\equiv \lambda xy . xyx \\
 \mathbf{Y} &\equiv \lambda f . ((\lambda x . f(xx)) (\lambda x . f(xx)))
 \end{aligned}$$

Из этих определений, используя β -конверсию, получаем, что для любых термов E_1, E_2 и E_3 выполнено

$$\begin{aligned}
 \mathbf{B} E_1 E_2 E_3 &= E_1 (E_2 E_3) \\
 \mathbf{C} E_1 E_2 E_3 &= E_1 E_3 E_2 \\
 \mathbf{I} E_1 &= E_1 \\
 \mathbf{K} E_1 E_2 &= E_1 \\
 \mathbf{M} E_1 &= E_1 E_1 \\
 \mathbf{N} E_1 E_2 E_3 E_4 &= E_1 (E_2 E_4) (E_3 E_4) \\
 \mathbf{S} E_1 E_2 E_3 &= E_1 E_3 (E_2 E_3) \\
 \mathbf{W} E_1 E_2 &= E_1 E_2 E_2
 \end{aligned}$$

Понятие *базиса* в λ -исчислении аналогично соответствующему понятию в линейной алгебре: базис B для векторного пространства W есть конечное множество векторов, чье замыкание относительно операций сложения и скалярного умножения равно W . Ситуация в λ -исчислении похожа: **базис** B для множества термов W есть множество λ -термов (которое не обязано быть конечным), чье замыкание относительно операции аппликации равно W .

Следствием этого подобия понятий являются такие же вопросы для λ -исчисления, какие возникают и в линейной алгебре: «Как представить произвольный λ -терм в данном базисе?» и «Как мал может быть базис для представления всех комбинаторов?»

Замыкание относительно аппликации

Пусть S — произвольное множество λ -термов. Множество S^+ термов называется *замыканием множества S относительно операции аппликации*, если оно есть наименьшее множество W , удовлетворяющее условиям:

- 1) $S \subseteq W$.
- 2) Для любых $M, N \in W$ имеем $(M N) \in W$.

Базис

Множество термов B называется *базисом* для множества термов L , если для любого $M \in L$ существует $N \in B^+$, такой, что $M = N$.

Алгоритм абстракции

Пусть множество термов B есть базис для множества термов L . Алгоритм, который для каждого $M \in L$ выдает соответствующий терм $N \in B^+$, называется *алгоритмом абстракции*.

Теорема.

1. Множество $B = \{I, K, B, C, S\} \cup \{\text{переменные}\}$ является базисом для множества Λ всех λ -выражений.

2. Множество $\{K, S\}$ является базисом для множества всех комбинаторов.

Доказательство. Определим алгоритм абстракции в виде отображения $comb: \Lambda \rightarrow B^+$ в виде рекурсивных правил.

1. $comb(E) \stackrel{def}{=} E$, если $E \in B$.
2. $comb(E_1 E_2) \stackrel{def}{=} comb(E_1) comb(E_2)$.
3. $comb(\lambda x . E) \stackrel{def}{=} abs(x, comb(E))$.

Отображение $abs: \{\text{переменные}\} \times B^+ \rightarrow B^+$ также задается в виде рекурсивных правил ($FreeVars(E)$ обозначает множество свободных переменных терма E).

1. $abs(x, x) \stackrel{def}{=} I$.
2. $abs(x, E) \stackrel{def}{=} K E$, если $E \in B \setminus \{x\}$.
3. $abs(x, E_1 E_2) \stackrel{def}{=} C abs(x, E_1) E_2$, если $x \in FreeVars(E_1)$ и $x \notin FreeVars(E_2)$.
4. $abs(x, E_1 E_2) \stackrel{def}{=} B E_1 abs(x, E_2)$, если $x \notin FreeVars(E_1)$ и $x \in FreeVars(E_2)$.
5. $abs(x, E_1 E_2) \stackrel{def}{=} S abs(x, E_1) abs(x, E_2)$, если $x \in FreeVars(E_1)$ и $x \in FreeVars(E_2)$.

Для доказательства утверждения 1 теоремы мы должны доказать, что для любого λ -выражения E имеем равенство

$$comb(E) = E. \quad (1)$$

Заметим, что из определения отображения abs следует, что для произвольной переменной x и произвольного аппликативного выражения $E \in B^+$ значение отображения $abs(x, E)$ не содержит переменной x .

Для доказательства (1) нам понадобятся два вспомогательных утверждения. Сначала докажем, что для произвольной переменной x и произвольного аппликативного выражения $E \in B^+$ имеем равенство

$$abs(x, E) x = E. \quad (2)$$

Кроме того, покажем, что для произвольной переменной x и произвольного аппликативного выражения $E \in B^+$ имеем равенство

$$abs(x, E) = \lambda x . E. \quad (3)$$

Равенство (2) будем доказывать, используя индукцию по построению терма.

Базис индукции.

Случай 1 для abs . Имеем $abs(x, x) x \stackrel{def}{=} I x = x$.

Случай 2 для abs . Имеем $abs(x, E) x \stackrel{def}{=} K E x = E$.

Индуктивный переход.

Случай 3 для abs . $abs(x, E_1 E_2) x \stackrel{def}{=} C abs(x, E_1) E_2 x = abs(x, E_1) x E_2 = (abs(x, E_1) x) E_2 =$ (по индуктивному предположению) $E_1 E_2$.

Случай 4 для abs . $abs(x, E_1 E_2) x \stackrel{def}{=} B E_1 abs(x, E_2) x = E_1 (abs(x, E_2) x) =$ (по индуктивному предположению) $E_1 E_2$.

Случай 5 для abs . $abs(x, E_1 E_2) x \stackrel{def}{=} S abs(x, E_1) abs(x, E_2) x = abs(x, E_1) x (abs(x, E_2) x) =$ (по индуктивному предположению) $E_1 E_2$. Таким образом, равенство (2) справедливо.

Теперь докажем (3). Применяя правило ξ к (2), получаем $\lambda x . abs(x, E) x = \lambda x . E$. С другой стороны, так как, $abs(x, E)$ не имеет вхождений переменной x , то в силу η -конверсии $\lambda x . abs(x, E) x = abs(x, E)$. Поэтому имеем $abs(x, E) = \lambda x . E$.

Доказательство равенства (1) проведем индукцией по построению λ -терма. Базис индукции (случай 1 для $comb$) справедлив по определению.

Индуктивный переход. Случай 2 для $comb$: $comb(E_1 E_2) \stackrel{def}{=} comb(E_1) comb(E_2) = E_1 E_2$, по индуктивному предположению.

Случай 3 для $comb$: $comb(\lambda x . E) \stackrel{def}{=} abs(x, comb(E)) =$ (по равенству (3)) $\lambda x . comb(E) =$ (по индуктивному предположению) $\lambda x . E$.

Утверждение 1 доказано. Заметим, что все связанные переменные в исходном λ -терме при переводе в комбинаторную форму исчезают, а новые переменные появиться не могут, поэтому любой замкнутый терм представляется в виде аппликативного выражения только из комбинаторов I, K, B, C, S .

Утверждение 2 усиливает последний вывод. Для его справедливости достаточно обнаружить, что комбинаторы I , B , C можно представить в виде некоторых комбинаций только двух комбинаторов K и S . Но это действительно так:

$$\begin{aligned} I &= SKK, \\ B &= S(KS)K, \\ C &= S(BBS)(KK) = S(S(KS)K(S(KS)K)S)(KK). \end{aligned} \tag{4}$$

Для доказательства этих равенств воспользуемся экстенсionalностью. Пусть x , y и z — произвольные переменные. Если мы покажем, что

$$\begin{aligned} \text{a) } Ix &= SKKx, \\ \text{b) } Bxyz &= S(KS)Kxyz, \\ \text{c) } Cxyz &= S(S(KS)K(S(KS)K)S)(KK)xyz, \end{aligned}$$

то, в силу принципа экстенсionalности, равенства (4) будут доказаны.

а. Комбинатор I обладает свойством $Ix = x$. С другой стороны, $SKKx = Kx(Kx)$ (по свойству комбинатора S) $= x$ (по свойству комбинатора K).

б. Комбинатор B обладает свойством $Bxyz = x(yz)$. С другой стороны,

$$\begin{aligned} S(KS)Kxyz &= \\ (\text{правило для } S) KSx (Kx)yz &= \\ (\text{правило для } K) S(Kx)yz &= \\ (\text{правило для } S) Kxz (yz) &= \\ (\text{правило для } K) x(yz). \end{aligned}$$

с. Комбинатор C обладает свойством $Cxyz = xzy$. С другой стороны,

$$\begin{aligned} S(S(KS)K(S(KS)K)S)(KK)xyz &= \\ S(BBS)(KK)xyz &= \\ (\text{правило для } S) BBSx (KKx)yz &= \\ (\text{правило для } K) BBSxKyz &= \\ (\text{правило для } B) B(Sx)Kyz &= \\ (\text{правило для } B) (Sx)(Ky)z &= \\ (\text{правило для } S) xz (Kyz) &= \\ (\text{правило для } K) xzy. \end{aligned}$$

Доказательство теоремы закончено.

Пример трансляции терма $\lambda xy . xyx$ в комбинаторную форму.

$$\begin{aligned}
 comb(\lambda xy . xyx) &\equiv \\
 comb(\lambda x.(\lambda y . xyx)) &= \\
 abs(x, comb(\lambda y . xyx)) &= \\
 abs(x, abs(y, comb(xyx))) &= \\
 abs(x, abs(y, comb(xy) comb(y))) &= \\
 abs(x, abs(y, (comb(x) comb(y)) comb(y))) &= \\
 abs(x, abs(y, (xy)y)) &= \\
 abs(x, S abs(y, xy) abs(y, y)) &= \\
 abs(x, S (Bx abs(y, y)) abs(y, y)) &= \\
 abs(x, S (BxI) I) &\equiv \\
 abs(x, (S (BxI)) I) &= \\
 C abs(x, S (BxI)) I &= \\
 C (BS abs(x, BxI)) I &= \\
 C (BS (C abs(x, Bx) I)) I &= \\
 C (BS (C (BB abs(x, x)) I)) I &= \\
 C (BS (C (BB I) I)) I.
 \end{aligned}$$

При переводе λ -выражений используется отображение abs с пятью правилами. Можно использовать упрощенную версию abs с правилами:

1. $abs(x, x) \stackrel{def}{=} I$.
2. $abs(x, E) \stackrel{def}{=} K E$, если $E \in B \setminus \{x\}$.
3. $abs(x, E_1 E_2) \stackrel{def}{=} S abs(x, E_1) abs(x, E_2)$.

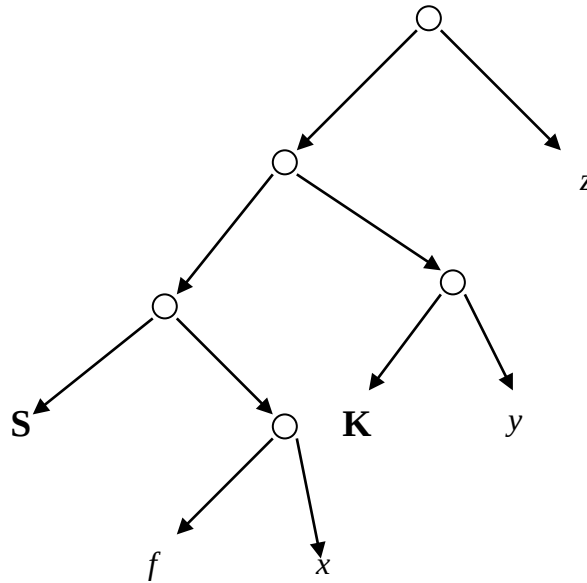
В этом случае трансляция приводит сразу только к комбинаторам I , K и S .

Убедимся, что правило $abs(x, E_1 E_2) \stackrel{def}{=} S abs(x, E_1) abs(x, E_2)$ заменяет как правило для случая B , так и правило для случая C .

Используем экстенциональность:

Имеем $S abs(x, E_1) abs(x, E_2) y = abs(x, E_1) y (abs(x, E_2) y) = (\lambda x. E_1) y ((\lambda x. E_2) y)$ (равенство (3) в теореме). Если $x \notin FreeVars(E_1)$, то $(\lambda x. E_1) y ((\lambda x. E_2) y) = E_1((\lambda x. E_2) y)$. Если же $x \notin FreeVars(E_2)$, то $(\lambda x. E_1) y ((\lambda x. E_2) y) = (\lambda x. E_1) y E_2$.

С другой стороны, $C \text{ abs}(x, E_1) E_2 y = C (\lambda x. E_1) E_2 y = (\lambda x. E_1) y E_2$ и $B E_1 \text{ abs}(x, E_2) y = B E_1 (\lambda x. E_2) y = E_1((\lambda x. E_2) y)$.



До работ Давида Тёрнера (David Turner) комбинаторы рассматривались как факт чистой математики. Д. Тёрнер предложил (см., например, [4]) транслировать λ -выражения в комбинаторы с дальнейшим редуцированием как практический способ реализации функциональных языков.

Идея Тёрнера заключалась в представлении комбинатора в виде дерева. Например, комбинатор $S (f x) (K y) z$ представляется как показано на рисунке выше.

Такие структуры можно реализовать в памяти компьютера с помощью ссылок.

Таким образом, вычисление λ -выражения сводится к следующим шагам:

- преобразование λ -терма к комбинаторному виду;
- представление комбинатора в виде дерева;
- специальное программное обеспечение (лучше аппаратное обеспечение) преобразовывает такие деревья (до тех пор, пока это возможно) в соответствии с правилами для I , K , B , C , S — комбинаторов.

6.2 Лямбда-исчисление как язык программирования

В описанном виде λ -исчисление является примитивным языком. Однако его можно использовать для представления объектов и структур современного языка программирования. Идея состоит в представлении этих объектов и структур в виде λ -выражений таким образом, чтобы они обладали требуемыми свойствами.

Например, для языка программирования необходимо, как минимум, иметь данные, представляющие натуральные числа. Поэтому будут введены комбинаторы, обозначающие натуральные числа. Требуется определение основных арифметических операций: сложение, умножение, тест на ноль и т.д. Также, конечно, необходимы истинностные значения и булевские операции. Введение новых обозначений для лямбда-термов, моделирующих необходимые для языка программирования структуры данных и функции, является «синтаксическим сахаром».

Выбор λ -выражений для представления программных объектов достаточно произволен, но все предлагаемые определения согласованы так, что они могут работать в унисон. Поиск подходящих комбинаторов требует некоторого искусства. Впрочем, мы стараемся указать те идеи, которые приводят к предложенным определениям комбинаторов.

Нумералы Чёрча

Мы начнем с представления натуральных чисел в лямбда-исчислении. Соответствующие комбинаторы для представления натуральных чисел ввел Алонзо Чёрч, и поэтому они называются в его честь.

Введем новое обозначение. Для произвольных термов A, B и натурального $n > 0$ пусть $A^n B$ обозначает n -кратную аппликацию: $A(A(\dots(A B)\dots))$, где A повторяется n раз. Условимся также, что $A^0 B$ обозначает просто B . Таким образом, выражение $A^n B$ определено для любого целого $n \geq 0$.

Предположим, что мы имеем некоторое λ -выражение z , представляющее число 0, и λ -выражение s , представляющее

функцию следования $s: n \rightarrow n+1$ на множестве натуральных чисел. Тогда аппликация $(s\ z)$ представляет число 1, $(s\ (s\ z))$ представляет число 2 и т.д. Поэтому n -ый нумерал Чёрча определяется с помощью абстракции над n -кратной аппликацией s к z :

Определение: нумералы Чёрча.

$$\langle 0 \rangle \equiv \lambda sz . z$$

$$\langle n+1 \rangle \equiv \lambda sz . \underbrace{(s \dots (s\ z) \dots)}_{n+1 \text{ раз}} = \lambda sz . (s^{n+1} z)$$

Нумералы Чёрча называются также **итераторами**. Из определения следует, что для произвольных термов M и E имеем

$$\langle 0 \rangle M E \rightarrow E$$

$$\langle n+1 \rangle M E \rightarrow \underbrace{M(M(\dots(M\ E)\dots))}_{n+1 \text{ раз}}$$

Таким образом, видим, что итератор $\langle n \rangle$ применяет n раз терм M к терму E . Имеем следующее важное свойство:

$$\langle n \rangle M E \rightarrow M^n E \quad (1)$$

Используя определение нумералов Чёрча, определим элементарные функции над числами, начиная с функции следования. Пусть x и y — произвольные переменные. Имеем, по определению, $\langle n \rangle x y \rightarrow x^n y$. Применяя x к обеим сторонам, получаем

$$x (\langle n \rangle x y) \rightarrow x (x^n y).$$

Применяя абстракцию с переменными x и y к обеим частям равенства, получаем

$$\lambda xy . x (\langle n \rangle x y) \rightarrow \lambda xy . x (x^n y),$$

что дает

$$\langle n+1 \rangle = \lambda xy . x (\langle n \rangle x y).$$

Если терм $\langle n \rangle$ заменить переменной n и провести абстракцию относительно этой переменной, то получаем комбинатор, представляющий функцию $\langle n \rangle \rightarrow \langle n+1 \rangle$.

Комбинатор **succ** $\equiv \lambda nxy . x (n x y)$ представляет функцию следования на нумералах Чёрча.

Проверим:

$$\begin{aligned}
 & \mathbf{succ} \langle n \rangle x y \\
 & \equiv (\lambda nxy . x (n x y)) \langle n \rangle \\
 & \rightarrow \lambda xy . x (\langle n \rangle x y) \\
 & \rightarrow (\text{в силу (1)}) \lambda xy . x (x^n y) \\
 & \equiv \langle n+1 \rangle
 \end{aligned}$$

Комбинатор **succ** равен простой комбинации **SB** основных комбинаторов. Докажем это, используя экстенциональность.

$$\mathbf{SB} \, nxy = \mathbf{B}x(nx)y = x(nxy) = \mathbf{succ} \, nxy$$

Мы сейчас применим отношение (1), чтобы определить комбинатор, с помощью которого представляется сложение нумералов Чёрча: терм $\langle n \rangle \mathbf{succ}$ обозначает функцию, которая n раз увеличивает свой аргумент на 1. Таким образом,

$$(\langle a \rangle \mathbf{succ}) \langle b \rangle \rightarrow \langle a+b \rangle. \quad (2)$$

Заменим $\langle a \rangle$ и $\langle b \rangle$ на соответствующие переменные a и b и, применяя абстракцию относительно переменных a и b над левой частью (2), мы получаем комбинатор сложения:

Комбинатор **add** $\equiv \lambda ab. a \mathbf{succ} b$ представляет функцию сложения над нумералами Чёрча.

Проверим:

$$\begin{aligned}
 & \mathbf{add} \langle a \rangle \langle b \rangle \\
 & \equiv (\lambda ab. a \mathbf{succ} b) \langle a \rangle \langle b \rangle \\
 & \rightarrow \langle a \rangle \mathbf{succ} \langle b \rangle \\
 & \rightarrow \langle a+b \rangle.
 \end{aligned}$$

Комбинатор **NB** также представляет сложение нумералов Чёрча. Проверим это. Сначала вычислим выражение

$$\begin{aligned} & \mathbf{NB}mnfx \\ &= \mathbf{B}(mf)(nf)x \\ &= (mf)(nfx) \end{aligned}$$

Поэтому имеем

$$\begin{aligned} & \mathbf{NB}\langle m \rangle \langle n \rangle fx \\ &= (\langle m \rangle f)(\langle n \rangle fx) \\ & \text{(используя свойство (1))} \\ &= (\langle m \rangle f)(f^n x) \\ & \text{(используя свойство (1))} \\ &= f^m(f^n x) \\ &\equiv f^{m+n} x. \end{aligned}$$

Но точно такое же выражение мы получаем при вычислении $\langle m+n \rangle f x \equiv (\lambda sz. s^{m+n} z) f x = f^{m+n} x$. Поэтому, в силу экстенсionalности, $\mathbf{NB}\langle m \rangle \langle n \rangle = \langle m+n \rangle$.

Хотя на нумералах Чёрча два комбинатора **add** и **NB** действуют одинаково, они не равны. Докажем от противного. Из равенства $\mathbf{add} = \mathbf{NB}$ следует равенство $\mathbf{add} K hxy = \mathbf{NBK} hxy$. Но имеем

$$\begin{aligned} & \mathbf{NBK}hxy \\ &= \mathbf{B}(Kx)(hx)y \\ &= Kx(hxy) \\ &= x \end{aligned}$$

С другой стороны,

$$\begin{aligned} & \mathbf{add} K hxy \\ &\equiv (\lambda ab. a (\mathbf{SB}) b) K hxy \\ &= K (\mathbf{SB}) hxy \\ &= \mathbf{SB}xy \\ &= \mathbf{B}y(xy) \\ &= (\lambda xyz. x(yz)) y(xy) \\ &= \lambda z. y(xyz) \end{aligned}$$

Термы x и $\lambda z. y(xyz)$ находятся в нормальной форме и не совпадают, что противоречит следствию из теоремы Чёрча-Россера.

Понятно, что **add** $\langle a \rangle$ представляет функцию, которая прибавляет к своему аргументу нумерал $\langle a \rangle$. Следовательно, $\langle b \rangle$ (**add** $\langle a \rangle$) делает такое сложение b раз, и поэтому

$$\langle b \rangle (\text{add } \langle a \rangle) \langle 0 \rangle \rightarrow \langle a \times b \rangle. \quad (3)$$

Заменим $\langle a \rangle$ и $\langle b \rangle$ на соответствующие переменные a и b , и, применяя абстракцию относительно переменных a и b над левой частью (3), мы получаем комбинатор умножения:

Комбинатор **times** $\equiv \lambda ab. b (\text{add } a) \langle 0 \rangle$ представляет функцию умножения над нумералами Чёрча.

Проверим:

$$\begin{aligned} & \text{times } \langle a \rangle \langle b \rangle \\ & \equiv (\lambda ab. b (\text{add } a) \langle 0 \rangle) \langle a \rangle \langle b \rangle \\ & = \langle b \rangle (\text{add } \langle a \rangle) \langle 0 \rangle \\ & = (\text{add } \langle a \rangle)^b \langle 0 \rangle \\ & \equiv \underbrace{\text{add } \langle a \rangle (\text{add } \langle a \rangle (\dots (\text{add } \langle a \rangle \langle 0 \rangle) \dots))}_{b \text{ раз}} \\ & = \langle a \times b \rangle \end{aligned}$$

Комбинатор **B** также представляет умножение нумералов Чёрча. Проверим это. Пусть x и y — переменные. Тогда **B** $\langle n \rangle \langle m \rangle x y = \langle n \rangle (\langle m \rangle x) y$. Так как $\langle n \rangle$ и $\langle m \rangle$ — итераторы, то $\langle n \rangle (\langle m \rangle x)$ есть функция, которая применяет терм $(\langle m \rangle x)$ n раз к терму y . Но $\langle m \rangle x$ есть m -кратная аппликация x , поэтому $\langle n \rangle (\langle m \rangle x)$ применяет x к y всего $m \times n$ раз. Получили **B** $\langle n \rangle \langle m \rangle x y = x^{m \times n} y$, но имеем также $\langle n \times m \rangle x y = x^{m \times n} y$, поэтому по экстенсionalità **B** $\langle n \rangle \langle m \rangle = \langle n \times m \rangle$.

Хотя на нумералах Чёрча два комбинатора **times** и **B** действуют одинаково, они не равны. Докажем от противного.

Из равенства $\mathbf{times} = \mathbf{B}$ следует равенство $\mathbf{times} \langle 1 \rangle \mathbf{K} \langle 0 \rangle = \mathbf{B} \langle 1 \rangle \mathbf{K} \langle 0 \rangle$. Но имеем

$$\begin{aligned} & \mathbf{times} \langle 0 \rangle \mathbf{K} \langle 0 \rangle \\ & \equiv (\lambda ab. b (\mathbf{add} a) \langle 0 \rangle) \langle 0 \rangle \mathbf{K} \langle 0 \rangle \\ & \rightarrow \mathbf{K} (\mathbf{add} \langle 0 \rangle) \langle 0 \rangle \langle 0 \rangle \\ & \rightarrow (\mathbf{add} \langle 0 \rangle) \langle 0 \rangle \\ & \rightarrow \langle 0 \rangle \end{aligned}$$

$$\begin{aligned} & \text{С другой стороны,} \\ & \mathbf{B} \langle 0 \rangle \mathbf{K} \langle 0 \rangle \\ & = \langle 0 \rangle (\mathbf{K} \langle 0 \rangle) \\ & \equiv (\lambda xy. y) (\mathbf{K} \langle 0 \rangle) \\ & \rightarrow \lambda y. y \end{aligned}$$

Термы $\langle 0 \rangle$ и $\lambda y. y$ находятся в нормальной форме и не совпадают, что противоречит следствию из теоремы Чёрча-Россера.

Понятно, что $\mathbf{times} \langle a \rangle$ представляет функцию, которая умножает свой аргумент на нумерал $\langle a \rangle$. Следовательно, $\langle b \rangle (\mathbf{times} \langle a \rangle)$ делает такое умножение b раз, и поэтому

$$\langle b \rangle (\mathbf{times} \langle a \rangle) \langle 1 \rangle \rightarrow \langle a^b \rangle. \quad (4)$$

Заменим $\langle a \rangle$ и $\langle b \rangle$ на соответствующие переменные a и b , и, применяя абстракцию относительно переменных a и b над левой частью (4), мы получаем комбинатор для возведения в степень:

Комбинатор $\mathbf{power} \equiv \lambda ab. b (\mathbf{times} a) \langle 1 \rangle$ представляет возведение в степень над нумералами Чёрча.

Проверим:

$$\begin{aligned} & \mathbf{power} \langle a \rangle \langle b \rangle \\ & \equiv (\lambda ab. b (\mathbf{times} a) \langle 1 \rangle) \langle a \rangle \langle b \rangle \\ & = \langle b \rangle (\mathbf{times} \langle a \rangle) \langle 1 \rangle \\ & = (\mathbf{times} \langle a \rangle)^b \langle 1 \rangle \\ & \equiv \underbrace{\mathbf{times} \langle a \rangle (\mathbf{times} \langle a \rangle (\dots (\mathbf{times} \langle a \rangle \langle 1 \rangle) \dots))}_{b \text{ раз}} = \langle a^b \rangle \end{aligned}$$

Истинностные значения и условное выражение

Определим булевские значения *true*, *false* и условное выражение *if* так, чтобы выполнялись следующие свойства:

$$\begin{aligned} \text{if } \text{true } A \ B &= A \\ \text{if } \text{false } A \ B &= B \end{aligned} \quad (1)$$

Существуют много различных способов для представления истинностных значений — для кодирования булевских значений «истина» и «ложь» можно использовать любые два неравные лямбда-термы. Здесь предлагаются традиционные комбинаторные определения.

$$\begin{aligned} \text{true} &\equiv K \\ \text{false} &\equiv KI \end{aligned}$$

Как известно,

$$K A B = A$$

Комбинатор *KI*, в свою очередь, отбрасывает свой первый аргумент:

$$(KI) A B \equiv (KI A) B = I B = B$$

Комбинатор *if* должен удовлетворять уравнениям (1); если *if* будет обладать свойством $\text{if } M A B = M A B$ для произвольных комбинаторов *M*, *A* и *B*, то тогда для *if* будет выполнено и (1). Поэтому условное выражение определяем следующим образом

$$\text{if} \equiv \lambda x. \lambda y. \lambda z. x y z$$

В комбинаторной записи $\text{if} = I$. Это легко доказать, используя экстенциональность.

Комбинаторное выражение $M A B$ легко вычисляется, если M равно **true** или **false**. Это наблюдение помогает определить комбинатор **not** для операции отрицания. Тогда **not** есть решение уравнения $\text{not } A = A \text{ false true}$. Пусть **not** обладает данным свойством, тогда

$$\begin{aligned}\text{not true} &= \text{true false true} = \text{false}, \\ \text{not false} &= \text{false false true} = \text{true}.\end{aligned}$$

Поэтому **not** определяем следующим образом:

$\text{not} \equiv \lambda x. x \text{ false true}$

В комбинаторной записи $\text{not} = S(SI(K(KI)))(KK)$.

Комбинаторы для операций конъюнкции и дизъюнкции также являются решениями соответствующих уравнений:

$$\begin{aligned}\text{and } A B &= A B \text{ false}, \\ \text{or } A B &= A \text{ true } B.\end{aligned}$$

Поэтому комбинаторы **and** и **or** определяем следующим образом:

$\text{and} \equiv \lambda xy. x y \text{ false}$
$\text{or} \equiv \lambda xy. x \text{ true } y$

В комбинаторной записи

$$\begin{aligned}\text{and} &= SS(K(K(KI))) \\ \text{or} &= SI(KK)\end{aligned}$$

Пары и кортежи

Следующие комбинаторы используются для представления пар в λ -исчислении.

$$\begin{aligned}
fst &\equiv \lambda x. x \text{ true} \\
snd &\equiv \lambda x. x \text{ false} \\
(E_1, E_2) &\equiv \lambda f. f \ E_1 \ E_2
\end{aligned}$$

Выражение (E_1, E_2) представляет упорядоченную пару, причем доступ к первой компоненте осуществляется с помощью функции ***fst***, а вторую компоненту возвращает функция ***snd***. Проверим:

$$\begin{aligned}
&fst \ (E_1, E_2) \\
&\equiv (\lambda x. x \ \text{true}) \ (\lambda f. f \ E_1 \ E_2) \\
&\rightarrow_{\beta} (\lambda f. f \ E_1 \ E_2) \ \text{true} \\
&\rightarrow_{\beta} \text{true} \ E_1 \ E_2 \\
&\rightarrow_{\beta} (\lambda x. \lambda y. x) \ E_1 \ E_2 \\
&\rightarrow_{\beta} (\lambda y. E_1) \ E_2 \\
&\rightarrow_{\beta} E_1
\end{aligned}$$

Пара есть структура данных с двумя компонентами. Обобщенная структура с n компонентами называется n -кой или *кортежем* и легко определяется через пары.

$$(E_1, E_2, \dots, E_n) \equiv (E_1, (E_2, (\dots(E_{n-1}, E_n)\dots)))$$

$(E_1, E_2, \dots, E_n)$ есть n -кортеж с компонентами $E_1, E_2, \dots, E_n$ и длиной n . Пары суть кортежи длиной 2. С помощью следующих выражений получаем доступ к отдельным компонентам кортежа.

$$\begin{aligned}
E \downarrow 1 &\equiv fst \ E \\
E \downarrow 2 &\equiv fst \ (snd \ E) \\
E \downarrow i &\equiv fst \ (snd \ (\underbrace{snd \ (\dots(snd \ E)\dots)}_{i-1 \text{ раз}})), \text{ если } i \text{ меньше длины } E \\
E \downarrow n &\equiv snd \ (\underbrace{snd \ (\dots(snd \ E)\dots)}_{n-1 \text{ раз}}), \text{ если } n \text{ равно длине } E
\end{aligned}$$

Проверим, что эти определения правильно работают:

$$\begin{aligned} & (E_1, E_2, \dots, E_n) \downarrow 1 \\ & \equiv \mathbf{fst} (E_1, (E_2, (\dots (E_{n-1}, E_n) \dots))) \\ & \rightarrow E_1 \end{aligned}$$

$$\begin{aligned} & (E_1, E_2, \dots, E_n) \downarrow 2 \\ & \equiv \mathbf{fst} (\mathbf{snd} (E_1, (E_2, (\dots (E_{n-1}, E_n) \dots)))) \\ & \rightarrow \mathbf{fst} (E_2, (\dots (E_{n-1}, E_n) \dots)) \\ & \rightarrow E_2 \end{aligned}$$

В общем случае $(E_1, E_2, \dots, E_n) \downarrow i = E_i$ для $1 \leq i \leq n$.

Тест на ноль, предшествование, вычитание и сравнение

Введенных функций для чисел явно недостаточно, чтобы с удобством использовать λ -исчисление как язык программирования. Необходимо также ввести еще некоторые функции: **pred**, **zero?** (представляющие функции предшествования $n \rightarrow n-1$ и тест для нуля) и др.

Будем искать комбинатор **zero?** в виде $\lambda n. n A B$, где A и B — некоторые, пока еще неизвестные нам λ -термы. Комбинатор должен удовлетворять следующим свойствам:

$$\begin{aligned} \mathbf{zero?} \langle 0 \rangle & \rightarrow \mathbf{true} \\ \mathbf{zero?} \langle n+1 \rangle & \rightarrow \mathbf{false} \end{aligned} \tag{1}$$

Поэтому должно выполняться (так как **zero?** $n \rightarrow n A B$)

$$\begin{aligned} \langle 0 \rangle A B & \rightarrow \mathbf{true} \\ \langle n+1 \rangle A B & \rightarrow \mathbf{false} \end{aligned}$$

Но $\langle 0 \rangle A B \equiv (\lambda ab. b) A B \rightarrow B$. Значит, $B \equiv \mathbf{true}$.

Теперь найдем A . Имеем $\langle n+1 \rangle \equiv \lambda ab. a M_{a,b}$, где $M_{a,b}$ есть одно из выражений b , $(a b)$, $a(a b)$, $a(a(a b))$ и т.п. Поэтому $\langle n+1 \rangle A B \rightarrow A M_{A,B} \rightarrow \mathbf{false}$. Таким образом, комбинатор A , действуя

на любой терм $M_{A,B}$, дает **false**. Следовательно, можно взять постоянную функцию $A \equiv \lambda x. \text{false}$.

Поэтому **zero?** определяем следующим образом:

$\text{zero?} \equiv \lambda n. n (\lambda x. \text{false}) \text{true}$
--

Для доказательства достаточно проверить справедливость (1).

Определим на натуральных числах **усеченное вычитание** (функцию *монус*):

$$a \div b = \begin{cases} a - b, & \text{если } a \geq b; \\ 0, & \text{иначе.} \end{cases}$$

Функция предшествования **pred** моделирует $a \div 1$ и на нумералах Чёрча удовлетворяет соотношениям (для любого натурального числа n)

$$\begin{aligned} \text{pred } \langle 0 \rangle &= \langle 0 \rangle \\ \text{pred } \langle n+1 \rangle &= \langle n \rangle \end{aligned} \tag{2}$$

Функция предшествования **pred** определяется с большим трудом, чем предыдущие функции. Сам Алонсо Чёрч бился несколько месяцев над тем, чтобы определить эту функцию. Чёрч так и не справился с этой задачей и уже уверился в неполноте своего исчисления, но в 1932 г. Стефен Клини, тогда молодой аспирант, нашел решение, и это было его первым математическим результатом.

Трудность в определении функции предшествования состоит в том, что, имея терм $\lambda x. y^n x$, надо избавиться от первой аппликации y в y^n . Клини рассмотрел функцию f , отображающую пару (a, b) в пару $(\text{succ } a, a)$. Можно определить f следующим образом

$$\lambda p. (\text{succ } (\text{fst } p), \text{fst } p)$$

Заметим, для любого λ -терма E имеем

$$\begin{aligned}
&<n+1> f(<0>, E) \\
&\rightarrow_{\beta} f^{n+1}(<0>, E) \\
&\rightarrow (<n+1>, <n>)
\end{aligned}$$

Если взять $E \equiv <0>$, то мы получаем

$$<n> f(<0>, <0>) \rightarrow (<n>, <n \div 1>)$$

и, следовательно, для любого натурального n имеем

$$\mathbf{snd} (<n> f(<0>, <0>)) \rightarrow <n \div 1>$$

Поэтому **pred** определяем следующим образом (заменяя $<n>$ на n и проводя абстракцию относительно n):

$\mathbf{pred} \equiv \lambda n. (\mathbf{snd} (n (\lambda p. (\mathbf{succ} (\mathbf{fst} p), \mathbf{fst} p)) (<0>, <0>)))$

Для доказательства достаточно проверить справедливость (2).

Ту же самую технику можно применить и для создания других теоретико-числовых функций. Например, рассмотрим функцию f , отображающую пару (a, b) в пару $(\mathbf{succ} a, \mathbf{times} a b)$. Легко видеть, что функцию f можно определить как

$$\lambda p. (\mathbf{succ} (\mathbf{fst} p), \mathbf{times} (\mathbf{fst} p) (\mathbf{snd} p))$$

Заметим, что

$$\begin{aligned}
&<n> f(<1>, <1>) \\
&\rightarrow_{\beta} f^n(<1>, <1>) \\
&\rightarrow (<n+1>, <n!>)
\end{aligned}$$

Подставим в выражение $<n> f(<1>, <1>)$ определение функции f , применим комбинатор **snd**, заменим $<n>$ на переменную n и окончательно проведем λ -абстракцию над этой переменной. Получается определение функции, вычисляющей факториал

$$\mathbf{fact} \equiv \lambda n. \mathbf{snd} ((\lambda p. (\mathbf{succ} (\mathbf{fst} p), \mathbf{times} (\mathbf{fst} p) (\mathbf{snd} p))) (<1>, <1>))$$

Имея определение функции предшествования, мы теперь можем определить усеченное вычитание (функцию монус) над нумералами Чёрча. Необходимо только сделать b -кратную итерацию аппликации функции ***pred*** к терму $<a>$, чтобы получить $<a \div b>$:

$$\mathbf{monus} \equiv \lambda ab. (b \mathbf{pred} a)$$

Для доказательства достаточно проверить, что $\mathbf{monus} <a> \rightarrow <a \div b>$.

Равенство двух нумералов Чёрча можно определить через усеченное вычитание:

$$\mathbf{equal?} \equiv \lambda ab. (\mathbf{and} (\mathbf{zero?} (\mathbf{monus} a b)) (\mathbf{zero?} (\mathbf{monus} b a)))$$

Для доказательства надо проверить, что для любых $a, b \in N$, $a \neq b$, выполнено:

$\mathbf{equal?} <a> <a> \rightarrow \mathbf{true}$,
 $\mathbf{equal?} <a> \rightarrow \mathbf{false}$.

Следующие функции сравнения получаются подобным (как и для ***equal?***) образом:

$$\begin{aligned} \mathbf{notEqual?} &\equiv \lambda ab. \mathbf{not} (\mathbf{equal?} a b) \\ \mathbf{lessThanOrEqual?} &\equiv \lambda ab. \mathbf{zero?} (\mathbf{monus} a b) \\ \mathbf{greaterThanOrEqual?} &\equiv \lambda ab. \mathbf{zero?} (\mathbf{monus} b a) \\ \mathbf{lessThan?} &\equiv \lambda ab. \mathbf{not} (\mathbf{greaterThanOrEqual?} a b) \\ \mathbf{greaterThan?} &\equiv \lambda ab. \mathbf{not} (\mathbf{lessThanOrEqual?} a b) \end{aligned}$$

Рекурсивные функции

Использование рекурсивных функций — одна из важнейших особенностей функционального программирования: все итерации заменяются рекурсивными вызовами. На первый взгляд

это невозможно сделать в ламбда-исчислении, поскольку в λ -исчислении все функции анонимны, и мы не можем использовать их имена, чтобы организовать рекурсивный вызов. Как это ни удивительно, но рекурсия в λ -исчислении возможна! Но этот факт, как и существование функции предшествования, был открыт только после значительных усилий.

Ключевой идеей послужило использование так называемого *комбинатора неподвижной точки*. В математике неподвижной точкой для данной функции f называется точка $x_0 \in \text{Domain}(f) \cap \text{Range}(f)$, такая, что $f(x_0) = x_0$. В λ -исчислении имеется подобное определение: *неподвижной точкой λ -терма M* называется λ -терм N , для которого выполнено $M N = N$.

Замкнутый λ -терм Y называется комбинатором неподвижной точки, если для любого терма f выполнено соотношение $f (Y f) = Y f$. Другими словами, комбинатор неподвижной точки, примененный для любой функции f , возвращает неподвижную точку этой функции. Первый такой комбинатор был найден Карри и обычно обозначается Y . Этот комбинатор напоминает парадокс Рассела и поэтому называется «парадоксальным комбинатором». Если мы определим

$$R = \lambda x. \text{not } (x x),$$

то обнаружим, что

$$R R = \text{not } (R R).$$

Таким образом, терм $R R$ служит неподвижной точкой для оператора отрицания.¹¹ Для того чтобы получить общий комбинатор неподвижной точки, мы должны заменить отрицание **not** на произвольный терм f . Поэтому мы определяем:

$Y \equiv \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

Проверим, что Y удовлетворяет требуемым условиям. С одной стороны,

¹¹ Последнее равенство не является противоречием, поскольку $R R$ не является термом, представляющим булевское значение.

$$\begin{aligned}
Y E &\equiv (\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))) E \\
&\rightarrow_{\beta} (\lambda x. E (x x)) (\lambda x. E (x x)) \\
&\rightarrow_{\beta} E ((\lambda x. E (x x)) (\lambda x. E (x x)))
\end{aligned}$$

С другой стороны,

$$\begin{aligned}
E (Y E) &\equiv E ((\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))) E) \\
&\rightarrow_{\beta} E ((\lambda x. E (x x)) (\lambda x. E (x x)))
\end{aligned}$$

Так как оба терма $Y E$ и $E (Y E)$ редуцируются к терму $E ((\lambda x. E (x x)) (\lambda x. E (x x)))$, то $E (Y E) = Y E$. Таким образом, мы получили, что каждое λ -выражение E имеет неподвижную точку $Y E$.

Комбинатор Карри Y представляется в базисе $\{S, K, I\}$ как $Y = S(K(SII))(S(S(KS)K)(K(SIII)))$.

Рассмотрим теперь представление рекурсивных функций в λ -исчислении. В качестве примера попробуем определить λ -выражение **summa**, такое, что

$$\mathbf{summa} \langle m \rangle = \mathbf{add} \langle m \rangle (\mathbf{add} \langle m-1 \rangle (\dots (\mathbf{add} \langle 1 \rangle \langle 0 \rangle) \dots))$$

Нетрудно увидеть, что **summa** должно удовлетворять уравнению:

$$\mathbf{summa} \langle m \rangle = \mathbf{If} (\mathbf{zero?} \langle m \rangle) \langle 0 \rangle (\mathbf{add} \langle m \rangle (\mathbf{summa} (\mathbf{pred} \langle m \rangle)))$$

Пусть это имеет место, тогда, например,

$$\begin{aligned}
\mathbf{summa} \langle 2 \rangle &= \mathbf{If} (\mathbf{zero?} \langle 2 \rangle) \langle 0 \rangle (\mathbf{add} \langle 2 \rangle (\mathbf{summa} (\mathbf{pred} \langle 2 \rangle))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{summa} (\mathbf{pred} \langle 2 \rangle))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{summa} \langle 1 \rangle)) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{If} (\mathbf{zero?} \langle 1 \rangle) \langle 0 \rangle (\mathbf{add} \langle 1 \rangle (\mathbf{summa} (\mathbf{pred} \langle 1 \rangle))))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{add} \langle 1 \rangle (\mathbf{summa} (\mathbf{pred} \langle 1 \rangle)))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{add} \langle 1 \rangle (\mathbf{summa} \langle 0 \rangle))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{add} \langle 1 \rangle (\mathbf{If} (\mathbf{zero?} \langle 0 \rangle) \langle 0 \rangle (\mathbf{add} \langle 0 \rangle (\mathbf{summa} (\mathbf{pred} \langle 0 \rangle)))))) \\
&= (\mathbf{add} \langle 2 \rangle (\mathbf{add} \langle 1 \rangle \langle 0 \rangle)) \\
&= (\mathbf{add} \langle 2 \rangle \langle 1 \rangle) \\
&= \langle 3 \rangle
\end{aligned}$$

Приведенное выше уравнение для **summa** предполагает, что терм **summa** может быть определен как

$$\mathbf{summa} = \lambda m. \mathbf{If} (\mathbf{zero?} \ m) \ <0> (\mathbf{add} \ m \ (\mathbf{summa} \ (\mathbf{pred} \ m)))$$

Но такое определение невозможно в λ -исчислении, поскольку определяемый терм не может присутствовать в правой части определения. И здесь на помощь приходит комбинатор неподвижной точки. Определим вспомогательный терм

$$\mathbf{summaf} = \lambda f m. \mathbf{If} (\mathbf{zero?} \ m) \ <0> (\mathbf{add} \ m \ (f \ (\mathbf{pred} \ m)))$$

и затем определим $\mathbf{summa} = \mathbf{Y} \ \mathbf{summaf}$. Получаем искомое уравнение:

$$\begin{aligned} \mathbf{summa} \ <m> &= (\mathbf{Y} \ \mathbf{summaf}) \ <m> \\ &= \mathbf{summaf} \ (\mathbf{Y} \ \mathbf{summaf}) \ <m> \\ &= (\text{по определению } \mathbf{summa}) \ \mathbf{summaf} \ \mathbf{summa} \ <m> \\ &= (\lambda f m. \mathbf{If} (\mathbf{zero?} \ m) \ <0> (\mathbf{add} \ m \ (f \ (\mathbf{pred} \ m)))) \ \mathbf{summa} \ <m> \\ &= \mathbf{If} (\mathbf{zero?} \ <m>) \ <0> (\mathbf{add} \ <m> \ (\mathbf{summa} \ (\mathbf{pred} \ <m>))) \end{aligned}$$

Уравнение вида $f \ x_1 \dots x_n = E$ называется рекурсивным, если f имеет свободное вхождение в E . Комбинатор $\mathbf{Y}$ обеспечивает общий путь решения такого уравнения. Начиная с уравнения в форме $f \ x_1 \dots x_n = \sim f \sim$, где $\sim f \sim$ есть некоторое λ -выражение, содержащее f . Затем определяем f как

$$f = \mathbf{Y} (\lambda f \ x_1 \dots x_n . \sim f \sim)$$

Проверим, что f удовлетворяет необходимому уравнению:

$$\begin{aligned} f \ x_1 \dots x_n &= (\mathbf{Y} (\lambda f \ x_1 \dots x_n . \sim f \sim)) \ x_1 \dots x_n \\ &= (\text{по свойству } \mathbf{Y}) (\lambda f \ x_1 \dots x_n . \sim f \sim) (\mathbf{Y} (\lambda f \ x_1 \dots x_n . \sim f \sim)) \ x_1 \dots x_n \\ &= (\text{по определению } f) (\lambda f \ x_1 \dots x_n . \sim f \sim) f \ x_1 \dots x_n \\ &= \sim f \sim \end{aligned}$$

Хотя с математической точки зрения использование комбинатора Карри совершенно безукоризненно, но с точки зрения программирования вычислительный переход от выражения $E (Y E)$ к выражению $Y E$ и обратно нельзя осуществить только с помощью β -конверсии. По этой причине более удобен комбинатор неподвижной точки, предложенный Тьюрингом:

$$T \equiv (\lambda x y. y(x x y)) (\lambda x y. y(x x y))$$

Для того чтобы проверить необходимое свойство, обозначим терм $\lambda x y. y(x x y)$ через a . Тогда имеем

$$T f \equiv (\lambda x y. y(x x y)) a f \rightarrow_{\beta} f(a a f) \equiv f(T f)$$

Ранее мы представляли возможность записывать безымянные функции как одно из преимуществ лямбда-исчисления. Рекурсивные функции можно определить, не вводя каких-либо имен. Тем не менее возможность давать имена выражениям часто бывает полезна, поскольку позволяет избежать долгих и утомительных повторений. Простая форма именования может быть введена как еще одна форма «синтаксического сахара» над чистым лямбда-исчислением: выражение¹²

$let\ x = S\ in\ T$

по определению обозначает $(\lambda x. T)S$. Например,

$$\begin{aligned} & (let\ z = \mathbf{add}\ \langle 2 \rangle\ \langle 3 \rangle\ in\ \mathbf{add}\ z\ z) \\ & \equiv (\lambda z. \mathbf{add}\ z\ z) (\mathbf{add}\ \langle 2 \rangle\ \langle 3 \rangle) \\ & = \mathbf{add}\ (\mathbf{add}\ \langle 2 \rangle\ \langle 3 \rangle) (\mathbf{add}\ \langle 2 \rangle\ \langle 3 \rangle) \\ & = \langle 10 \rangle \end{aligned}$$

Вместо префиксной формы с *let* можно ввести постфиксный вариант, который иногда более читаем: $T\ where\ x = S$. Например, можно записать

$\mathbf{add}\ z\ z\ where\ z = \mathbf{add}\ \langle 2 \rangle\ \langle 3 \rangle$

¹² Как в языке Haskell.

Функции с несколькими аргументами

В математических обозначениях применение n -местной функции f к аргументам $x_1 \dots x_n$ записывается как $f(x_1, \dots, x_n)$. Существует два способа представления n -кратной аппликации в λ -исчислении:

- 1) как $(f x_1 \dots x_n)$ или
- 2) как аппликация f к n -ке $(x_1, \dots, x_n)$.

В первом случае f применяется к своим аргументам по очереди. Шейфинкель заметил, что не обязательно вводить функции более чем одной переменной [3]. Действительно, для функции, скажем, от двух переменных $f(x,y)$ мы можем рассмотреть функцию g_x с соотношением $g_x(y) = f(x,y)$, а затем f' с соотношением $f'(x) = g_x$. Отсюда $(f'(x))(y) = f(x,y)$. Позднее Карри [1] переоткрыл это свойство, и поэтому сейчас сведение функций с несколькими переменными только к функциям одного переменного носит название *карринг*. Функции **and**, **or** или **add**, определенные ранее, применяют карринг. Преимущество функций с каррингом заключается в том, что такие функции можно применять частично с меньшим числом аргументов, чем требуется по определению этих функций. Например, **add** <1> есть результат частичного применения функции **add** к <1> и обозначает функцию $n \rightarrow n+1$.

Хотя часто удобно представлять n -местные функции с помощью карринга, полезно также иметь возможность представлять аргументы с помощью единственной n -ки. Например, вместо представления сложения λ -термом **add**, так что выполняется условие

$$\mathbf{add} \langle m \rangle \langle n \rangle = \langle m+n \rangle$$

может быть более удобно представить λ -термом **sum**, для которого выполнено

$$\mathbf{sum} (\langle m \rangle, \langle n \rangle) = \langle m+n \rangle$$

Таким образом, n -местные функции можно определить как с каррингом, так и без карринга.

Можно определить два комбинатора, применяя которые к функциям, можно менять наличие карринга у последних.

$$\begin{aligned} \mathbf{curry} &\equiv \lambda f x y. f(x, y) \\ \mathbf{uncurry} &\equiv \lambda f p. f(\mathbf{fst} p) (\mathbf{snd} p) \end{aligned}$$

Имеем для произвольного комбинатора E :

$$\begin{aligned} \mathbf{curry} (\mathbf{uncurry} E) &= E \\ \mathbf{uncurry} (\mathbf{curry} E) &= E \end{aligned}$$

Проверим первое равенство:

$$\begin{aligned} &\mathbf{curry} (\mathbf{uncurry} E) \\ &\equiv (\lambda f x y. f(x, y)) (\mathbf{uncurry} E) \\ &= \lambda x y. ((\mathbf{uncurry} E) (x, y)) \\ &\equiv \lambda x y. ((\lambda f p. f(\mathbf{fst} p) (\mathbf{snd} p)) E (x, y)) \\ &= \lambda x y. (E (\mathbf{fst} (x, y)) (\mathbf{snd} (x, y))) \\ &= \lambda x y. E x y \\ &\rightarrow_{\eta} E \end{aligned}$$

Аналогично доказывается второе равенство. Нетрудно увидеть, что

$$\begin{aligned} \mathbf{sum} &= \mathbf{uncurry} \mathbf{add} \\ \mathbf{add} &= \mathbf{curry} \mathbf{sum} \end{aligned}$$

Представление вычислимых функций

Один из подходов к формализации понятия алгоритма принадлежит Гёделю и Клини (1936). Основная идея Гёделя состояла в том, чтобы получить все вычислимые функции из существенно ограниченного множества базисных функций с помощью простейших алгоритмических средств.

Пусть $\mathbf{N}$ обозначает множество натуральных чисел $\{0, 1, 2, \dots\}$. Объекты, которые мы будем рассматривать, будут функциями с областью определения $D_f \subseteq \mathbf{N}^k$ (k — целое положительное число) и с областью значений $R_f \subseteq \mathbf{N}$. Такие функции будем называть *k-местными частичными*. Слово «частичная» должно напомнить о том, что функция определена на подмножестве $\mathbf{N}^k$

(конечно, в частном случае может быть $D_f = \mathbf{N}^k$, тогда функция называется *всюду определенной*).

Множество исходных функций таково ($\mathbf{x} \in \mathbf{N}^k$):

- постоянная функция $0(\mathbf{x}) = 0$;
- одноместная функция следования $s(x) = x+1$;
- функции проекций pr_i , $1 \leq i \leq k$, $pr_i(\mathbf{x}) = x_i$.

Нетривиальные вычислительные функции можно получать с помощью композиции (суперпозиции) уже имеющихся функций. Этот способ явно алгоритмический.

- Оператор *суперпозиции*. Говорят, что k -местная функция $f(x_1, x_2, \dots, x_k)$ получена с помощью суперпозиции из m -местной функции $\varphi(y_1, y_2, \dots, y_m)$ и k -местных функций $g_1(x_1, x_2, \dots, x_k)$, $g_2(x_1, x_2, \dots, x_k)$, ..., $g_m(x_1, x_2, \dots, x_k)$, если $f(x_1, x_2, \dots, x_k) = \varphi(g_1(x_1, x_2, \dots, x_k), g_2(x_1, x_2, \dots, x_k), \dots, g_m(x_1, x_2, \dots, x_k))$.

Второй (несколько более сложный) способ действует так.

- *Примитивная рекурсия*. При $n \geq 0$ из n -местной функции f и $(n+2)$ -местной функции g строится $(n+1)$ -местная функция h по следующей схеме:

$$h(x_1, \dots, x_n, 0) = f(x_1, \dots, x_n),$$

$$h(x_1, \dots, x_n, y+1) = g(x_1, \dots, x_n, y, h(x_1, \dots, x_n, y)).$$

При $n = 0$ получаем (a — константа)

$$h(0) = a;$$

$$h(y+1) = g(y, h(y)).$$

Два упомянутых способа позволяют задать только всюду определенные функции. Частично-определенные функции порождаются с помощью третьего гёделева механизма.

- Оператор *минимизации*. Эта операция ставит в соответствие частичной функции $f: \mathbf{N}^{k+1} \rightarrow \mathbf{N}$ частичную функцию $h: \mathbf{N}^k \rightarrow \mathbf{N}$, которая определяется так:

1. область определения $D_h = \{ \langle x_1, \dots, x_k \rangle \mid \text{существует } x_{k+1} \geq 0, f(x_1, \dots, x_k, x_{k+1}) = 0 \text{ и } \langle x_1, \dots, x_k, y \rangle \in D_f \text{ для всех } y \leq x_{k+1} \}$;
2. $h(x_1, \dots, x_k) =$ наименьшее значение y , при котором $f(x_1, \dots, x_k, y) = 0$.

Оператор минимизации обозначается так $h(x_1, \dots, x_k) = \mu y [f(x_1, \dots, x_k, y) = 0]$. Очевидно, что даже если f всюду определено, но нигде

не обращается в 0, то $\mu y[f(x_1, \dots, x_k, y) = 0]$ нигде не определено. Естественный путь вычисления $h(x_1, \dots, x_k)$ состоит в подсчете значения $f(x_1, \dots, x_k, y)$ последовательно для $y = 0, 1, 2, \dots$ до тех пор, пока не найдется y , обращающее $f(x_1, \dots, x_k, y)$ в 0. Этот алгоритм не остановится, если $f(x_1, \dots, x_k, y)$ нигде не обращается в 0.

Все функции, которые можно получить из базисных функций за конечное число шагов только с помощью трех указанных механизмов, называются *частично-рекурсивными*. Если функция получается всюду определенная, то тогда она называется *общерекурсивной*. Если функция получена без механизма минимизации, то в этом случае она называется *примитивно-рекурсивной*.

После того как Чёрч ввел λ -исчисление, Клини доказал, что любую частично-рекурсивную функцию можно представить в виде λ -терма.

Говорят, что частичная функция φ с k аргументами λ -определима термом M , когда

- $M\langle n_1 \rangle \langle n_2 \rangle \dots \langle n_k \rangle = \langle \varphi(n_1, n_2, \dots, n_k) \rangle$, если значение $\varphi(n_1, n_2, \dots, n_k)$ определено,
- $M\langle n_1 \rangle \langle n_2 \rangle \dots \langle n_k \rangle$ не имеет нормальной формы, если $\varphi(n_1, n_2, \dots, n_k)$ не определено.

Теорема Клини [5, с. 189]. Частичная функция является частично-рекурсивной тогда и только тогда, когда она λ -определима.

Таким образом, мы получаем, что множество λ -определимых функций также является формализацией интуитивного понятия алгоритма. Это утверждение является содержанием известного *тезиса Черча* (в узком смысле) [7].

Типовое ламбда-исчисление

Типовое ламбда-исчисление необходимо потому, что, во-первых, функция не является полностью определенной, пока не указаны ее область определения (множество всех допустимых входных данных) и область значений (множество всех возможных выходов), и, во-вторых, структура терма, представляющего функцию, должна нести информацию о ее области определения и области значений.

Чтобы реализовать это на практике, надо определить некоторые выражения, которые называются типами и предназначены для обозначения множеств. Тип терма может быть определен с использованием *типовых переменных*, которые будут обозначаться маленькими греческими буквами. В наших обозначениях предполагается, что типовые переменные неявно квантифицированы квантором всеобщности, т.е. если имя типа содержит типовую переменную, то это означает, что вместо данной переменной может присутствовать любой тип.

Мы можем определить тип произвольного лямбда-терма следующим образом.

- Переменная, скажем x , может иметь любой тип.
- Если переменная x имеет тип α , и выражение E имеет тип β , то $\lambda x.E$ имеет тип, который обозначается как $\alpha \rightarrow \beta$; т.е. это выражение можно рассматривать как функцию, которая берет произвольное значение типа α и возвращает результат типа β .
- Рассмотрим выражение $(E F)$. Если F имеет тип α , то E должно иметь тип $\alpha \rightarrow \beta$ (т.е. терм E должен быть функцией, которая требует аргумент типа α) и, следовательно, выражение $(E F)$ должно иметь тип β .

Например, рассмотрим комбинатор $K \equiv \lambda a.\lambda b.a$. Предположим, что a имеет тип α и b имеет тип β . Тогда $\lambda b.a$ имеет тип $\beta \rightarrow \alpha$, и $\lambda a.\lambda b.a$ имеет тип $\alpha \rightarrow (\beta \rightarrow \alpha)$. Мы принимаем соглашение, что операция $\rightarrow$ право ассоциативна, Так что последний тип может быть записан и в виде $\alpha \rightarrow \beta \rightarrow \alpha$. Это означает, что комбинатор K берет аргумент типа α и возвращает функцию, которая для любого аргумента типа β возвращает величину типа α .

Разберем теперь более сложный пример: $S \equiv \lambda abc.(ac)(bc)$. Пусть переменная c имеет тип γ . Рассматривая выражение (bc) , заключаем, что для некоторого типа α терм b имеет тип $\gamma \rightarrow \alpha$, а (bc) имеет тип α . Рассматривая выражение (ac) , заключаем, что для некоторого типа δ терм a имеет тип $\gamma \rightarrow \delta$, а (ac) имеет тип δ . Теперь рассмотрим терм $(ac)(bc)$; (bc) имеет тип α , следовательно, для некоторого типа β терм (ac) имеет тип $\alpha \rightarrow \beta$, а весь терм $(ac)(bc)$ имеет тип β . Получили, что (ac) имеет одновременно тип $\alpha \rightarrow \beta$ и тип δ , поэтому имеем равенство типов $\delta = \alpha \rightarrow \beta$, следовательно, тип a есть $\gamma \rightarrow (\alpha \rightarrow \beta)$.

Теперь мы в состоянии определить тип $S \equiv \lambda a.(\lambda b.(\lambda c.(ac)(bc)))$. Тип $(ac)(bc)$ есть β , поэтому тип $\lambda c.(ac)(bc)$ имеет тип $\gamma \rightarrow \beta$, поэтому $\lambda b.(\lambda c.(ac)(bc))$ имеет тип $(\gamma \rightarrow \alpha) \rightarrow (\gamma \rightarrow \beta)$ и окончательно S имеет тип $(\gamma \rightarrow (\alpha \rightarrow \beta)) \rightarrow ((\gamma \rightarrow \alpha) \rightarrow (\gamma \rightarrow \beta))$. Если использовать соглашение о скобках для типов, то последний тип можно записать в виде $\gamma \rightarrow (\alpha \rightarrow \beta) \rightarrow (\gamma \rightarrow \alpha \rightarrow (\gamma \rightarrow \beta))$.

Вариант лямбда-исчисления, в котором каждый терм помечен некоторым типом, называется *типовым лямбда-исчислением*. Если множество типов ограничено так, что терм не может применяться сам к себе, то в языке исчезает один из видов незавершающихся вычислений. Большинство функциональных языков, например Haskell, использует типовое лямбда-исчисление как промежуточный код при трансляции на машинный язык.

Расширение лямбда-исчисления

Хотя возможно представлять структуры данных в виде λ -выражений, но этот способ является неэффективным. Например, большинство компьютеров аппаратно поддерживают арифметику, и разумно использовать такие возможности вместо β -конверсии при работе с числами. Математически строгий способ взаимодействия исчисления с компьютерными вычислениями осуществляется с помощью так называемых *δ -правил*. Идея состоит в том, чтобы добавить некоторое множество новых констант и определить правила, называемые δ -правилами, для редуцирования аппликаций, содержащих эти константы. Например, можно добавить натуральные числа и новую константу «+» вместе с δ -правилом:

$$+ \ m \ n \rightarrow_{\delta} \ m+n$$

При добавлении новых констант и правил надо быть уверенным, что остается справедливой теорема Чёрча–Россера [5, стр. 84].

Расширенное лямбда-исчисление, как правило, является типовым. Сначала выбираются некоторые символы для обозначения элементарных типов. Например, пусть *Integer* обозначает множество всех целых чисел, а *Bool* обозначает множество истинностных значений. Тогда константа «+» имеет тип *Integer* $\rightarrow$ *Integer*. Таким образом, в расширенном типовом λ -исчислении типовые выражения строятся из элементарных типов и переменных.

Понятия и конструкции высокого уровня способны сократить решение и повысить его эффективность в башню экспонент раз, тогда как стандартная оптимизация или повышение мощностей техники лишь в линейное число раз.

Н.Н. Непейвода

7 ФУНКЦИИ ВЫСОКОГО ПОРЯДКА

Ламбда-абстракция в Лиспе

Лисп заимствовал из λ -исчисления три основных идеи:

- для определения функций используются λ -термы;
- основное правило вычисления β -редукция;
- определения функций являются полноправными данными.

Лисп использует аппликативный порядок редукций, это влечет за собой то, что аргументы функции всегда вычисляются и значения аргументов передаются в тело функции (передача параметров по значению). Но существуют функции, которые при вызове не вычисляют сразу значения своих аргументов (`quote`, `if`, `cond` и т.д.). Благодаря этому, в частности, можно отличить списки, воспринимаемые как данные, от списков, воспринимаемых как указания к вычислению функций.

В Лиспе ламбда-абстракция имеет вид:

`(lambda <список формальных параметров> <тело функции>)`

Такое ламбда-выражение определяет анонимную функцию. Вызов анонимной функции представляется обычным образом, но вместо имени функции указывается соответствующая ламбда-абстракция. Например, чтобы вычислить $3^2 + 4^2$, можно вызвать ламбда-абстракцию, определяющую анонимную функцию, которая возвращает сумму квадратов двух своих аргументов:

```
> ((lambda (x y) (+ (* x x) (* y y))) 3 4)
```

25

Вычисление лямбда-вызова или применение лямбда-выражения к фактическим параметрам производится в два этапа. Сначала вычисляются значения фактических параметров, и соответствующие формальные параметры связываются с полученными значениями. Этот этап называется связыванием параметров. На следующем этапе с учетом новых связей вычисляется форма, являющаяся телом лямбда-выражения, и полученное значение возвращается в качестве значения лямбда-вызова.

Лямбда-абстракции можно по-разному комбинировать. У лямбда-вызова тело вновь может быть лямбда-вызовом:

```
> ((lambda (y) ((lambda (x) (list x y)) 0)) 1)
(0 1)
```

Или аргумент может быть лямбда-вызовом:

```
> ((lambda (x) (list 2 x)) ((lambda (y) (list y) ) 1))
( 2 (1))
```

Лямбда-выражение — это безымянная функция, которая пропадает тотчас после вычисления значения формы. Ее трудно использовать снова, так как нельзя вызывать по имени, хотя ранее выражение было доступно как списочный объект. Безымянные функции используются как фактические параметры в функциях высших порядков (в чем, собственно, и состоит сила функционального программирования).

Функциональные аргументы и значения

Функции в функциональных языках являются **величинами первого класса** наравне с традиционными видами данных, такими, как числа или списки. Они могут быть результатами вычислений других функций, или параметрами других функций, или, комбинируя имеющие функции, можно конструировать новые функции и т.д. В функциональном программировании имеется возможность оперировать функциями всевозможными способами, без ограничений. В частности, функция сама по себе может рассматриваться как структура данных (или как её часть).

Функциями **первого порядка** называются функции, аргументами для которых служат величины, сами не являющиеся функциями. Функции второго порядка оперируют с функциями первого порядка и т.д. **Функцией высокого порядка** называется функция, не являющаяся функцией первого порядка, другими словами, такая функция имеет в качестве аргумента какую-нибудь функцию.

Различие между понятиями «данные» и «функция» определяется в Лиспе не на основе их структуры, а в зависимости от их использования. Если аргумент используется в функции лишь как объект, участвующий в вычислениях, то мы имеем дело с обыкновенным аргументом, представляющим данные. Если же он используется как средство, определяющее вычисления, т.е. играет в вычислениях, например, роль лямбда-абстракции, которая применяется к другим аргументам, то мы имеем дело с функцией.

Одно и то же выражение может в связи с различными аспектами выступать, с одной стороны, как обыкновенный аргумент, а с другой стороны, как функциональный. Роль в интерпретации выражения зависит от его синтаксической позиции. Приведем пример:

```
> (first '(lambda (x) (list x)))
lambda
> ((lambda (x) (list x)) 'first)
(first)
```

(сравните с позициями лямбда-выражений в операции аппликации).

Далее мы будем использовать понятия функции, вызова функции и значения функции в следующем смысле:

- 1) **функция** сама есть изображение вычислений или определение;
- 2) **вызов функции** есть применение этого изображения;
- 3) **значение функции** есть результат такого применения.

Функция является **функционалом**, если в качестве её аргумента используется лисповский объект типа (1), который интерпретируется как функция (2) в теле функционала. Таким функциональным объектом может быть:

- символьное имя, представляющее определение функции (системная функция или функция, определенная пользователем);
- лямбда-абстракция;
- так называемое замыкание (рассматриваемое позже).

Функция может быть и результатом функции. Такие функции (или функционалы), говорят, имеют **функциональное значение**.

В Xlisp'е предполагается, что в вызове функции на месте имени функции находится символ, определенный с помощью `defun` как имя функции. Переданный в качестве параметра функциональный объект можно использовать лишь через явный вызов применяющих функционалов `funcall` или `apply`.

Если `f` функция первого порядка, то ее определение выглядит так:

- Обыкновенный вызов (`defun f ... (g ...) ...`). Здесь тело функции содержит вызов какой-либо функции `g` (отличной от `f`).
- Рекурсивный вызов (`defun f ... (f ...) ...`).
- Вложенный рекурсивный вызов (`defun f ... (f ... (f ...) ...) ...`).

Примером может служить функция Аккермана.

Если же имеем функцию высокого порядка, то в теле функции для передачи функционального параметра используется `apply` (или `funcall`):

функциональный аргумент (`defun f (... g ...) ... (apply g ...) ...`);
 рекурсивный функциональный аргумент (`defun f (... g ...) ... (apply g ... g ...) ...`).

Применяющие функционалы

Применяющие функционалы позволяют применять функциональный аргумент к другим параметрам. Применяющие функционалы дают возможность интерпретировать и преобразовывать данные в программу и применять ее в вычислениях.

APPLY применяет функцию к списку аргументов:
 (`apply` <функция> <список>).

Если <список> имеет вид (`x1 x2 ... xn`), то вызов `apply` равносильно (`<функция> 'x1 'x2 ... 'xn`).

Предупреждение. В Xlisp'е функциональным аргументом `apply` не должна быть встроенная функция `or` или `and`.

```
> (apply '+ '(2 3))
5
> (apply 'eval '(+ 2 3))
5
> (apply 'apply '(+ (2 3)))
5
> (apply '(lambda (x y) (+ x y)) '(2 3))
5
```

Использование `apply` дает большую гибкость по сравнению с прямым вызовом функции: с помощью одной и той же функции `apply` можно в зависимости от функционального аргумента осуществлять различные вычисления. Например, определим функцию высокого порядка `f` следующим образом:

```
(defun f (g s) (apply g s))
```

Тогда имеем:

```
> (f '+ '(2 3))
5
> (f '* '(1 2 3 4 5))
120
> (f '(lambda (x y) (+ (* x x) (* y y))) '(3 4))
25
```

Функционал **FUNCALL** вызывает функцию с аргументами. Аргументы для вызываемой функции `funcall` принимает не списком, а по отдельности:

`(funcall fn x1 x2 ... xn)` эквивалентно вызову `(fn x1 x2 ... xn)`.

Так же как и для `apply`, функция `fn` не должна быть встроенной функцией `or` или `and`.

Примеры:

```
> (funcall '+ 2 3)
5
> (funcall 'eval '(+ 2 3))
5
> (funcall '(lambda (x y) (+ x y)) 2 3)
5
> (funcall 'list 'apply '+ '(2 3))
5
> (apply (quote +) (quote (2 3)))
5
```

Можно определить функцию высокого порядка f следующим образом:

```
(defun f (g x y) (funcall g x y))
```

Тогда имеем:

```
> (f '+ 2 3)
5
> (f '* 2 3)
6
> (f '(lambda (x y) (+ (* x x) (* y y))) 3 4)
25
```

В отличие от примеров с `apply` теперь у функции f функциональный аргумент g только двуместный:

```
> (f '* 1 2 3 4 5)
error: too many arguments
```

Пример функции с двумя функциональными аргументами:

```
(defun do (x f1 f2)
  (funcall (if (< x 0) f1 f2) x))

> (do -1 '- '(lambda (x) (* x x x)))
1
> (do 4 '- '(lambda (x) (* x x x)))
64
```

Отображение и фильтрация списка

Рассмотрим две наиболее полезные операции над списками:

- отображение одного списка в другой путем преобразование каждого элемента исходного списка по какому-то правилу;
- выделение из списка подсписка элементов, обладающих заданным свойством (фильтрация списка).

Эти операции решаются следующими функциями высокого порядка:

```
(defun map (f s)
  (if (null s) nil (cons (funcall f (first s)) (map f (rest s))))
)
```

```
(defun filter (f s)
  (cond ((null s) nil)
        ((funcall f (first s)) (cons (first s) (filter f (rest s))))
        (t (filter f (rest s))))
```

В XLisp'е имеется встроенная функция `mapcar`, которая является обобщением функции `map`: функциональный аргумент может быть любой местности (а не только одноместный) и вместо одного исходного списка используется соответствующее количество списков. Если функциональный аргумент есть n -местная функция f и имеются n списков $(a_{11} a_{12} \dots a_{1k})$, $(a_{21} a_{22} \dots a_{2m})$, $\dots (a_{n1} a_{n2} \dots a_{np})$, вообще говоря, различной длины, то вызов

```
(mapcar f (a11 a12 ... a1k) (a21 a22 ... a2m) ... (an1 an2 ... anp))
```

дает список

```
((f a11 a21 ... an1) (f a12 a22 ... an2) ... (f a1q a2q ... anq)),
```

где $q = \min(k, m, \dots p)$ — длина самого короткого списка.

Примеры:

```
> (mapcar '1+ '(1 2 3 4 5))
(2 3 4 5 6)
> (mapcar '(lambda (x) (* 2 (1+ x))) '(1 2 3))
(4 6 8)
> (mapcar '+ '(1 2 3) '(5 6 7 8 9))
(6 8 10)
> (mapcar 'list '(+ + +) '(1 2 3 4 5) '(a b c d))
((+ 1 A) (+ 2 B) (+ 3 C))
> (mapcar 'numberp '(a 2 c 4))
(NIL T NIL T)
> (mapcar '(lambda (x) (zerop (mod x 2))) '(1 2 3))
(NIL T NIL)
> (mapcar 'max '(1 2 3 4) '(1 4 9 16) '(11 1 2 10))
(11 4 9 16)
```

В XLisp'е имеется встроенная функция `remove-if-not`, которая является некоторым обобщением функции `filter`. Пример вызова:

```
> (remove-if-not 'numberp '(1 a 2 b 3 c))
(1 2 3)
```

Встроенная функция `(remove-if p s)`, наоборот, оставляет в списке `s` только те элементы, которые не удовлетворяют предикату `p`.

```
> (remove-if 'numberp '(1 a 2 b 3 c))
(a b c)
```

Замыкания

Чтобы на этапе вызова функционала можно было отличить функциональный аргумент от обычного, функциональный аргумент помечают с помощью предотвращающей вычисления формы **FUNCTION**. Примеры:

```
(function (lambda (x) (list x y)))
(function first)
```

Форму `function` называют **функциональной блокировкой**. Вместо `(function f)` можно кратко писать `#'f`.

Если нужно передать функции данные в том виде, как они записаны, то используется обычная форма `quote`. Формы `quote` достаточно и для передачи имени функции или лямбда-абстракции, если в ней не используются свободные переменные. В системных функциях свободных переменных нет, поэтому все следующие формы имеют одинаковое значение:

```
(+ 2 3)
(funcall '+ 2 3)
(funcall (function +) 2 3)
(funcall #' + 2 3)
((lambda (x y) (+ x y)) 2 3)
(funcall #'(lambda (x y) (+ x y)) 2 3)
```

Замыкание — это функция и контекст ее определения. Часто бывает полезным и необходимым, чтобы функция для продолжения вычислений могла запомнить связи и состояние более раннего контекста. Это достигается с помощью замыкания — функционального объекта, в котором вместе с самим описанием вычислений сохраняется контекст момента определения функции, защищенный от более позднего контекста вызова. В `XLisp`'е замыкание создается формой `function`, например `#'(lambda (x) (+ x y))`. В замыкание из контекста определения функции включаются лишь связи свободных переменных функции (в приведенном примере глобальное значение `y`). Если в замыкаемой функции нет свободных переменных, то форма `function` ничем не отличается от формы `quote`.

В следующем примере результат вызова функции `inc` — снова функция:

```
(defun inc (n) #'(lambda (z) (+ z n)))
```

Значение вызова `(inc 4)` дает замыкание `#'(lambda (z) (+ z n))`, в котором свободная переменная `n` получила значение 4. Поэтому

сейчас (inc 4) можно рассматривать как функцию, которая увеличивает свой аргумент на 4. Действительно,

```
> (funcall (inc 4) 5)
9
```

Еще пример. Определим функцию (f x s), которая возвращает из списка чисел s элементы, меньшие x.

```
(defun f (x s) (remove-if-not #'(lambda (y) (> x y)) s))
```

```
> (f 3 '(1 2 3 4 5))
(1 2)
```

Абстрактный подход

Абстракция отображения (отображение данных) основана на параметризации лямбда-выражений. Можно с помощью механизма параметризации абстрагировать сами функции, т.е. параметризация касается первого элемента вызова функции, — это называется **абстракцией вычислений**.

Обобщение функций, имеющих одинаковый вид. Рассмотрим обобщение с помощью функции высокого порядка функций, определения которых с точки зрения абстракции вычислений сходны по строению, но которые довольно различны с точки зрения производимых действий.

Рассмотрим задачи со списками чисел. Определим рекурсивную функцию sum, возвращающую сумму элементов списка. Функция должна быть определена для двух видов аргумента: пустого списка и списка, содержащего элементы. Очевидно, сумма пустого множества чисел равна нулю. В общем случае, чтобы вычислить сумму элементов списка, мы вычисляем сумму хвоста списка и прибавляем значение первого элемента:

```
(defun sum (s)
  (if (null s) 0 (+ (sum (rest s)) (first s)))
)
```

Аналогично выглядит определение функции, вычисляющей произведение элементов списка.

```
(defun product (s)
  +----+ +----+
  (if (null s) | 1 | (| * | (product (rest s)) (first s)))
  +----+ +----+
)
```

Изучая эти определения, мы обнаруживаем, что только части, ограниченные рамками, связаны именно с вычислением суммы (произведения). Это означает, что вычисление суммы (произведения) может быть модуляризовано с помощью «склеивания» вместе общего рекурсивного образца и частей, ограниченных прямоугольниками. Если выражения, помещенные в рамки, сделать параметрами, то можно написать функцию высокого порядка `fold`¹³.

```
(defun fold (g s a)
  +----+ +-----+
  (if (null s) | a | (| funcall g | (fold g (rest s) a) (first s)))
  +----+ +-----+
)
```

Теперь функции `sum` и `product` могут быть записаны в виде:

```
(defun sum (s) (fold '+ s 0))
(defun product (s) (fold '* s 1))
```

```
> (sum '(1 2 3))
6
> (product '(4 5 6))
120
```

Чтобы увидеть в каком порядке функциональный параметр `g` применяется к элементам списка `s`, достаточно вызвать

¹³ Это имя для подобной функции (`fold` — «свертка») используется в современных функциональных языках (но не в Лиспе).

```
> (fold 'list '(a b c d) nil)
((((NIL D) C) B) A)
```

В XLisp'е существует встроенная функция `reduce`, аналогичная `fold`. Вызов функции имеет вид:

```
(reduce <двуместная функция> <список> :initial-value
<начальное значение>)
```

Вызов функции `reduce` использует так называемые «ключевые параметры». Все такие параметры имеют имя, начинающееся с двоеточия «:», после имени следует значение параметра. Ключевые параметры могут явно не указываться, тогда в качестве их значений берутся некоторые значения по умолчанию¹⁴.

Следующий вызов показывает, что порядок обработки элементов функциями `reduce` и `fold` различается.

```
> (reduce 'list '(a b c d) :initial-value nil)
((((NIL A) B) C) D)
```

В общем случае вызов

```
(fold <функция> <список> <начальное значение>)
```

эквивалентен вызову

```
(reduce <функция> (reverse <список>) :initial-value <начальное
значение>)
```

Если начальное значение явно не задавать, то `reduce` использует по умолчанию первый элемент списка.

```
> (reduce 'list '(a b c d))
(((A B) C) D)
```

Сумму и произведение элементов списка можно вычислить с помощью функции `reduce`:

¹⁴ На самом деле вызов `reduce` может содержать еще два ключевых параметра, но они мало используются.

```
> (reduce '+ '(1 2 3))
6
> (reduce '* '(4 5 6))
120
```

Если вернуться к задаче нахождения суммы и произведения элементов списка чисел (результат выдается в виде списка из двух значений), то с использованием `reduce` получаем короткую программу:

```
(defun sumpr (s)
  (list (reduce '+ s)
        (reduce '* s)))
```

Еще пример:

```
> (reduce '(lambda (x y) (list x '+ y)) '(a b c d))
(((A+B)+C)+D)
```

Определим логические функции — конъюнкцию и дизъюнкцию¹⁵:

```
(defun and (x y)
  (if x y nil))
(defun or (x y)
  (if x t y))
```

Теперь с помощью `reduce` можно определить функциональные предикаты `forsome` и `forall`. Функциональный предикат `(forsome p s)` истинен тогда и только тогда, когда `(p x)` — истина для некоторого элемента `x` списка `s`. Функциональный предикат `(forall p s)` истинен тогда и только тогда, когда `(p x)` — истина для всех элементов списка `s`.

```
(defun forsome (p s)
  (reduce 'or (mapcar p s) :initial-value nil))
```

¹⁵ Встроенные функции `and` и `or` для этой цели не годятся.

```
(defun forall (p s)
  (reduce 'and (mapcar p s) initial-value t))
```

```
> (forall 'zerop '(0 8 7))
NIL
> (forsome 'zerop '(0 8 7))
T
```

С помощью `reduce` и `fold` можно представить большинство функций со списками.

Соединение списков:

```
(defun append (x y) (fold '(lambda (a b) (cons b a)) x y))

> (append '(1 2 3) '(4 5 6))
(1 2 3 4 5 6)
```

Функция слияния двух упорядоченных списков в один упорядоченный:

```
(defun merge (x y) (fold '(lambda (a b) (insert b a)) x y))

(defun insert (a s)
  (cond ((null s) (list a))
        ((< a (first s)) (cons a s))
        (t (cons (first s) (insert a (rest s)))))
)
```

```
> (merge '(1 3 5) '(2 3 4 6))
(1 2 3 3 4 5 6)
```

Впрочем, `merge` можно определить и с помощью `reduce`.

```
(defun merge (x y) (reduce '(lambda (a b) (insert b a)) x :initial-value
y))
```

Длина списка:

```
(defun length (s) (reduce '(lambda (x y) (1+ x)) s :initial-value 0))
```

Функция map:

```
(defun map (f s) (fold #'(lambda (x y) (cons (funcall f y) x)) s nil))
```

```
> (map '(lambda (x) (* x x)) '(1 2 3))
(1 4 9)
```

Принадлежит ли элемент списку?

```
(defun member (x s) (reduce #'(lambda (a b) (or (equal b x) a)) s
:initial-value nil))
```

```
> (member 5 '(1 5 10))
T
```

Число вхождений элемента в список:

```
(defun count (x s) (reduce #'(lambda (a b) (if (equal b x) (1+ a) a)) s
:initial-value 0))
```

```
> (count 'a '(a b a c a))
3
```

Обратите внимание: использование функций высокого порядка, таких, как map, filter, fold и reduce, для определения новых функций наряду с более краткой записью позволяет убрать явную рекурсию из тела функций.

Абстрагирование вычислений с помощью функций с функциональными значениями. Композицию двух функций можно определить так:

```
(defun comp (f g)
  #'(lambda (x) (funcall f (funcall g x))))
)
```

В результате конкретного вызова функции `comp` возвращается одноместная функция (в Лиспе она трактуется как замыкание). Мы можем теперь строить пирамиды новых функционалов. Например, определим функцию `twice`:

```
(defun twice (f) (comp f f))
```

Примеры вызовов:

```
> (funcall (twice '(lambda (x) (* x x x))) 2)
512
> (defun inc3 (x) (+ x 3))
(INC3)
> (funcall (twice 'inc3) 10)
16
> (funcall (twice (twice '1+)) 1)
5
```

Определим n -ую степень функции $f^n(x) \equiv f(f(f(\dots f(x))))$ (f применяется n раз).

```
(defun n-comp (f n)
  (if (zerop n) '(lambda (x) x)
      (comp f (n-comp f (1- n)))))
```

Предположим, что в Лиспе нет встроенных операций сложения, умножения и возведения в степень. Вот как с помощью n -кратной композиции можно было бы эти функции определить. Сложение двух натуральных чисел:

```
(defun summa (n m)
  (funcall (n-comp #'1+ n) m))

> (summa 4 9)
13
```

Умножение двух натуральных чисел:

```
(defun times (n m)
  (funcall (n-comp #'(lambda (x) (summa x n)) m) 0))

> (times 5 6)
30
```

Возведение натурального числа в натуральную степень:

```
(defun power (n m)
  (funcall (n-comp #'(lambda (x) (times x n)) m) 1))

> (power 2 5)
32
```

Автофункции

В функциональном программировании рассматриваются автоаппликативные и авторепликативные функционалы как функции, которые получают или возвращают в качестве результата самих себя, или, точнее, копии самих себя. Конечно, не очевидно, что такие функции вообще существуют. Однако никаких принципиальных препятствий для их существования и определения не существует. В Лиспе их можно определить и использовать довольно просто.

Для начала рассмотрим автоаппликативный вариант факториала:

```
(defun factorial (f n)
  (if (zerop n) 1 (* n (funcall f f (1- n)))))
```

Для вычисления $10!$ необходимо функционалу *factorial* в его вызове в качестве функционального аргумента передать его собственное определение.

```
> (factorial 'factorial 10)
3628800
```


Соответственно тому, как мы можем через функциональный аргумент определить автоаппликативную рекурсивную функцию, мы можем определить и возвращающую себя рекурсивно функцию с функциональным значением.

Возьмем в качестве примера простейшую возможную функцию, которая возвращает себя в качестве результата. Это лямбда-выражение, примененное к такому же лямбда-выражению с формой `quote`, т.е. это лямбда-вызов:

```
((lambda (x) (list x (list 'quote x))) ' (lambda (x) (list x (list 'quote x))))
```

Форма является одновременно автоаппликативной! Она содержит изображение самой себя, но не целиком, поскольку такое изображение было бы бесконечно длинным. Изображение получается ограниченным благодаря механизму блокировки и лямбда-механизму, а также благодаря автоаппликации.

Для проверки присвоим форму переменной `self`:

```
(setq self
  '((lambda (x) (list x (list 'quote x))) ' (lambda (x) (list x (list 'quote x)))))
```

```
> (eval self)
((lambda (x) (list x (list 'quote x))) ' (lambda (x) (list x (list 'quote x)))))
```

Форму можно вычислять вновь и вновь, и все равно получается тот же результат: само определение функции (его копия). Кроме того, значение функции совпадает со значением аргумента ее вызова. Такое значение, как вы знаете, называется неподвижной точкой функции.

Форма `self` аналогична известному комбинатору $(\lambda x. x \ x) (\lambda x. x \ x)$: β -редукция терма $(\lambda x. x \ x) (\lambda x. x \ x)$ приводит к бесконечному циклу

$$(\lambda x. x \ x) (\lambda x. x \ x) \rightarrow (\lambda x. x \ x) (\lambda x. x \ x) \rightarrow \dots$$

Автоаппликация как форма рекурсии с теоретической точки зрения очень интересна, но в практическом программировании не находит особого применения. С нею связаны теоретические вопросы и проблемы, которые во многом еще не изучены. В формализмах и постановке проблем, которые основаны на похожих на автоаппликацию ссылках на себя (self-reference), очень легко приходят к логическим парадоксам, подобным известному из учебников по теории множеств парадоксу Рассела. Их разрешение предполагает учет типов объектов и кратности ссылок.

Однако автоаппликация может открыть новый подход к программированию. Возможными применениями могли бы быть, например, системы, сохраняющие неизменными определенные свойства, хотя какие-то их части изменяются. Такими свойствами могли бы быть наряду с применимостью и репродуцируемостью, например, способность к самоизменениям (self-modification), таким, как приспособляемость, согласованность и обучаемость, самосознание и сознательность и т.д. Самосознание предполагает существование модели мира внутри такой системы, включая саму систему в эту модель.

Автофункции — это новый класс функций, которые отличаются от обыкновенных рекурсивных функций так же, как рекурсивные функции отличаются от нерекурсивных. Чтобы их можно было использовать в программировании столь же удобно, как и рекурсию, нужно использовать механизмы, которые при программировании учитывают инвариантные свойства вычислений таким же образом, как механизмы определения, вызова и вычисления функций учитывают рекурсию.

Я занимался до сих пор решением ряда задач, ибо при изучении наук примеры полезней правил.

И. Ньютон Всеобщая арифметика

8 НЕСКОЛЬКО БОЛЕЕ СЛОЖНЫХ ЗАДАЧ

Задачи, решения которых мы до этого программировали, были довольно просты. Рассмотрим более сложные задачи. Как мы увидим, программы на Лиспе получаются небольшие по размеру.

Задача 1 . Дан список. Преобразовать список, заменив в нем каждую группу стоящих рядом одинаковых элементов одним элементом.

Алгоритм

- Если список содержит не более одного элемента, то результат равен аргументу.
- В противном случае если первые два элемента списка совпадают, то функция рекурсивно применяется к хвосту списка.
- Иначе функция применяется к хвосту списка и к полученному результату добавляется в голову первый элемент.

```
(defun f (xs)
  (cond
    ((null xs) nil)
    ((null (rest xs)) xs)
    ((eql (first xs) (second xs)) (f (rest xs)))
    (t (cons (first xs) (f (rest xs)))))
  )
)
```

Задача 2. Дан список. Определить наибольшее число нулей, подряд расположенных в этом списке.

Для решения используется локальная функция $(h\ n\ m\ y)$, где y — исходный список (по нему осуществляется рекурсия), n — накапливающий параметр, в котором хранится текущее (при движении по списку) максимальное количество подряд идущих нулей. Параметр m — счетчик, в котором хранится текущее число нулей после последнего ненулевого элемента. Функция h вызывается в теле основной функции со значениями $n=0$ и $m=0$.

Алгоритм

- Если голова списка — нулевой элемент, то функция h вызывается для хвоста, причем m увеличивается на 1.
- Если же голова списка — не нулевой элемент, то функция h вызывается для хвоста, где m равно 0, а n при необходимости корректируется (если m больше n , то n становится равным m).

```
(defun f (xs)
  (labels ((h (n m y)
    (cond
      ((null y) (max n m))
      ((zerop (first y)) (h n (1+ m) (rest y)))
      (t (h (max n m) 0 (rest y)))
    )
  ))
  (h 0 0 xs)
)
```

Задача 3. Дан список. Каждый подсписок длиной $n > 1$, состоящий из одинаковых элементов, заменить на подсписок из тех же элементов в количестве $2*n$ штук. Например, дан список $(k\ j\ h\ h\ s\ d\ a\ a\ a)$. Необходимо получить и вывести на экран список: $(k\ j\ h\ h\ h\ h\ s\ d\ a\ a\ a\ a\ a\ a)$.

Алгоритм.

1) Исходный список разбивается на список списков, в котором внутренние списки состоят только из одинаковых элементов.

2) Теперь обрабатываются подписки в полученном результате: подписки, состоящие из одного элемента, не меняются, а количество элементов в других подписках удваивается.

3) Полученный на предыдущем этапе список списков преобразуется в одноуровневый список.

```
; главная функция
(defun f (xs)
  (concat (mapcar 'p (p1 (reverse xs) nil)))
)
```

```
; следующая функция применяется только к тем спискам, у кото-
рых все элементы
; одинаковые; если список x состоит из одного элемента, то ре-
зультат функции
; равен аргументу, иначе результат равен аргументу с удвоенным
числом элементов.
```

```
(defun p (x)
  (if (= 1 (length x)) x (append x x))
)
```

```
; Список s разбивается на список списков, в котором внутренние
списки состоят
; только из одинаковых элементов (элементы исходного списка
располагаются в
; обратном порядке). Параметр w используется для накопления
результата, исходное его значение — пустой список.
```

```
(defun p1 (s w)
  (cond
    ((null s) w)
    ((and (null (rest w)) (null (first w)))
     (p1 (rest s) (list (list (first s)))))
    ((eql (first s) (caar w))
     (p1 (rest s) (cons (cons (first s) (first w)) (rest w))))
    (t (p1 (rest s) (cons (list (first s)) w))))
  )
)
```

```
> (p1 '(a a b c c c d) nil)
((d) (c c c) (b) (a a))
```

; Список списков преобразуется в одноуровневый список

```
(defun concat (s)
  (if (null s) nil (append (first s) (concat (rest s)))))
)
```

```
> (concat '((a b) (c d e) (f)))
(a b c d e f)
```

Задача 4. Напишите функцию $(f\ k\ n)$, которая возвращает список всех последовательностей (списков) длиной k из чисел $1, 2, \dots, n$. Пример:

```
> (f 3 2)
(1 1) (2 2) (3 3) (1 2) (1 3) (2 1) (3 1) (2 3) (3 2))
(элементы списка могут быть в другом порядке) .
```

Будем использовать рекурсию по k .

- Базовый случай $k=1$ — результат равен списку вида: $((n) \dots (2) (1))$.
- Если $k>1$, то получаем промежуточный результат — рекурсивный вызов f для $k-1$. Теперь каждый подсписок (его длина $k-1$) в промежуточном результате преобразуем в список подсписков длиной k , путем добавления по одному числу из чисел $1, 2, \dots, n$.
- И окончательно список списков подсписков преобразуем в двухуровневый список.

```
(defun f (k n)
  (if (= k 1) (mapcar 'list (make-list n))
      (concat (mapcar #'(lambda (xs) (g xs n)) (f (1- k) n) ))))
)
```

```
(defun g (s n)
  (mapcar #'(lambda (m) (cons m s)) (make-list n))
)
```

```
; функция для данного n создает список (1 2 3 ... n).
(defun make-list (n)
  (if (zerop n) nil
      (cons n (make-list (1- n)))))
)
```

Примеры вызовов функции g:

```
> (g '(1) 3)
((3 1) (2 1) (1 1))
> (g '(2 2) 3)
((3 2 2) (2 2 2) (1 2 2))
```

Задача 5. Напишите функцию (f n), которая возвращает список всех последовательностей (списков) из n нулей, единиц и двоек, в которых никакая цифра не повторяется два раза подряд (нет куска вида X X).

Будем использовать рекурсию по k.

- Базовый случай $n=1$ — результат равен списку: ((0) (1) (2)).
- Если $n>1$, то получаем промежуточный результат — рекурсивный вызов f для $n-1$.
- Теперь каждый подсписок (его длина $n-1$) в промежуточном результате преобразуем в список подсписков длиной n, путем добавления по одному числу из чисел 0,1,2, так чтобы не нарушалось требуемое условие.
- И окончательно список списков подсписков преобразуем в двухуровневый список.

```
(defun f (n)
  (if (= n 1) '((0) (1) (2))
      (concat (mapcar 'q (f (1- n))))))
)
```

```
(defun q (xs) (remove-if-not '(lambda (y) (/= (first y) (second y)))
                             (list (cons 0 xs) (cons 1 xs) (cons 2 xs))))
)
```

```
> (q '(1 0))
((0 1 0) (2 1 0))
```

```
> (f 3)
((0 1 0) (2 1 0) (0 2 0) (1 2 0) (1 0 1) (2 0 1) (0 2 1) (1 2 1) (1 0 2)
(2 0 2) (0 1 2) (2 1 2))
```

Задача 6. Напишите функцию, строящую список всех подмножеств данного множества.¹⁶

Встроенная функция (`union list1 list2`) создает список всех элементов, входящих в список `list1` или в `list2` (объединение множеств):

```
> (union '(1 2 3) '(2 5 7 1))
(7 5 1 2 3)
```

Множество $P(X)$ («множество—степень») всех подмножеств множества X равно объединению двух множеств 1) $P(X')$ (где $X' = X \setminus \{a\}$, a — произвольный элемент из X) и 2) $\bigcup_{Y \in X'} (Y \cup \{a\})$

```
(defun f(s) (if (null s) (list s) (union (mapcar '(lambda (x) (cons (car s)
x)) (f (cdr s))) (f (cdr s)))
)
)
```

Задача 7. Требуется найти в неориентированном графе путь из данной вершины в целевую.

Удобно представлять граф в виде двух списков: список вершин и список ребер. Каждое ребро, в свою очередь, есть список из двух вершин. Например,

```
(setq v '(1 2 3 4 5 6))
(setq r '((1 2) (2 3) (3 1) (1 4) (4 5) (5 6) (6 1)))
```

¹⁶ Данное решение принадлежит Клыкову Виктору, студенту ТУСУРа кафедры АСУ группы 437-1.

Целевую вершину можно также представить с помощью однократного присваивания:

```
(setq end 6)
```

Пути в графе будем представлять в виде списка пройденных вершин. Нам понадобится запрограммировать функцию (nbg x p). Эта функция будет выдавать список всех соседей вершины x, не входящих в путь p.

Алгоритм:

- просматривая все ребра графа, выбираем вершины, соседние для x (если какое-то ребро не инцидентно вершине x, то в создаваемый список добавляем nil);
- удаляем из полученного списка nil и элементы, принадлежащие пути p.

```
((defun ngb (x p) (remove-if #'(lambda (y) (or (null y) (member y p)))
                             (mapcar
                              #'(lambda (s)
                                   (cond
                                    ((eql x (first s)) (second s))
                                    ((eql x (second s)) (first s))
                                    (t nil)
                                   )
                              )
                              r)
                             )
  )

> (ngb 1 r)
(2 3 4 6)
```

Будем использовать алгоритм поиска в глубину. Для этого главную функцию определим так:

```
(defun path (x) (depth x (ngb x nil) nil))
```

Функция `(depth x y p)` выдает путь (список вершин), начиная из вершины `x`, `p` — накапливающий параметр — содержит пройденный путь, `y` — список вершин, соседних для `x` и не входящих в `p` (`y ≡ (nbg x p)`).

Алгоритм:

- Если `x` — целевая вершина, то получаем результат, добавляя к пути `p` вершину `x`.
- Если список `y` вершин-соседей пуст, то результат равен `ni`.
- Запускаем рекурсивно `depth`, накапливая параметр `p` и меняя параметры `x` и `y`.
- Если предыдущая ветвь заканчивается неудачей (результат `nil`), то вызываем функцию `depth` для хвоста списка `y`.

```
(defun depth (x y p)
  (cond
    ((eql end x) (cons x p))
    ((null y) nil)
    ((depth (first y) (nbg (first y) (cons x p)) (cons x p)))
    (t (depth x (rest y) p))
  )
)
```

```
> (setq end 6)
```

```
6
```

```
> (path 1 r)
```

```
(6 5 4 1)
```

Отметим особенности: 1) алгоритм выдает путь в обратном порядке прохождения вершин; 2) полученный путь не самый короткий — если ребро `(6 1)` в списке ребер поместить раньше, чем ребро `(1 4)`, то найденный путь будет `(6 1)`; 3) список вершин для данного решения задачи излишен.

9 ЗАКЛЮЧЕНИЕ

В российском образовании в области информатики, по сравнению с американской системой образования, придается традиционно большее значение фундаментальному математическому образованию и несколько меньшее значение естественным наукам и профессиональной подготовке. Это связано с тем, что компьютерные науки в России создавались преимущественно математиками, традиционно отдающими предпочтение фундаментальным знаниям. Овладение такими знаниями в наибольшей степени способствует формированию мышления обучающихся. Причина различий здесь в том, что в России компьютерные науки и соответствующие образовательные программы исторически преимущественно математические, а в Америке — инженерные.

Следствием этого является то, что американский стиль обучения ориентирован на изучение процесса набора стандартных решений под типовые ситуации. Российский стиль обучения подразумевает осмысление принципов получения решения и понимания важности развития решения задачи до обобщенного подхода.

Несомненно, функциональный стиль программирования является наиболее фундаментальным из всех парадигм программирования. И овладение им — это дополнительная возможность не потерять преимущества российского образования.

ЛИТЕРАТУРА

1. Curry H. B., Feys R. Combinatory Logic, vol. I, Amsterdam: North Holland Co., 1958.
2. Church A. The Calculi of Lambda Conversion. Princeton University Press, Princeton, 1941.
3. Schönfinkel M. Über die Bausteine der mathematischen Logik. Math. Annalen, 92, 1924, s. 305–316.
4. Turner D. A. A new implementation technique for applicative languages. Software-Practice and Experience, 9, 1979, pp. 31-49.
5. Барендрегт Х. Ламбда-исчисление. Его синтаксис и семантика. — М.: Мир, 1985. — 606 с.
6. Дейкстра Э. Заметки по структурному программированию // У. Дал, Э. Дейкстра, К. Хоор. Структурное программирование. — М.: Мир, 1975. — С. 7–97.
7. Непейвода Н.Н. Прикладная логика: Учебное пособие. — Новосибирск: Изд-во Новосиб. ун-та, 2000. — 521 с.
8. Филд А., Харрисон П. Функциональное программирование. — М.: Мир, 1993. — 637 с.
9. Хьювенен Э., Сеппянен Й. Мир Лиспа. В 2-х т. — М.: Мир, 1990.